

Randomized Test – AWS Certified Developer Associate

1. Question

A company has a development team that's heavily relying on AWS CodeBuild, and CodeDeploy. The management would like to further automate its CI/CD process. They requested a system that monitors the status of each code change, from the moment it's committed through to its deployment.

Which of the following AWS services will help you achieve this?

- Amazon CodeGuru
- AWS Elastic Beanstalk
- AWS CodePipeline
- AWS Fault Injection Simulator

Correct

AWS CodePipeline is a fully managed continuous delivery service that helps you automate your release pipelines for fast and reliable application and infrastructure updates. CodePipeline automates the build, test, and deploy phases of your release process every time there is a code change, based on the release model you define. This makes it a good choice for automating your CI/CD process and centrally monitoring application activity.

Moreover, AWS CodePipeline integrates with AWS CloudWatch, which provides a reliable, scalable, and flexible monitoring solution. You can create dashboards in CloudWatch to centrally monitor application activity and manage day-to-day development tasks.

The option that says: **AWS Fault Injection Simulator** is incorrect because this is just a managed service that is commonly used in chaos engineering, and not for application development. It enables you to perform fault injection experiments on your AWS workloads to improve the performance and resiliency of your applications.

The option that says: **Elastic Beanstalk** is incorrect because it is an orchestration service to quickly deploy and manage applications in AWS.

The option that says: **Amazon CodeGuru** is incorrect because this is simply a developer tool that provides intelligent recommendations to improve the quality of your codebase and for identifying an application's most "expensive" lines of code in terms of resource intensiveness, CPU performance, and code efficiency.

References:

<https://docs.aws.amazon.com/codepipeline/latest/userguide/detect-state-changes-cloudwatch-events.html>

<https://aws.amazon.com/codepipeline/>

Check out this AWS CodePipeline Cheat Sheet:

<https://tutorialsdoido.com/aws-codepipeline/>

2. Question

An EBS-backed EC2 instance has been recently reported to contain a malware that could spread to your other instances. To fix this security vulnerability, you will need to attach its root EBS volume to a new EC2 instance which hosts a security program that can scan viruses, worms, Trojan horses, or spyware.

What steps would you take to detach the root volume from the compromised EC2 instance?

- Detach the volume from the AWS Console. AWS takes care of unmounting the volume for you.
- Unmount the volume from the OS and then detach.
- Unmount the volume, stop the instance, and then detach.
- Stop the instance then detach the volume.

Correct

You can detach an Amazon EBS volume from an instance explicitly or by terminating the instance. However, if the instance is running, you must first unmount the volume from the instance.

If an EBS volume is the root device of an instance, you must **stop the instance before you can detach the volume**.

The options that say **unmount the volume from the OS and then detach** and **unmount the volume, stop the instance, and then detach** are both incorrect because you can't unmount the root volume on a running instance.

The option that says: **Detach the volume from the AWS Console. AWS takes care of unmounting the volume for you** is incorrect because unmounting the volume is not managed by AWS.

Reference:

Check out this Amazon EC2 Cheat Sheet:

<https://tutorialsdojo.com/amazon-elastic-compute-cloud-amazon-ec2/>

3. Question

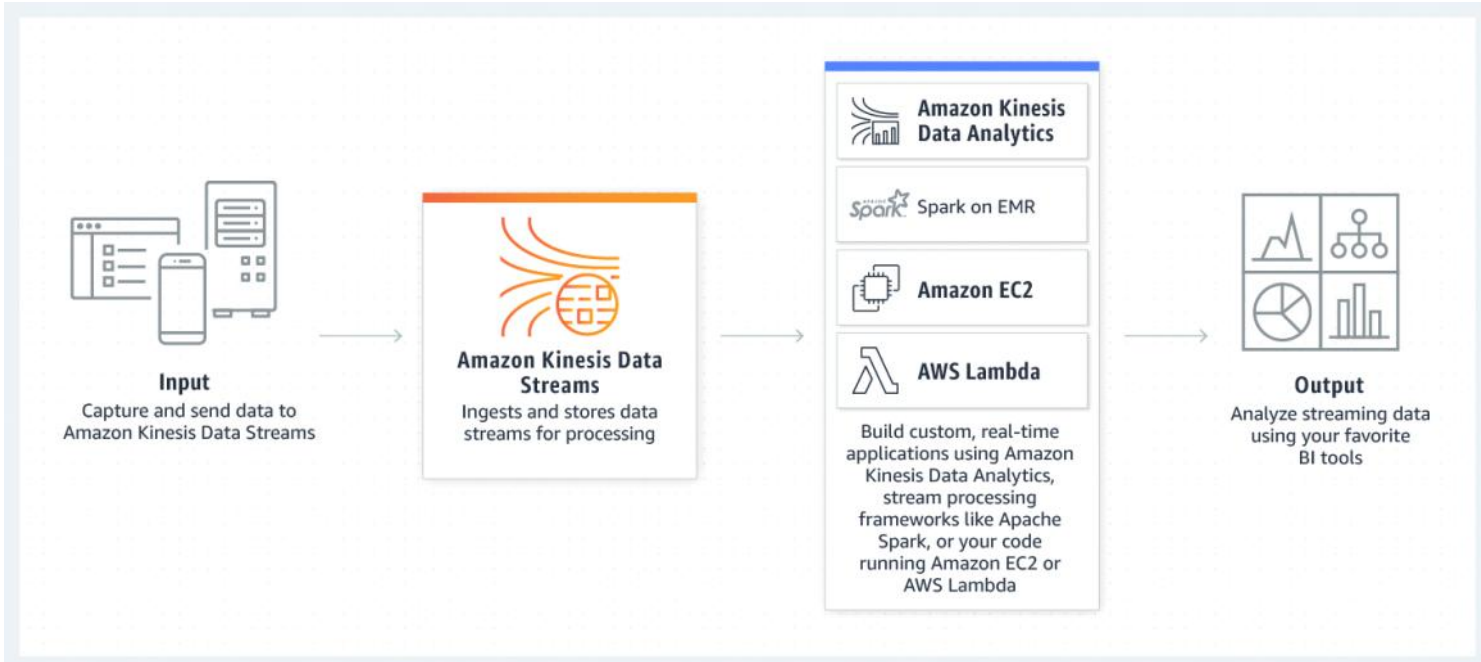
You are using an AWS Lambda function to process records in an Amazon Kinesis Data Streams stream which has 100 active shards. The Lambda function takes an average of 10 seconds to process the data and the stream is receiving 50 new items per second.

Which of the following statements are TRUE regarding this scenario?

- There will be at most 100 Lambda function invocations running concurrently.
- The Kinesis shards must be merged to increase the data capacity of the stream as well as the concurrency execution of the Lambda function.
- The Lambda function will throttle the incoming requests due to the excessive number of Kinesis shards.
- The Lambda function has 500 concurrent executions.

Incorrect

You can use an AWS Lambda function to process records in an Amazon Kinesis data stream. With Kinesis, you can collect data from many sources and process them with multiple consumers. Lambda supports standard data stream iterators and HTTP/2 stream consumers. Lambda reads records from the data stream and invokes your function synchronously with an event that contains stream records. Lambda reads records in batches and invokes your function to process records from the batch.



Concurrent executions refers to the number of executions of your function code that are happening at any given time. You can estimate the concurrent execution count, but it will differ depending on whether or not your Lambda function is processing events from a poll-based event source.

For Lambda functions that process Kinesis or DynamoDB streams, the number of shards is the unit of concurrency. If your stream has 100 active shards, there will be at most 100 Lambda function invocations running concurrently. This is because Lambda processes each shard's events in sequence.

Hence, the correct answer in this scenario is that: **there will be at most 100 Lambda function invocations running concurrently.**

The option that says: **the Lambda function has 500 concurrent executions** is incorrect because the number of concurrent executions for poll-based event sources is different from push-based event sources. This number of concurrent executions would have been correct if the Lambda function is integrated with a push-based event source such as API Gateway or Amazon S3 Events. Remember that the Kinesis and Lambda integration is using a poll-based event source, which means that the number of shards is the unit of concurrency for the function.

The option that says: **the Lambda function will throttle the incoming requests due to the excessive number of Kinesis shards** is incorrect because, by default, AWS Lambda will automatically scale the function's concurrency execution in response to increased traffic, up to your concurrency limit. Moreover, having 100 shards is not excessive at all as long as there is a sufficient number of workers or consumers of the stream.

The option that says: **the Kinesis shards must be merged to increase the data capacity of the stream as well as the concurrency execution of the Lambda function** is incorrect because, in the first place, you have to split the shards in order to increase the data capacity of the stream and not merge them. Since the Lambda function is using a poll-based event source mapping for Kinesis, the number of shards is the unit of concurrency for the function.

References:

<https://docs.aws.amazon.com/lambda/latest/dg/with-kinesis.html>

<https://docs.aws.amazon.com/lambda/latest/dg/concurrent-executions.html>

<https://docs.aws.amazon.com/lambda/latest/dg/scaling.html>

Check out this AWS Lambda Cheat Sheet:

<https://tutorialsdojo.com/aws-lambda/>

4. Question

A developer is building a photo-sharing application that automatically enhances images uploaded by users to Amazon S3. When a user uploads an image, its S3 path is sent to an image-processing application hosted on AWS Lambda. The Lambda function applies the selected filter to the image and stores it back to S3.

If the upload is successful, the application will return a prompt telling the user that the request has been accepted. The entire processing typically takes an average of 5 minutes to complete, which causes the application to become unresponsive.

Which of the following is the MOST suitable and cost-effective option which will prevent the application from being unresponsive?

- Configure the application to asynchronously process the requests and use the default

invocation type of the Lambda function.

- Use a combination of Lambda and Step Functions to orchestrate service components and asynchronously process the requests.
- Use AWS Serverless Application Model (AWS SAM) to allow asynchronous requests to your Lambda function.
- Configure the application to asynchronously process the requests and change the invocation type of the Lambda function to `Event`.

Correct

AWS Lambda supports synchronous and asynchronous invocation of a Lambda function. You can control the invocation type only when you invoke a Lambda function (referred to as *on-demand invocation*). The following examples illustrate on-demand invocations:

– Your custom application invokes a Lambda function.

– You manually invoke a Lambda function (for example, using the AWS CLI) for testing purposes.

In both cases, you invoke your Lambda function using the `Invoke` operation, and you can specify the invocation type as synchronous or asynchronous.

When you use AWS services as a trigger, the invocation type is predetermined for each service.

You have no control over the invocation type that these event sources use when they invoke your Lambda function.

In the `Invoke` API, you have 3 options to choose from for the `InvocationType`:

RequestResponse (default) – Invoke the function synchronously. Keep the connection open until the function returns a response or times out. The API response includes the function response and additional data.

Event – Invoke the function asynchronously. Send events that fail multiple times to the function's dead-letter queue (if it's configured). The API response only includes a status code.

DryRun – Validate parameter values and verify that the user or role has permission to invoke the function.

By configuring the application to asynchronously process requests by changing the invocation type of the Lambda function to "Event," the function can run in the background without blocking the main application. When the processing is complete, Lambda can store it back to S3 and trigger another event, such as a notification to the user that the image is ready.

Hence, the correct answer is to **configure the application to asynchronously process the requests and change the invocation type of the Lambda function to `Event`**.

Configuring the application to asynchronously process the requests and use the default invocation type of the Lambda function is incorrect because this will invoke your Lambda function synchronously. The default invocation type is `RequestResponse` which invokes the function synchronously and keeps the connection open until the function returns a response or times out.

Using AWS Serverless Application Model (AWS SAM) to allow asynchronous requests to your Lambda function is incorrect because AWS SAM just is an open-source framework that you can use to build serverless applications on AWS.

Using a combination of Lambda and Step Functions to orchestrate service components and asynchronously process the requests is incorrect because the AWS Step Functions service just lets you coordinate multiple AWS services into serverless workflows so you can build and update apps quickly. Although this can be a valid solution, it is not cost-effective since the application does not have a lot of components to orchestrate. Lambda functions can effectively meet the requirements in this scenario without using Step Functions by processing the requests asynchronously.

References:

<https://docs.aws.amazon.com/lambda/latest/dg/invocation-options.html>
https://docs.aws.amazon.com/lambda/latest/dg/API_Invoke.html

Check out this AWS Lambda Cheat Sheet:

<https://tutorialsdojo.com/aws-lambda/>

5. Question

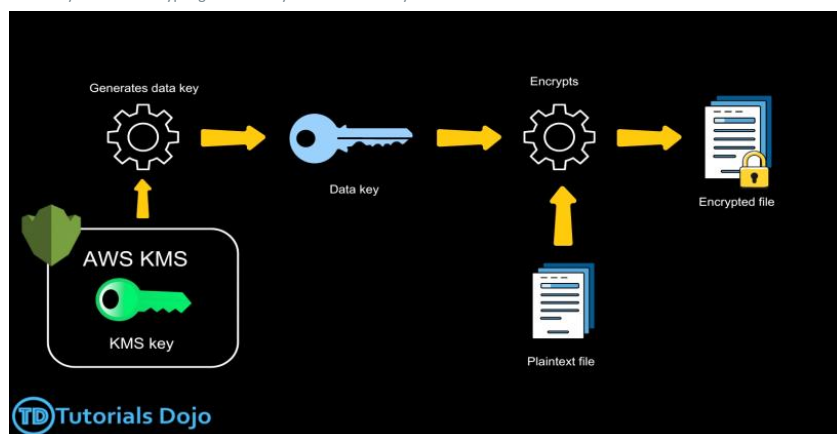
Your development team is currently developing a financial application in AWS. One of the requirements is to create and control the encryption keys used to encrypt your data using the envelope encryption strategy to comply with the strict IT security policy of the company.

Which of the following correctly describes the process of envelope encryption?

- Encrypt plaintext data with a KMS key and then encrypt the KMS key with a top-level encrypted data key.
- Encrypt plaintext data with a data key and then encrypt the data key with a top-level plaintext key.
- Encrypt plaintext data with a KMS key and then encrypt the KMS key with a top-level plaintext data key.
- Encrypt plaintext data with a data key and then encrypt the data key with a top-level encrypted key.

Incorrect

When you encrypt your data, your data is protected, but you have to protect your encryption key. One strategy is to encrypt it. **Envelope encryption** is the practice of encrypting plaintext data with a data key and then encrypting the data key under another key.



You can even encrypt the data encryption key under another encryption key, and encrypt that encryption key under another encryption key. But, eventually, one key must remain in plaintext so you can decrypt the keys and your data. This top-level plaintext key encryption key is known as the *root key*.

AWS KMS helps you protect your encryption keys by storing and managing them securely. Root keys stored in AWS KMS, known as AWS KMS keys, never leave the AWS KMS FIPS validated hardware security modules unencrypted. To use an AWS KMS key, you must call AWS KMS.

Envelope encryption offers several benefits:

Protecting data keys

When you encrypt a data key, you don't have to worry about storing the encrypted data key, because the data key is inherently protected by encryption. You can safely store the encrypted data key alongside the encrypted data.

Encrypting the same data under multiple keys

Encryption operations can be time-consuming, particularly when the data being encrypted are large objects. Instead of re-encrypting raw data multiple times with different keys, you can re-encrypt only the data keys that protect the raw data.

Combining the strengths of multiple algorithms

In general, symmetric key algorithms are faster and produce smaller ciphertexts than public-key algorithms, but public-key algorithms provide inherent separation of roles and easier key management. Envelope encryption lets you combine the strengths of each strategy.

Therefore, the correct answer is: **Encrypt plaintext data with a data key and then encrypt the data key with a top-level plaintext key.**

The option that says: **Encrypt plaintext data with a KMS key and then encrypt the KMS key with a top-level plaintext data key** is incorrect because you have to encrypt your plaintext data with a data key and not a KMS key. Moreover, the top-level plaintext key should be the root key and not the data key.

The option that says: **Encrypt plaintext data with a KMS key and then encrypt the KMS key with a top-level encrypted data key** is incorrect because plaintext data should be encrypted with a data key, not a KMS key. Also, the top-level key should be plaintext, not encrypted.

The option that says: **Encrypt plaintext data with a data key and then encrypt the data key with a top-level encrypted key** is incorrect. While it is correct to encrypt plaintext data with a data key, the top-level key (root key) must be kept in plaintext and not be encrypted.

References:

<https://docs.aws.amazon.com/kms/latest/developerguide/concepts.html#enveloping>

<https://docs.aws.amazon.com/kms/latest/developerguide/overview.html>

Check out this AWS Key Management Service (KMS) Cheat Sheet:

<https://tutorialsdodo.com/aws-key-management-service-aws-kms/>

6. 6. Question

A leading technology company is building a serverless application in AWS using the C++ programming language. The application will use DynamoDB as its data store, Lambda as its compute service, and API Gateway as its API Proxy. You are tasked to handle the deployment of the compute resources to AWS.

Which of the following steps should you implement to properly deploy the serverless application?

- Upload the deployment package to S3 and then use CloudFormation to deploy Lambda function with a reference to the S3 URL of the package.
- Use AWS Serverless Application Model (AWS SAM) to deploy the Lambda function.
- Create a new layer which contains the Custom Runtime for C++ and then launch a Lambda function which uses that runtime.
- Create a Lambda function with the C++ code and directly upload it to AWS.

Incorrect

You can implement an AWS Lambda runtime in any programming language. A runtime is a program that runs a Lambda function's handler method when the function is invoked. You can include a runtime in your function's deployment package in the form of an executable file named `bootstrap`.

A runtime is responsible for running the function's setup code, reading the handler name from an environment variable, and reading invocation events from the Lambda runtime API. The runtime passes the event data to the function handler, and posts the response from the handler back to Lambda.

Your custom runtime runs in the standard Lambda [execution environment](#). It can be a shell script, a script in a language that's included in Amazon Linux, or a binary executable file that's compiled in Amazon Linux.

Hence, the correct answer in this scenario is to **create a new layer which contains the Custom Runtime for C++ and then launch a Lambda function which uses that runtime.**

Uploading the deployment package to S3 and then using CloudFormation to deploy Lambda function with a reference to the S3 URL of the package is incorrect because you have to implement a Custom Runtime in order to execute the C++ code. Take note that this programming language is not natively supported yet in Lambda, which is why the use of a Custom Runtime is essential.

Creating a Lambda function with the C++ code and directly uploading it to AWS is incorrect because there is a 50 MB deployment package size limit in Lambda if you'll directly upload the package. Just as mentioned above, you have to implement a Custom Runtime for this scenario.

Using AWS Serverless Application Model (AWS SAM) to deploy the Lambda function is incorrect because using SAM alone is not enough to run the C++ code in Lambda. You have to use a Custom Runtime.

References:

<https://docs.aws.amazon.com/lambda/latest/dg/runtimes-custom.html>

<https://docs.aws.amazon.com/lambda/latest/dg/runtimes-walkthrough.html>

Check out this AWS Lambda Cheat Sheet:

<https://tutorialsdodo.com/aws-lambda/>

7. 7. Question

An application is sending thousands of log files to an S3 bucket everyday. The request to retrieve the list of objects using the AWS CLI `aws s3api list-objects` command is timing out due to the high volume of data being fetched. In order to rectify this issue, you have to use pagination to control the number of results returned on your request.

Which of the following parameters should you include in CLI command for this scenario? (Select TWO.)

- `--summarize`
- `--page-size`
- `--size-only`
- `--exclude`
- `--max-items`

Correct

For commands that can return a large list of items, the AWS Command Line Interface (AWS CLI) adds three options that you can use to control the number of items included in the output when the AWS CLI calls a service's API to populate the list. By default, the AWS CLI uses a page size of 1000 and retrieves all available items.

If you see issues when running list commands on a large number of resources, the default page size of 1000 might be too high. This can cause calls to AWS services to exceed the maximum allowed time and generate a "timed out" error. You can use the `--page-size` option to specify that the AWS CLI request a smaller number of items from each call to the AWS service. The CLI still retrieves the full list, but performs a larger number of service API calls in the background and retrieves a smaller number of items with each call. This gives the individual calls a better chance of succeeding without a timeout.

To include fewer items at a time in the AWS CLI output, use the `--max-items` option. The AWS CLI still handles pagination with the service as described above, but prints out only the number of items at a time that you specify. If the number of items output is fewer than the total number of

items returned by the underlying API calls, the output includes a `NextToken` that you can pass to a subsequent command to retrieve the next set of items.

Hence, the correct ones that you should include in the AWS CLI command are the `--page-size` and `--max-items` parameters.

The `--size-only` parameter is incorrect because this just accepts a boolean value and is typically used along with “s3 sync” command. It makes the size of each key the only criteria to use to decide whether to sync from source to destination.

The `--exclude` parameter is incorrect because it simply makes Amazon S3 exclude all files or objects that match a specified pattern from the result of the command.

The `--summary` parameter is incorrect because this only displays the summary information (number of objects, total size) of objects returned from an “s3 ls” command.

References:
<https://docs.aws.amazon.com/cli/latest/userguide/cli-usage-pagination.html>
<https://docs.aws.amazon.com/cli/latest/reference/s3/index.html#cli-aws-s3>

8. 8. Question

A company offers a Generative Artificial Intelligence (AI) service exposed through a REST API managed by Amazon API Gateway. They recently rolled out a subscription tier where users receive API keys to access premium features. The company uses the `CreateApiKey` API for generating these keys.

During testing, developers noticed that while existing users can access the service without issues, new premium subscribers get a `403 Forbidden` error when using their API keys.

What must be done to give new users access to the service?

- Associate the API keys for the premium users with the intended usage plan using the `CreateUsagePlanKey` operation.
- Use the `UpdateAuthorizer` operation to modify the authorization settings. Promote the changes to the production stage by calling the `CreateDeployment` operation.
- Use the `ImportApiKeys` operation to import the premium users’ keys, then apply the `UpdateUsagePlan` operation to set the new tier access.
- Instruct users to send their API key in a custom header. In the integration request, adjust the mapping template to extract and evaluate this header to distinguish between free-tier and premium subscribers.

Incorrect

In Amazon API Gateway, API keys by themselves do not grant access to execute an API. They need to be associated with a usage plan, and that usage plan then determines which API stages and methods the API key can access.

tdojo-paid-sub

Actions ▾Export usage data

Usage plan details

Usage plan ID	Rate
dawggp	10000 requests per second
Description	Burst
-	5000 requests
AWS Marketplace product code	Quota
-	10000 requests per month

Associated stagesAssociated API keysTags

API keys (1) info

RemoveGrant usage extensionAdd API key

Name	Status	ID	API key	Requests remaining this month
paid-YbyuVdMstnY	Enabled	i38nptom9tc	llnz3fcCvq4TZ1tomL4n42Q8umHpU8fC6k2DmLkl	9980

If the API key is not associated with a usage plan, it will not have permission to access any of the resources, which will result in a “403 Forbidden” error.

In the given scenario, existing users can access the service, but new premium subscribers cannot. This indicates that while the API keys were created for new users, they might not have been associated with the appropriate usage plan. Hence, after generating an API key, it must be added to a usage plan by calling the `CreateUsagePlanKey` method.

Hence, the correct answer is: **Associate the API keys for the premium users with the intended usage plan using the `CreateUsagePlanKey` operation.**

The option that says: **Use the `ImportApiKeys` operation to import the premium users’ keys, then apply the `UpdateUsagePlan` operation to set the new tier access** is incorrect. The `importApiKeys` API is primarily used for bulk importing API keys, not for associating them with a usage plan. Although the `updateUsagePlan` API modifies properties of a usage plan; it doesn’t handle direct association of API keys.

The option that says: **Use the `UpdateAuthorizer` operation to modify the authorization settings. Promote the changes to the production stage by calling the `CreateDeployment` operation** is incorrect. The `updateAuthorizer` operation is only used to modify the settings of an existing custom authorizer, which handles custom authorization logic for APIs. In the scenario, the issue is not related to custom authorization but rather to the association of API keys with a usage plan.

The option that says: **Instruct users to send their API key in a custom header. In the integration request, adjust the mapping template to extract and evaluate this header to distinguish between free-tier and premium subscribers** is incorrect. Changing the way users provide their API key adds unnecessary complexity and won’t solve the issue at hand. The problem isn’t with how the API key is being sent but with the API key not having appropriate permissions because it’s not associated with a usage plan.

References:
https://docs.aws.amazon.com/apigateway/latest/api/API_UpdateUsagePlan.html
https://docs.aws.amazon.com/apigateway/latest/api/API_CreateApiKey.html

Check out this Amazon API Gateway Cheat Sheet:
<https://tutorialsdojo.com/amazon-api-gateway/>

9. 9. Question

A company has a global multi-player game with a multi-master DynamoDB database topology which stores data in multiple AWS regions. You were assigned to develop a real-time data analytics application which will track and store the recent changes on all the tables from various regions. Only the new data of the recently updated item is needed to be tracked by your application.

Which of the following is the MOST suitable way to configure the data analytics application to detect and retrieve the updated database entries automatically?

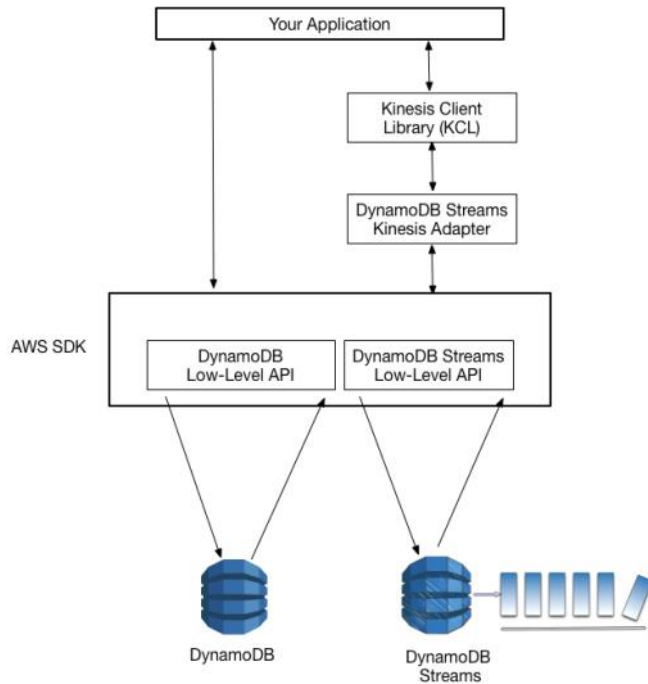
- Enable DynamoDB Streams and set the value of `StreamViewType` to `NEW_IMAGE`. Create a trigger in AWS Lambda to capture stream data and forward it to your

application.

- Enable DynamoDB Streams and set the value of `StreamViewType` to `NEW_AND_OLD_IMAGE`. Create a trigger in AWS Lambda to capture stream data and forward it to your application.
- Enable DynamoDB Streams and set the value of `StreamViewType` to `NEW_AND_OLD_IMAGE`. Use Kinesis Adapter in the application to consume streams from DynamoDB.
- Enable DynamoDB Streams and set the value of `StreamViewType` to `NEW_IMAGE`. Use Kinesis Adapter in the application to consume streams from DynamoDB.

Correct

DynamoDB Streams provides a time-ordered sequence of item-level changes in any DynamoDB table. The changes are de-duplicated and stored for 24 hours. Applications can access this log and view the data items as they appeared before and after they were modified, in near real time. The **Kinesis Adapter** is the recommended way to consume streams from DynamoDB for real-time processing. The DynamoDB Streams API is intentionally similar to that of Kinesis Streams, a service for real-time processing of streaming data at a massive scale. You can write applications for Kinesis Streams using the Kinesis Client Library (KCL). The KCL simplifies coding by providing useful abstractions above the low-level Kinesis Streams API. As a DynamoDB Streams user, you can leverage the design patterns found within the KCL to process DynamoDB Streams shards and stream records. To do this, you use the DynamoDB Streams Kinesis Adapter. The Kinesis Adapter implements the Kinesis Streams interface, so that the KCL can be used for consuming and processing records from DynamoDB Streams.



When an item in the table is modified, `StreamViewType` determines what information is written to the stream for this table. Valid values for `StreamViewType` are:

KEYS_ONLY – Only the key attributes of the modified item are written to the stream.
NEW_IMAGE – The entire item, as it appears after it was modified, is written to the stream.
OLD_IMAGE – The entire item, as it appeared before it was modified, is written to the stream.
NEW_AND_OLD_IMAGES – Both the new and the old item images of the item are written to the stream.

Hence, the correct answer is: **Enable DynamoDB Streams and set the value of `StreamViewType` to `NEW_IMAGE` then use Kinesis Adapter in the application to consume streams from DynamoDB.**

The option that says: **Enable DynamoDB Streams and set the value of `StreamViewType` to `NEW_AND_OLD_IMAGE`. Create a trigger in AWS Lambda to capture stream data and forward it to your application** is incorrect. Using Lambda for real-time data analytics is not a suitable solution for this scenario since it reads records in batches. A more appropriate service to use is the Kinesis service. In addition, using the `StreamViewType` of `NEW_AND_OLD_IMAGE` is wrong since this will send both the old and the new values of the item. Remember that it is specifically mentioned in the scenario that only the new values should be tracked.

The option that says: **Enable DynamoDB Streams and set the value of `StreamViewType` to `NEW_IMAGE`. Create a trigger in AWS Lambda to capture stream data and forward it to your application** is incorrect because just like what is mentioned above, it is better to use Kinesis instead of Lambda for the real-time data analytics application.

The option that says: **Enable DynamoDB Streams and set the value of `StreamViewType` to `NEW_AND_OLD_IMAGE`. Use Kinesis Adapter in the application to consume streams from DynamoDB** is incorrect because this will send both the old and the new values of the item to the data analytics application. The correct `StreamViewType` to use here should be `NEW_IMAGE`.

References:

<https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/Streams.html>
<https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/Streams.KCLAdapter.html>
https://docs.aws.amazon.com/amazondynamodb/latest/APIReference/API_StreamSpecification.html

Check out this Amazon DynamoDB Cheat Sheet:

<https://tutorialsdojo.com/amazon-dynamodb/>

AWS Lambda Integration with Amazon DynamoDB Streams:

<https://tutorialsdojo.com/aws-lambda-integration-with-amazon-dynamodb-streams/>

10. Question

A developer is creating an analytics REST API service that is powered by API Gateway. Analysts from a separate AWS account must interact with the service through an IAM role. The IAM role already has a policy that grants permission to invoke the API.

What else should the developer do to meet the requirement without too much overhead?

- Create an API Key for the API. Attach a resource policy to the API that grants

permission to the specified IAM role to invoke the `GetAPIKeys` action.

- Create a Lambda function authorizer for the API. In the Lambda function, write a logic that verifies the requester's identity by extracting the information from the `context` object.
- Create a Cognito User Pool authorizer. Add the IAM role to the user pool. Authenticate the requester's identity using Cognito. Ask the analysts to pass the token returned by Cognito in their request headers.
- Set `AWS_IAM` as the method authorization type for the API. Attach a resource policy to the API that grants permission to the specified IAM role to invoke the `execute-api:Invoke` action.

Correct

By using `AWS_IAM` as the method authorization type, it ensures that the API can only be accessed by IAM identities such as IAM users or IAM roles. Attaching a resource policy to the API that grants permission to the specified IAM role to invoke the `execute-api:Invoke` action allows the specified IAM role to make authorized requests to the API while denying access to any other unauthorized users or roles.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "AWS": [
          "arn:aws:iam::account-id:role/Analyst"
        ]
      },
      "Action": "execute-api:Invoke",
      "Resource": [
        "execute-api:/stage/GET/reports"
      ]
    }
  ]
}
```

This combination of method authorization and resource policy provides an additional layer of security for the API.

Hence, the correct answer in this scenario is to **Set `AWS_IAM` as the method authorization type for the API. Attach a resource policy to the API that grants permission to the specified IAM role to invoke the `execute-api:Invoke` action.**

The option that says: **Create an API Key for the API. Attach a resource policy to the API that grants permission to the specified IAM role to invoke the `GetAPIKeys` action** is incorrect. API Keys are just a way of identifying the calling parties that you trust, but they are not intended to be used to grant permissions to an IAM role.

The option that says: **Create a Lambda function authorizer for the API. In the Lambda function, write a logic that verifies the requester's identity by extracting the information from the `context` object** is incorrect. While this may be possible, Lambda function authorizer is more suitable for custom authorization scheme that uses a bearer token authentication strategy such as OAuth or SAML. Additionally, this approach requires you to write, test, and maintain custom authentication and authorization code, which can be complex and time-consuming.

The option that says: **Create a Cognito User Pool authorizer. Add the IAM role to the user pool. Authenticate the requester's identity using Cognito. Ask the analysts to pass the token returned by Cognito in their request headers** is incorrect. Adding a Cognito User Pool authorizer is unnecessary since the API will be accessed through an IAM role.

References:

<https://docs.aws.amazon.com/apigateway/latest/developerguide/apigateway-resource-policies-examples.html#apigateway-resource-policies-cross-account-example>
<https://docs.aws.amazon.com/apigateway/latest/developerguide/apigateway-authorization-flow.html>

Check out this AWS API Gateway Sheet:

<https://tutorialsdojo.com/amazon-api-gateway/>

11. Question

A startup wants an application that triggers processes in response to customer orders and inventory updates using AWS Lambda and Amazon EventBridge. The application requires a Lambda function to send the specific events to an Amazon EventBridge event bus. The developer uses the AWS SDK to call the `PutEvents` action on EventBridge. Upon deployment, the developer notices `AccessDeniedException` errors in the event logs, which is the reason why the function is not functioning as intended.

Which solution will help the developer in resolving the issue?

- Adjust the AWS Lambda function's execution role to grant it permissions for the `PutEvents` action on EventBridge.
- Update the developer's IAM role by including permission to explicitly allow the `PutEvents` operation on EventBridge.
- Implement a resource-based policy on the Lambda function that grants necessary permissions to perform the `PutEvents` action on EventBridge.
- Establish a Virtual Private Cloud (VPC) peering connection to facilitate communication between AWS Lambda and EventBridge.

Correct

AWS Lambda is a service that allows you to run your code without managing any servers. It only runs your code when it is needed and can scale from a few requests per day to thousands per second automatically. Moreover, AWS Lambda functions can be triggered by various AWS services, such as Amazon EventBridge. Amazon EventBridge is a serverless event bus that can connect application data from your apps, SaaS, and AWS services. Using EventBridge, you can build event-driven architectures that are highly scalable and decoupled from each other. This approach can make your applications more resilient and flexible.

EventBridge

Allow 1 Action

Specify what actions can be performed on specific resources in EventBridge

▼ Actions allowed

Specify actions from the service to be allowed.

Write

☒ PutEvents [Info](#)
☐ PutPartnerEvents [Info](#)
☐ PutRule [Info](#)

☐ PutTargets [Info](#)

Permissions management

☐ PutPermission [Info](#)

▼ Resources

Specify resource ARNs for these actions.

☒ All
 ☐ Specific

⚠ The all wildcard "*" may be overly permissive for the selected actions. Allowing specific ARNs for these service resources can improve security.

Effect

☒ Allow
 ☐ Deny

Lambda functions often need to interact with other AWS services, such as sending events to an EventBridge bus. However, to access and perform actions on these services, the function requires the necessary permissions. This is where the concept of an execution role comes into play. An execution role is an IAM role that grants the Lambda function permissions to access AWS services and resources. By adjusting the execution role of a Lambda function to include permissions for the `PutEvents` action on EventBridge, the function can safely interact with EventBridge. This method follows the AWS best practice of granting the least privilege, ensuring that the function only has the permissions needed to perform its designated tasks. As a result, the security posture of the application is enhanced.

In the provided scenario, where a developer's AWS Lambda function encounter `AccessDeniedException` errors when attempting to publish events to an EventBridge event bus, adjusting the Lambda function's execution role is the correct approach to resolving the issue. By granting the execution role permission for the `PutEvents` action on EventBridge, the Lambda function is authorized to execute the `PutEvents` call successfully. This solution emphasizes the importance of IAM roles in managing permissions within AWS environments, ensuring secure and efficient access management for serverless applications. Hence, the correct answer is: **Adjust the AWS Lambda function's execution role to grant it permissions for the `PutEvents` action on EventBridge.**

The option that says: **Establish a Virtual Private Cloud (VPC) peering connection to facilitate communication between AWS Lambda and EventBridge** is incorrect. VPC peering is a networking feature that allows traffic to be routed between two VPCs using private IP addresses. However, this option does not address the underlying issue, which is related to IAM permissions and not network connectivity. Amazon EventBridge is a serverless event bus service that doesn't require VPC peering with AWS Lambda for connectivity. The `AccessDeniedException` error indicates a permissions issue that needs IAM configuration adjustments rather than network infrastructure changes.

The option that says: **Update the developer's IAM role by including permission to explicitly allow the `PutEvents` operation on EventBridge** is incorrect because the permissions required to execute AWS service operations from within an AWS Lambda function are determined by the execution role attached to the Lambda function itself, not the IAM role of the developer. The execution role provides the Lambda function with the necessary AWS credentials to interact with other AWS services under the permissions defined in that role. Modifying the programmer's IAM role would not grant the Lambda function the permissions it needs to interact with EventBridge.

The option that says: **Implement a resource-based policy on the Lambda function that grants necessary permissions to perform the `PutEvents` action on EventBridge** is incorrect. Lambda functions do not use resource-based policies to grant permissions to access other AWS services. Instead, permissions are primarily managed through the execution role.

References:

<https://docs.aws.amazon.com/lambda/latest/dg/lambda-intro-execution-role.html>

<https://docs.aws.amazon.com/eventbridge/latest/userguide/eb-what-is.html>

Check out these AWS Lambda and Amazon EventBridge Cheat Sheets:

<https://tutorialsdojo.com/aws-lambda/>

<https://tutorialsdojo.com/amazon-eventbridge/>

12. Question

You were recently hired as a developer for a leading insurance firm in Asia which has a hybrid cloud architecture with AWS. The project that was assigned to you involves setting up a static website using Amazon S3 with a CORS configuration as shown below:

```
<?xml version="1.0" encoding="UTF-8"?>
<CORSConfiguration xmlns="http://s3.amazonaws.com/doc/2006-03-01/">
  <CORSRule>
    <AllowedOrigin>https://tutorialsdojo.com</AllowedOrigin>
    <AllowedMethod>GET</AllowedMethod>
    <AllowedMethod>PUT</AllowedMethod>
    <AllowedMethod>POST</AllowedMethod>
    <AllowedMethod>DELETE</AllowedMethod>
    <AllowedHeader>*</AllowedHeader>
    <ExposeHeader>ETag</ExposeHeader>
    <ExposeHeader>x-amz-meta-custom-header</ExposeHeader>
    <MaxAgeSeconds>3600</MaxAgeSeconds>
  </CORSRule>
</CORSConfiguration>
```

Which of the following statements are TRUE with regards to this S3 configuration? (Select TWO.)

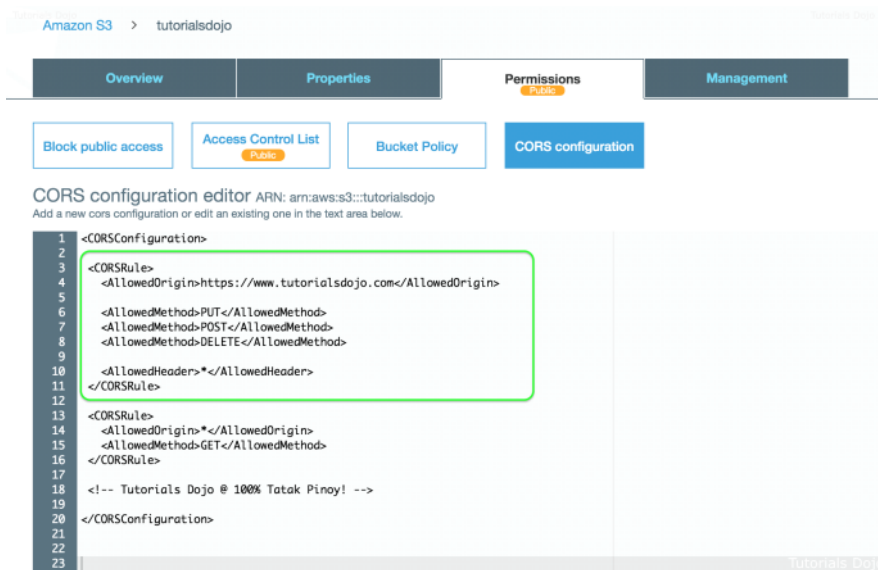
- It allows a user to view, add, remove or update objects inside the S3 bucket from the domain `tutorialsdojo.com`.
- The request will fail if the `x-amz-meta-custom-header` header is not included.
- This configuration authorizes the user to perform actions on the S3 bucket.
- This will cause the browser to cache the response of the preflight OPTIONS request for 1 hour.
- All HTTP Methods are allowed.

Incorrect

New Section 3 Page 8

Cross-origin resource sharing (CORS) defines a way for client web applications that are loaded in one domain to interact with resources in a different domain. With CORS support, you can build rich client-side web applications with Amazon S3 and selectively allow cross-origin access to your Amazon S3 resources.

To configure your bucket to allow cross-origin requests, you create a CORS configuration, which is an XML document with rules that identify the origins that you will allow to access your bucket, the operations (HTTP methods) that will support each origin, and other operation-specific information. You can add up to 100 rules to the configuration. You add the XML document as the cors subresource to the bucket either programmatically or by using the Amazon S3 console as shown below:



A CORS configuration is an XML file that contains a series of rules within a `<CORSRule>`. A configuration can have up to 100 rules. A rule is defined by one of the following tags:

AllowedOrigin – Specifies domain origins that you allow to make cross-domain requests.

AllowedMethod – Specifies a type of request you allow (GET, PUT, POST, DELETE, HEAD) in cross-domain requests.

AllowedHeader – Specifies the headers allowed in a preflight request.

Below are some of the `CORSRule` elements:

MaxAgeSeconds – Specifies the amount of time in seconds (in this example, 3000) that the browser caches an Amazon S3 response to a preflight OPTIONS request for the specified resource. By caching the response, the browser does not have to send preflight requests to Amazon S3 if the original request will be repeated.

ExposeHeader – Identifies the response headers (in this example, `x-amz-server-side-encryption`, `x-amz-request-id`, and `x-amz-id-2`) that customers are able to access from their applications (for example, from a JavaScript XMLHttpRequest object).

Hence, the correct answers in this scenario are:

- It allows a user to view, add, remove or update objects inside the S3 bucket from the domain tutorialsdjojo.com
- This will cause the browser to cache an Amazon S3 response of a preflight OPTIONS request for 1 hour

The option that says: **the request will fail if the x-amz-meta-custom-header header is not included** is incorrect because the `ExposeHeader` element refers to the header that will be exposed to the response and not a constraint for the request.

The option that says: **this configuration authorizes the user to perform actions on the S3 bucket** is incorrect because this configuration actually does the opposite. It doesn't authorize the user to perform actions on the S3 bucket.

The option that says: **all HTTP Methods are allowed** is incorrect because the configuration didn't include the HEAD HTTP method.

References:

<http://docs.aws.amazon.com/AmazonS3/latest/dev/cors.html>

<https://docs.aws.amazon.com/sdk-for-javascript/v2/developer-guide/cors.html>

13. Question

A web application hosted in Elastic Beanstalk has a configuration file named `.ebextensions/debugging.config` which has the following content:

```
option_settings:
  aws:elasticbeanstalk:xray:
    XRayEnabled: true
```

For its database tier, it uses RDS with Multi-AZ deployments configuration and Read Replicas. There is a new requirement to record calls that your application makes to RDS and other internal or external HTTP web APIs. The tracing information should also include the actual SQL database queries sent by the application, which can be searched using the filter expressions in the X-Ray Console.

Which of the following should you do to satisfy the above task?

- Add metadata in the subsegment section of the segment document.
- Add metadata in the segment document.
- Add annotations in the subsegment section of the segment document.
- Add annotations in the segment document.

Incorrect

Even with sampling, a complex application generates a lot of data. The AWS X-Ray console provides an easy-to-navigate view of the service graph. It shows health and performance information that helps you identify issues and opportunities for optimization in your application. For advanced tracing, you can drill down to traces for individual requests, or use **filter expressions** to find traces related to specific paths or users.

http.url CONTAINS '/api/user' Last 15 minutes			
Trace overview			
Group by: URL			
URL	Avg response time	% of Traces	Response
http://scorekeep-elasticbeanstalk.com/api/user	401 ms	95.56%	43 OK, 0 Throttled, 0 Errors, 0 Faults
http://scorekeep-elasticbeanstalk.com/api/user/KRGHASNF	300 ms	2.22%	1 OK, 0 Throttled, 0 Errors, 0 Faults
http://scorekeep-elasticbeanstalk.com/api/user/BAPP0F1S	303 ms	2.22%	1 OK, 0 Throttled, 0 Errors, 0 Faults

When you instrument your application, the X-Ray SDK records information about incoming and outgoing requests, the AWS resources used, and the application itself. You can add other information to the segment document as annotations and metadata.

Annotations are simple key-value pairs that are indexed for use with [filter expressions](#). Use annotations to record data that you want to use to group traces in the console or when calling the [GetTraceSummaries](#) API. X-Ray indexes up to 50 annotations per trace.

Metadata are key-value pairs with values of any type, including objects and lists, but that is not indexed. Use metadata to record data you want to store in the trace but don't need to use for searching traces. You can view annotations and metadata in the segment or subsegment details in the X-Ray console.

A trace segment is a JSON representation of a request that your application serves. A trace segment records information about the original request, information about the work that your application does locally, and subsegments with information about downstream calls that your application makes to AWS resources, HTTP APIs, and SQL databases.

Hence, **adding annotations in the subsegment section of the segment document** is the correct answer.

Adding annotations in the segment document is incorrect. Although the use of annotations is correct, you have to add this in the **subsegment** section of the **segment** document since you want to trace the downstream call to RDS and not the actual request to your application.

Adding metadata in the segment document is incorrect because metadata is primarily used to record custom data that you want to store in the trace but not for searching traces since this can't be picked up by filter expressions in the X-Ray Console. You have to use annotations instead. In addition, you have to add this in the **subsegment** section of the **segment** document since you want to trace the downstream call to RDS and not the actual request to your application.

Adding metadata in the subsegment section of the segment document is incorrect because, just as mentioned above, metadata is just used to record custom data that you want to store in the trace but not for searching traces.

References:

<https://docs.aws.amazon.com/xray/latest/devguide/xray-concepts.html#xray-concepts-annotations>

<https://docs.aws.amazon.com/xray/latest/devguide/xray-console-filters.html>

Check out this AWS X-Ray Cheat Sheet:

<https://tutorialsdjojo.com/aws-x-ray/>

14. Question

You are working as an IT Consultant for a top investment bank in Europe which uses several serverless applications in their AWS account. They just launched a new API Gateway service with a Lambda proxy integration and you were instructed to test out the new API. However, you are getting a Connection refused error whenever you use this Invoke URL

URL <http://779protaw8.execute-api.us-east-1.amazonaws.com/tutorialsdjojo/> of the API Gateway.

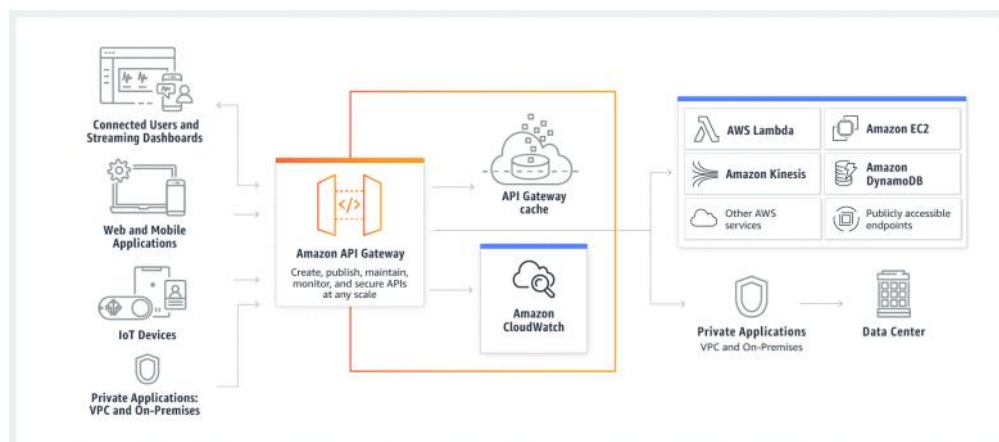
Which of the following is the MOST likely cause of this issue?

- You are not using WebSocket in invoking the API.
- You are not using FTP in invoking the API.
- You are not using HTTPS in invoking the API.
- You are not using HTTP/2 in invoking the API.

Correct

All of the APIs created with Amazon API Gateway expose HTTPS endpoints only. Amazon API Gateway does not support unencrypted (HTTP) endpoints. By default, Amazon API Gateway assigns an internal domain to the API that automatically uses the Amazon API Gateway certificate.

When configuring your APIs to run under a custom domain name, you can provide your own certificate for the domain.



Calling a deployed API involves submitting requests to the URL for the API Gateway component service for API execution, known as `execute-api`. The base URL for REST APIs is in the following format:

[https://\[restapi_id\].execute-api.\[region\].amazonaws.com/\[stage_name\]/](https://[restapi_id].execute-api.[region].amazonaws.com/[stage_name]/)

where `{restapi_id}` is the API identifier, `{region}` is the region, and `{stage_name}` is the stage name of the API deployment.

Hence, the most likely cause of the issue in the scenario is that **you are not using HTTPS in invoking the API**.

The option that says: **you are not using HTTP/2 in invoking the API** is incorrect because API Gateway only supports HTTPS.

The option that says: **you are not using FTP in invoking the API** is incorrect because API Gateway is using HTTPS to expose the APIs. FTP is primarily used for accessing file servers and not Web

APIs.

The option that says: **you are not using WebSocket in invoking the API** is incorrect because all of the APIs created with Amazon API Gateway expose HTTPS endpoints only.

References:

<https://docs.aws.amazon.com/apigateway/latest/developerguide/how-to-call-api.html>

<https://aws.amazon.com/api-gateway/faqs/>

Check out this Amazon API Gateway Cheat Sheet:

<https://tutorialsdojo.com/amazon-api-gateway/>

15. 15. Question

A company has a microservices application that must be integrated with API Gateway. The developer must configure custom data mapping between the API Gateway and the microservices.

In addition, the developer must specify how the incoming request data is mapped to the integration request and how the resulting integration response data is mapped to the method response.

Which of the following integration types is the MOST suitable one to use in API Gateway to meet this requirement?

- AWS_PROXY
- HTTP_PROXY
- AWS
- HTTP

Incorrect

You can integrate an API method in your API Gateway with a custom HTTP endpoint of your application in two ways:

- HTTP proxy integration
- HTTP custom integration

In your API Gateway console, you can define the type of HTTP integration of your resource by toggling the “Proxy resource” switch.

Create resource

Resource details

☒ Proxy resource [Info](#)
Proxy resources handle requests to all sub-resources. To create a proxy resource use a path parameter that ends with a plus sign, for example {proxy+}.

Resource path:

Resource name:

☐ CORS (Cross Origin Resource Sharing) [Info](#)
Create an OPTIONS method that allows all origins, all methods, and several common headers.

Tutorials Dojo

Cancel Create resource

With proxy integration, the setup is simple. You only need to set the HTTP method and the HTTP endpoint URI, according to the backend requirements, if you are not concerned with content encoding or caching.

With custom integration, setup is more involved. In addition to the proxy integration setup steps, you need to specify how the incoming request data is mapped to the integration request and how the resulting integration response data is mapped to the method response. API Gateway supports the following endpoint ports: 80, 443 and 1024-65535.

Programmatically, you choose an integration type by setting the `type` property on the `Integration` resource. For the Lambda proxy integration, the value is `AWS_PROXY`. For the Lambda custom integration and all other AWS integrations, it is `AWS`. For the HTTP proxy integration and HTTP integration, the value is `HTTP_PROXY` and `HTTP`, respectively. For the mock integration, the `type` value is `MOCK`.

Since the integration type that is being described in the scenario fits the definition of an *HTTP custom integration*, the correct answer in this scenario is to use the `HTTP` integration type.

Hence, the correct answer is: **HTTP**.

AWS is incorrect because this type is primarily used for Lambda custom integration. Since the scenario does not specify that the microservices are Lambda functions, the HTTP integration type is the most flexible and suitable for such a scenario.

AWS_PROXY is incorrect because this type is primarily used for Lambda proxy integration. The scenario didn't mention that it uses a serverless application or Lambda.

HTTP_PROXY is incorrect because this type is only used for HTTP proxy integration where you don't need to do data mapping for your request and response data.

References:

<https://docs.aws.amazon.com/apigateway/latest/developerguide/setup-http-integrations.html>

<https://docs.aws.amazon.com/apigateway/latest/developerguide/api-gateway-api-integration-types.html>

Check out this Amazon API Gateway Cheat Sheet:

<https://tutorialsdojo.com/amazon-api-gateway/>

16. 16. Question

A company has an application hosted in an On-Demand EC2 instance in your VPC. The developer has been instructed to create a shell script that fetches the instance's associated public and private IP addresses.

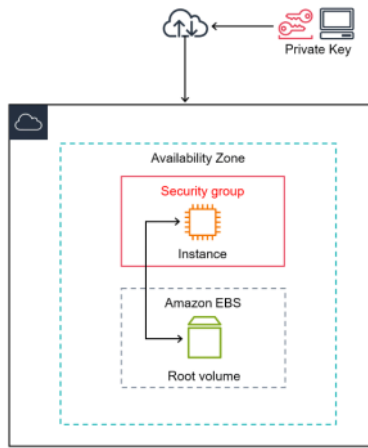
What should the developer do to complete this task?

- Get the public and private IP addresses from AWS CloudTrail.
- Get the public and private IP addresses from the instance metadata service using the <http://169.254.169.254/latest/meta-data/> endpoint.
- Get the public and private IP addresses from the instance user data service using the <http://169.254.169.254/latest/userdata/> endpoint.
- Get the public and private IP addresses from Amazon CloudWatch.

Correct

Instance metadata is data about your EC2 instance that you can use to configure or manage the running instance. Because your instance metadata is available from your running instance, you do not need to use the Amazon EC2 console or the AWS CLI. This can be helpful when you're writing

scripts to run from your instance. For example, you can access the local IP address of your instance from instance metadata to manage a connection to an external application.



To view the private IPv4 address, public IPv4 address, and all other categories of instance metadata from within a running instance, use the following URL: <http://169.254.169.254/latest/meta-data/>.

Hence, the correct answer is: **Get the public and private IP addresses from the instance metadata service using the <http://169.254.169.254/latest/meta-data/> endpoint.**

The option that says: **Get the public and private IP addresses from Amazon CloudWatch** is incorrect because there is no direct way to fetch the public and private IP addresses of the EC2 instance using CloudWatch.

The option that says: **Get the public and private IP addresses from AWS CloudTrail** is incorrect because CloudTrail is primarily used to track the API activity of each AWS service. Just like CloudWatch, there is no easy way to get the associated IP addresses of the EC2 instance using CloudTrail.

The option that says: **Get the public and private IP addresses from the instance user data service using the <http://169.254.169.254/latest/userdata/> endpoint** is incorrect because a user data is mainly used to perform common automated configuration tasks and run scripts after the instance starts. You will not find the associated IP addresses of the EC2 instance from its user data. You have to use the metadata service instead.

References:

<http://docs.aws.amazon.com/AWSEC2/latest/UserGuide/ec2-instance-metadata.html>
<https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/instance-identity-documents.html>

Check out this Amazon EC2 Cheat Sheet:

<https://tutorialsdojo.com/amazon-elastic-compute-cloud-amazon-ec2/>

17. Question

A developer is managing a serverless application orchestrated by AWS Step Functions. One of the Lambda functions sends an API call to a third-party payment service, which takes some time to complete. The Step Functions workflow needs to pause while the service validates the payment. It should only resume after the service sends a notification to a webhook endpoint.

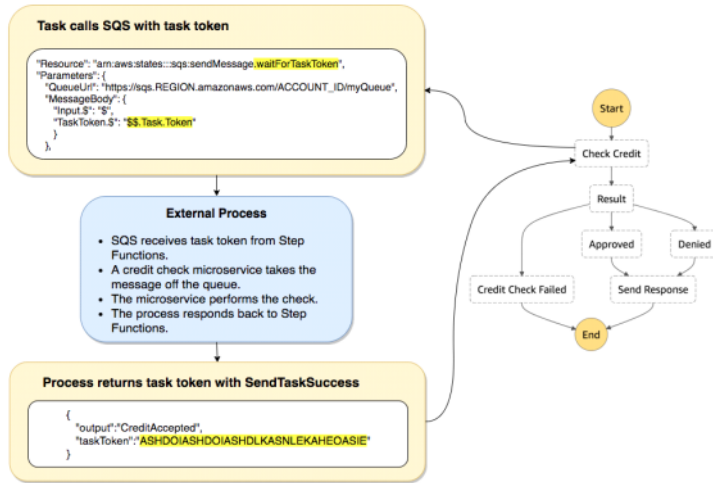
Which combination of actions will fulfill the requirements in the most cost-effective manner? (Select Two)

- Configure the Lambda function task state to use the `waitForTaskToken` option. Retrieve the task token from the context object of the state machine and include it as part of the Lambda function's payload body.
- Configure the webhook handler to call the `SendTaskSuccess` method after a successful notification.
- Configure the webhook handler to call the `SendTaskHeartbeat` method after a successful notification.
- Use a Wait State to pause the execution of the workflow. Configure the webhook handler to invoke the Lambda function synchronously.
- Set the invocation method of the Lambda function task state to asynchronous. Create an AWS SQS queue and configure the webhook handler to send the payment service's response to the queue. Use a combination of Wait State and Choice State to poll the queue.

Correct

In AWS Step Functions, the `waitForTaskToken` option allows a task to be paused until an external system signals its completion. When a task is configured with this option, Step Functions generates a unique token, which can be retrieved from the context object of the state machine. This token, for instance, can be stored in a data store for reference.

The diagram below depicts how `waitForTaskToken` is used for an SQS task state.



An external system, such as a webhook handler can then reference the token and call the `SendTaskSuccess` or `SendTaskFailure` method to signal Step Functions to resume the workflow. When the workflow is in a paused state, you're not billed for the time the workflow is paused, making it a cost-effective method for awaiting external processes or events.

Hence, the correct answers are:

- **Configure the Lambda function task state to use the `waitForTaskToken` option.** Retrieve the task token from the context object of the state machine and include it as part of the Lambda function's payload body.
- **Configure the webhook handler to call the `SendTaskSuccess` method after a successful notification.**

The option that says: **Set the invocation method of the Lambda function task state to asynchronous. Create an AWS SQS queue and configure the webhook handler to send the payment service's response to the queue. Use a combination of Wait State and Choice State to poll the queue** is incorrect. While this solution may work, every iteration involving the Wait State and Choice State incurs a cost as a state transition. If the third-party service takes an unpredictable amount of time, the state machine could go through multiple cycles of waiting and checking the SQS queue, resulting in a higher cost.

The option that says: **Use a Wait State to pause the execution of the workflow. Configure the webhook handler to invoke the Lambda function synchronously** is incorrect. A fixed Wait State is less cost-effective in scenarios where the waiting duration is unpredictable. If the third-party service finishes earlier than the wait duration, you're paying for unused time. If it takes longer, the workflow might proceed before the task is complete.

The option that says: **Configure the webhook handler to call the `SendTaskHeartbeat` method after a successful notification** is incorrect because this method is simply used for keeping tasks alive and preventing them from timing out. It also does not signal completion.

References:

<https://aws.amazon.com/blogs/compute/building-cost-effective-aws-step-functions-workflows/>
<https://docs.aws.amazon.com/step-functions/latest/dg/callback-task-sample-sqs.html>
<https://docs.aws.amazon.com/step-functions/latest/dg/connect-to-resource.html>

Check out this AWS Step Functions Cheat Sheet:

<https://tutorialsdojo.com/aws-step-functions/>

18. Question

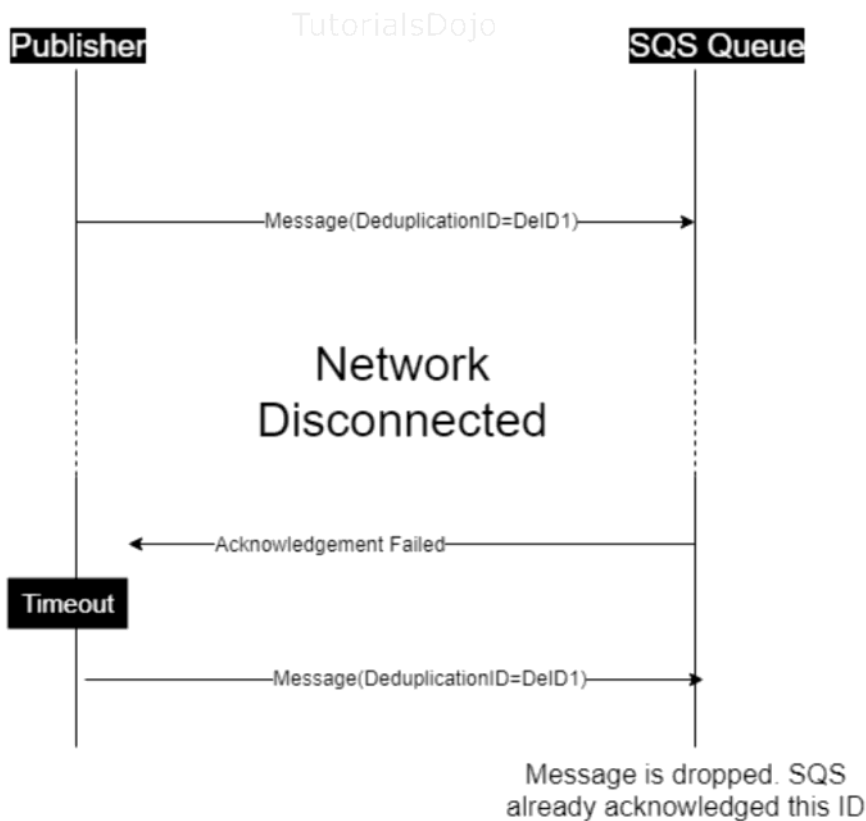
A developer monitors multiple sensors inside a data center which detects various environmental conditions that may affect their running servers. In the current architecture, the data is initially processed by an AWS Lambda function and then stored in a remote data warehouse. To make the system more durable and scalable, the developer plans to use an Amazon SQS FIFO queue to store the data, which will be polled by the Lambda function. There is a known issue with the sensor devices sending duplicate data intermittently.

What action can the developer take to lessen the chances of processing duplicate messages?

- Use an Amazon SQS Standard queue instead of a FIFO queue to avoid any duplicate messages.
- Refactor the Lambda function to store the message's content and drop the incoming messages with similar content within a 5-minute period.
- Add a `MessageDeduplicationId` parameter to the `SendMessage` API request.
- Configure the Amazon SQS queue to automatically drop a duplicate message whenever it arrives within the message's `VisibilityTimeout`.

Incorrect

Amazon SQS *FIFO First-In-First-Out* queues are designed to enhance messaging between applications when the order of operations and events is critical or where duplicates can't be tolerated.



Amazon SQS FIFO queues follow exactly-once processing. It introduces a parameter called **Message Deduplication ID**, which is the token used for deduplication of sent messages. Suppose a message with a particular message deduplication ID is sent successfully. In that case, any messages sent with the same message deduplication ID are accepted successfully but aren't delivered during the 5-minute deduplication interval.

Message Group ID ⓘ	MyMessageGroupId1234567890
Message Deduplication ID ⓘ	MyMessageDeduplicationId1234567890

[Cancel](#)
[Send Message](#)

SQS remembers the MessageDeduplicationId values it's seen for at least five minutes, which means deduplication IDs can only reduce, not completely eliminate, the chances of duplication occurring. For example, if a producer was unable to receive an acknowledgment after sending a message due to a network issue and then regains connection after 10 minutes and attempts to resend the message, there is a risk of duplication occurring. In this scenario, you can lessen the chances of the Lambda function processing duplicate messages by storing data in an SQS FIFO queue. You may provide a MessageDeduplicationId value so SQS can distinguish one message from another. Optionally, you may enable ContentBasedDeduplication to let SQS create an SHA-256 hash based on the message body and use it as the value for MessageDeduplicationId.

Hence, in this scenario, the correct answer is to: **Add**

a MessageDeduplicationId parameter to the SendMessage API request.

Refactoring the Lambda function to store the message's content and dropping the incoming messages with similar content within a 5-minute period is incorrect because Lambda functions do not share data amongst themselves during a scale-up event. Therefore, if a function is processing a message and another function handles the succeeding message, it would not be able to compare if it is indeed a duplicate or not. You have to configure the SQS FIFO queue to use a Message Deduplication ID in order to avoid having duplicate messages.

Configuring the Amazon SQS queue to automatically drop a duplicate message whenever it arrives within the message's VisibilityTimeout is incorrect because the visibility timeout is primarily used to prevent other consumers from processing the message again and not for detecting duplicate messages.

Using an Amazon SQS Standard queue instead of a FIFO queue to avoid any duplicate messages is incorrect because using standard queues will actually introduce duplicate messages. Take note that FIFO queues help you avoid sending duplicates and not the Standard-type queue.

References:

<https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/using-messagededuplicationid-property.html>
https://docs.amazonaws.cn/en_us/AWSSimpleQueueService/latest/SQSDeveloperGuide/FIFO-queues.html#FIFO-queues-exactly-once-processing
<https://aws.amazon.com/blogs/developer/how-the-amazon-sqs-fifo-api-works/>

Check out this Amazon SQS Cheat Sheet:

<https://tutorialsdojo.com/amazon-sqs/>

19. Question

A developer is building a prototype microservices that are running as tasks in an Amazon ECS Cluster. His manager instructed him to define a task placement strategy which needs to be both cost and resource efficient. The task placement should minimize the number of instances in use which will keep the cost down since high availability is not much of a concern for this prototype. What should the developer implement to meet the above requirements?

- Distribute tasks among all registered EC2 instances based on the least available amount of CPU or memory using the `binpack` task placement strategy.
- Distribute tasks evenly across all available EC2 instances using the `spread` task placement strategy.

- Place tasks randomly using the `random` task placement strategy.
- Distribute tasks evenly across Availability Zones, and then re-distribute the tasks among EC2 instances based on the least available amount of CPU/memory within each Availability Zone.

Correct

The binpack strategy tries to fit your workloads in as few instances as possible. It gets its name from the bin packing problem where the goal is to fit objects of various sizes in the smallest number of bins. It is well suited to scenarios for minimizing the number of instances in your cluster, perhaps for cost savings, and lends itself well to automatic scaling for elastic workloads, to shut down instances that are not in use.

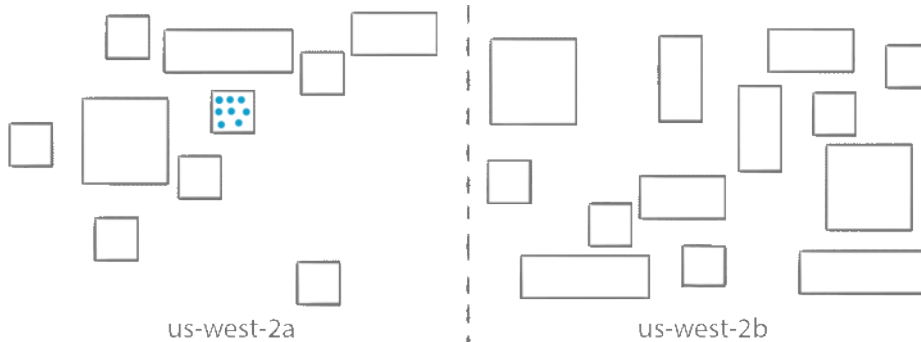
A **task placement strategy** is an algorithm for selecting instances for task placement or tasks for termination. Task placement strategies can be specified when either running a task or creating a new service.

Amazon ECS supports the following task placement strategies:

binpack – Place tasks based on the least available amount of CPU or memory. This minimizes the number of instances in use.

random – Place tasks randomly.

spread – Place tasks evenly based on the specified value. Accepted values are attribute key-value pairs, `instanceId`, or `host`.



When you use the binpack strategy, you must also indicate if you are trying to make optimal use of your instances' CPU or memory. This is done by passing an extra field parameter, which tells the task placement engine which parameter to use to evaluate how "full" your "bins" are. It then chooses the instance with the least available CPU or memory (depending on which you pick). If there are multiple instances with this CPU or memory remaining, it chooses randomly.

By spreading tasks among your EC2 instances using the binpack strategy, you can minimize costs and resource consumption since this strategy maximizes available CPU/memory of your already running instances.

Hence, the correct answer is: **Distribute tasks among all registered EC2 instances based on the least available amount of CPU or memory using the binpack task placement strategy.**

The option that says: **Distribute tasks evenly across all available EC2 instances using the spread task placement strategy** is incorrect because this strategy is typically used to achieve high availability by making sure that multiple copies of a task are scheduled across multiple instances based on attributes such as Availability Zones. Since the scenario is focused on cost rather than availability, this option is clearly not suitable for this scenario.

The option that says: **Place tasks randomly using the random task placement strategy** is incorrect. Random task placement just ensures tasks are run on instances with sufficient resources to complete them. Binpack has better cost-savings since it strategically places tasks in as few instances as possible.

The option that says: **Distribute tasks evenly across Availability Zones, and then re-distributing the tasks among EC2 instances based on the least available amount of CPU/memory within each Availability Zone** is incorrect. Although it will meet the required task placement, this method will use more unnecessary EC2 instances. Take note that the scenario requires you to minimize the number of instances in use, which will keep the cost down.

References:

<https://aws.amazon.com/blogs/compute/amazon-ecs-task-placement/>

<https://docs.aws.amazon.com/AmazonECS/latest/developerguide/task-placement.html>

<https://docs.aws.amazon.com/AmazonECS/latest/developerguide/task-placement-strategies.html>

Check out this Amazon ECS Cheat Sheet:

<https://tutorialsdojo.com/amazon-elastic-container-service-amazon-ecs/>

20. Question

A programmer is developing a Node.js application that will be run on a Linux server in their on-premises data center. The application will access various AWS services such as S3, DynamoDB, and ElastiCache using the AWS SDK.

Which of the following is the MOST suitable way to provide access for the developer to accomplish the specified task?

- Go to the AWS Console and create a new IAM User with the appropriate permissions. In the application server, create the credentials file at `~/.aws/credentials` with the username and the hashed password of the IAM User.
- Go to the AWS Console and create a new IAM user with programmatic access. In the application server, create the credentials file at `~/.aws/credentials` with the access keys of the IAM user.
- Create an IAM role with the appropriate permissions to access the required AWS services. Assign the role to the on-premises Linux server.
- Create an IAM role with the appropriate permissions to access the required AWS services and assign the role to the on-premises Linux server. Whenever the application needs to access any AWS services, request for temporary security credentials from STS using the `AssumeRole` API.

Incorrect

If you have resources that are running inside AWS that need programmatic access to various AWS services, then the best practice is always to use IAM roles. However, applications running outside of an AWS environment will need access keys for programmatic access to AWS resources. For example, monitoring tools running on-premises and third-party automation tools will need access keys.

Access keys are long-term credentials for an IAM user or the AWS account root user. You can use access keys to sign programmatic requests to the AWS CLI or AWS API (directly or using the AWS SDK).

Set user details

You can add multiple users at once with the same access type and permissions. [Learn more](#)

User name* tutorialsdojo-user

[Add another user](#)

Select AWS access type

Select how these users will access AWS. Access keys and autogenerated passwords are provided in the last step. [Learn more](#)

- Access type*
- ☒ **Programmatic access**
Enables an **access key ID** and **secret access key** for the AWS API, CLI, SDK, and other development tools.
 - ☐ **AWS Management Console access**
Enables a **password** that allows users to sign-in to the AWS Management Console.

In order to use the AWS SDK for your application, you have to create your credentials file first at `~/.aws/credentials` for Linux servers or at `C:\Users\USER_NAME\.aws\credentials` for Windows users and then save your access keys.

Hence, the correct answer is: **Go to the AWS Console and create a new IAM user with programmatic access. In the application server, create the credentials file at `~/.aws/credentials` with the access keys of the IAM user.**

The option that says: **Create an IAM role with the appropriate permissions to access the required AWS services and assign the role to the on-premises Linux server. Whenever the application needs to access any AWS services, request for temporary security credentials from STS using the AssumeRole API** is incorrect because the scenario says that the application is running in a Linux server on-premises and not on an EC2 instance. You cannot directly assign an IAM Role to a server on your on-premises data center. Although it may be possible to use a combination of STS and IAM Role, the use of access keys for AWS SDK is still preferred, especially if the application server is on-premises.

The option that says: **Create an IAM role with the appropriate permissions to access the required AWS services. Assign the role to the on-premises Linux server** is also incorrect because, just as mentioned above, the use of an IAM Role is not a suitable solution for this scenario.

The option that says: **Go to the AWS Console and create a new IAM User with the appropriate permissions. In the application server, create the credentials file at `~/.aws/credentials` with the username and the hashed password of the IAM User** is incorrect. An IAM user's username and password can only be used to interact with AWS via its Management Console. These credentials are intended for human use and are not suitable for use in automated systems, such as applications and scripts that make programmatic calls to AWS services.

References:

<https://aws.amazon.com/developers/getting-started/nodejs/>
<https://docs.aws.amazon.com/general/latest/gr/aws-sec-cred-types.html#access-keys-and-secret-access-keys>
<https://aws.amazon.com/blogs/security/guidelines-for-protecting-your-aws-account-while-using-programmatic-access/>

Check out this AWS IAM Cheat Sheet:

<https://tutorialsdojo.com/aws-identity-and-access-management-iam/>

21. Question

A developer is working with an AWS Serverless Application Model (AWS SAM) application composed of several AWS Lambda functions. The developer runs the application locally on his laptop using `sam local` commands. While testing, one of the functions returns `Access denied` errors. Upon investigation, the developer discovered that the Lambda function is using the AWS SDK to make API calls within a sandbox AWS account.

Which combination of steps must the developer do to resolve the issue? (Select TWO)

- Run the function using `sam local invoke` with the `--profile` parameter.
- Add the AWS credentials of the sandbox AWS account to the `Globals` section of the `template.yml` file and reference them in the `AWS::Serverless::Function` properties section of the Lambda function.
- Use the `aws configure` command with the `--profile` parameter to add a named profile with the sandbox AWS account's credentials.
- Run the function using `sam local invoke` with the `--parameter-overrides` parameter.
- Create an AWS SAM CLI configuration file at the root of the SAM project folder. Add the `AWS_ACCESS_KEY_ID` and `AWS_SECRET_ACCESS_KEY` environment variables to it.

Incorrect

AWS Lambda functions have an associated execution role that provides permissions to interact with other AWS services. However, when you run AWS Lambda functions locally using the SAM CLI, you're simulating the execution environment of the Lambda function but not replicating the AWS execution context, including the IAM execution role. This means that the function won't automatically assume any IAM execution role and instead will rely on the credentials stored in `~/.aws/credentials` file.

When testing locally with AWS SAM, you can specify a named profile from your AWS CLI configuration using the `--profile` parameter with the `sam local invoke` command. This will instruct the SAM CLI to use the credentials from the specified profile when invoking the Lambda function. You can run the `aws configure` with the `--profile` option to set the credentials for a named profile.

```
D:\>aws configure --profile tdojo-sandbox
AWS Access Key ID [None]: AKIAVZGOFAXEXAMPLE
AWS Secret Access Key [None]: wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY
Default region name [None]: us-east-1
Default output format [None]: json
```

In the scenario, the developer must first set up the sandbox AWS account's credentials using `aws configure --profile sandbox`. This creates a named profile 'sandbox' (note that you can use any name for the profile). For local testing with the SAM CLI, the developer can then specify this profile using the command `sam local invoke --profile sandbox`. This ensures that the locally executed Lambda function utilizes the correct credentials to access resources in the sandbox AWS account.

Hence, the correct answers are:

- Use the `aws configure` command with the `--profile` parameter to add a named profile with the sandbox AWS account's credentials.
- Run the function using `sam local invoke` with the `--profile` parameter.

The option that says: **Create an AWS SAM CLI configuration file at the root of the SAM project folder. Add the `AWS_ACCESS_KEY_ID` and `AWS_SECRET_ACCESS_KEY` environment variables to it** is incorrect. The SAM CLI relies on the AWS credentials stored in the `/.aws/credentials` file, which can be set through the `aws configure` command. While it's technically possible to place application credentials in a configuration file, SAM CLI doesn't support sourcing AWS credentials from it for authentication.

The option that says: **Add the AWS credentials of the sandbox AWS account to the `Globals` section of the `template.yml` file and reference them in the `AWS::Serverless::Function` properties section of the Lambda function** is incorrect. The `Globals` section in a SAM `template.yml` is primarily used for setting properties that apply to all AWS resources of a certain type. It's not a storage location for AWS credentials. Moreover, the `AWS::Serverless::Function` resource property does not have fields for AWS credentials. Even if you were to add the credentials as environment variables, it still wouldn't grant the locally running function the permissions associated with those credentials.

The option that says: **Run the function using `sam local invoke` with the `--parameter-overrides` parameter** is incorrect. The `--parameter-overrides` option is typically used to change template parameters during local testing. For instance, if you had a parameter in your SAM template for setting an environment variable, the `--parameter-overrides` option would allow you to test with different values for those parameters. Still, it does not interact with nor modify AWS credentials.

References:

<https://docs.aws.amazon.com/serverless-application-model/latest/developerguide/sam-cli-command-reference-sam-local-invoke.html>
<https://aws.amazon.com/blogs/aws/aws-serverless-application-model-sam-command-line-interface-build-test-and-debug-serverless-apps-locally/>
<https://docs.aws.amazon.com/cli/latest/userguide/cli-configure-files.html>

Check out this AWS SAM Cheat Sheet:

<https://tutorialsdojo.com/aws-serverless-application-model-sam/>

22. Question

Your serverless AWS Lambda functions are integrated with Amazon API gateway using Lambda proxy integration. The API caching feature is enabled in the API Gateway with a TTL value of 300 seconds. A client would like to fetch the latest data from your endpoints every time a request is sent and invalidate the existing cache.

What should the client do in order to get the latest data?

- Modify cache TTL value to a shorter period.
- Have the client send a request with the `Cached: false` header.
- Override API caching by allowing the client to send requests to the endpoint directly.
- Have the client send a request with the `Cache-Control: max-age=0` header.

Correct

A client of your API can invalidate an existing cache entry and reload it from the integration endpoint for individual requests. The client must send a request that contains the `Cache-Control: max-age=0` header.

test Stage Editor

Delete Stage

Invoke URL: <https://b.execute-api.us-east-1.amazonaws.com/test>

Settings Stage Variables SDK Generation Export Deployment History

Configure the metering and caching settings for the **test** stage.

Cache Settings

Cache status AVAILABLE

Flush entire cache

Enable API cache ☒

Enabling API cache increases cost and is not covered by the free tier. [See pricing for more details](#)

Cache capacity 0.5GB

Encrypt cache data ☒

Cache time-to-live (TTL) 300

Per-key cache invalidation

Require authorization ☒

Handle unauthorized requests ☒ ignore cache control header, Add a warning in response header

CloudWatch Settings

Enable CloudWatch Logs ☒

The client receives the response directly from the integration endpoint instead of the cache, provided that the client is authorized to do so. This replaces the existing cache entry with the new response, which is fetched from the integration endpoint.

Modifying the TTL value for the cached data to a lower value is incorrect because there is still no guarantee that the client will submit a request after the cache has expired. Also, you will not be fully utilizing the purpose of API caching since new data will be fetched from the endpoint more often. The best solution for this scenario is to use the Cache-Control header instead.

Allowing the client to access the endpoint directly is incorrect because the purpose of placing API Gateway in-front of your endpoints is to not expose your endpoints to the public and risk security issues. It also provides you the additional benefits of not burdening your endpoints with a massive number of requests and allowing developer-friendly data exchanges through APIs.

Having the client send a request with the `Cached: false` header is incorrect because this is a custom header. The correct way is to configure the `Cache-Control: max-age=0` header instead.

Reference:

<https://docs.aws.amazon.com/apigateway/latest/developerguide/api-gateway-caching.html#override-api-gateway-stage-cache-for-method-cache>

Check out this Amazon API Gateway Cheat Sheet:

<https://tutorialsdojo.com/amazon-api-gateway/>

23. Question

An application experiences a sluggish response whenever there is a surge in requests involving read queries. The developer has already attempted to improve performance by optimizing the queries. However, the problem still persists even after applying the change. The application is hosted in an Amazon ECS Cluster and uses a MySQL database backed by Amazon RDS.

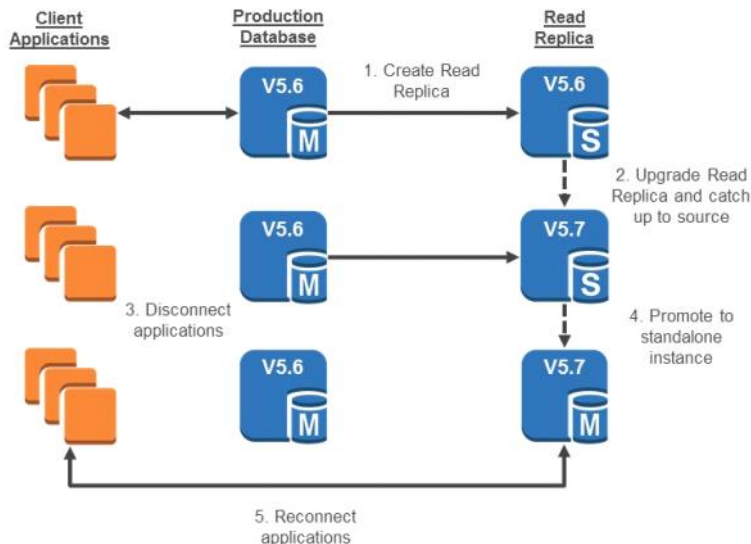
Which of the following could the developer do to resolve the performance issue? (Select TWO.)

- Replace the database with Amazon MemoryDB for Redis
- Set up read replicas for the RDS database instance and route read queries to these replicas.
- Cache the database response using Amazon CloudFront.
- Implement a Multi-AZ deployment configuration for the RDS DB instance.
- Implement database caching using Amazon ElastiCache.

Correct

Amazon RDS Read Replicas provide enhanced performance and durability for the database (DB) instances. This feature makes it easy to elastically scale out beyond the capacity constraints of a single DB instance for read-heavy database workloads. You can create one or more replicas of a given source DB instance and serve high-volume application read traffic from multiple copies of your data, thereby increasing aggregate read throughput.

You can reduce the load on your source DB instance by routing read queries from your applications to the read replica. Read replicas allow you to elastically scale out beyond the capacity constraints of a single DB instance for read-heavy database workloads.



Because read replicas can be promoted to master status, they are useful as part of a sharding implementation. To shard your database, add a read replica and promote it to master status, then, from each of the resulting DB Instances, delete the data that belongs to the other shard.

In-memory data caching can be one of the most effective strategies to improve your overall application performance and reduce your database costs. Caching can be applied to any type of database, including relational databases such as Amazon RDS or NoSQL databases such as Amazon DynamoDB, MongoDB, and Apache Cassandra. The best part of caching is that it's minimally invasive to implement, and by doing so, your application performance regarding both scale and speed is dramatically improved.

Amazon ElastiCache offers fully managed Redis and Memcached. Seamlessly deploy, run, and scale popular open-source compatible in-memory data stores. Build data-intensive apps or improve the performance of your existing apps by retrieving data from high throughput and low latency in-memory data stores.

Hence, the correct answers in this scenario are:

- Set up read replicas for the RDS database instance and route read queries to these replicas.
- Implement database caching using Amazon ElastiCache.

The option that says: **Replace the database with Amazon MemoryDB for Redis** is incorrect because Redshift is primarily used for online analytics processing applications (OLAP) and as a data warehouse. Hence, this will not improve the read performance of your application.

The option that says: **Cache the database response using Amazon CloudFront** is incorrect. Although CloudFront can provide caching and for CDN, it is not suitable to be used for database caching. Using Read Replicas and ElastiCache are more appropriate features to be used in this scenario.

The option that says: **Implement a Multi-AZ deployment configuration for the RDS DB instance** is incorrect because configuring a Multi-AZ RDS just improves the availability of the database but does not drastically improve the read performance, which Read Replicas can provide.

References:

- <https://aws.amazon.com/caching/database-caching/>
- <https://aws.amazon.com/rds/details/read-replicas/>
- <https://aws.amazon.com/elasticache/>

Check out these Amazon RDS and ElastiCache Cheat Sheets:

- <https://tutorialsdojo.com/amazon-relational-database-service-amazon-rds/>
- <https://tutorialsdojo.com/amazon-elasticache/>

Tutorials Dojo's AWS Certified Developer Associate Exam Study Guide:

- <https://tutorialsdojo.com/aws-certified-developer-associate/>

24. Question

A developer is instructed to collect data on the number of times that web visitors click the advertisement link of a popular news website. A database entry containing the count will be incremented for every click. Given that the website has millions of readers worldwide, your database should be configured to provide optimal performance to capture all the click events.

What is the BEST service that the developer should implement in this scenario?

- Use Amazon RDS for the database and setup SQL AUTO_INCREMENT on your tables.
- Set up Amazon DynamoDB for the database and implement atomic counters for UpdateItem operation of the website counter.
- Take advantage of Amazon Aurora's performance speed and AUTO_INCREMENT feature for item updates.
- Launch an Amazon Redshift for the database and apply a step count of 1 for the

IDENTITY column.

Correct

Amazon DynamoDB is a key-value and document database that delivers single-digit millisecond performance at any scale. Since fast performance is one of the requirements asked in the scenario, DynamoDB should be an option to consider.

In DynamoDB, an *item* is a collection of attributes. Each attribute has a name and a value. An attribute value can be a scalar, a set, or a document type. DynamoDB provides four operations for basic create/read/update/delete (CRUD) functionality:

PutItem – create an item.

GetItem – read an item.

UpdateItem – update an item.

DeleteItem – delete an item.

You can use the UpdateItem operation to implement an *atomic counter*—a numeric attribute that is incremented, unconditionally, without interfering with other write requests. With an atomic counter, the numeric value will increment each time you call UpdateItem.

```
aws dynamodb update-item \
  --table-name ProductCatalog \
  --key '{"Id": { "N": "601" }}' \
  --update-expression "SET Price = Price + :incr" \
  --expression-attribute-values '{":incr": {"N": "5" }}' \
  --return-values UPDATED_NEW
```

For example, you might use an atomic counter to keep track of the number of visitors to a website. In this case, your application would increment a numeric value, regardless of its current value. If an UpdateItem operation should fail, the application could simply retry the operation. This would risk updating the counter twice, but you could probably tolerate a slight overcounting or undercounting of website visitors.

Hence, the correct answer is to **setup Amazon DynamoDB for the database and implement atomic counters for the UpdateItem operation of the website counter.**

Using Amazon RDS for the database and setting up SQL AUTO_INCREMENT on your tables is incorrect because RDS is not scalable enough to handle millions of data being submitted by readers worldwide. Auto-increment allows a unique number to be generated automatically when a new record is inserted into a table. This is often the primary key field that we would like to be created automatically every time a new record is inserted. Since you would not want to add a new database entry for every link click and immediately consume all your storage space, it would be better to use DynamoDB's atomic counter instead.

Launching an Amazon Redshift for the database and applying a step count of 1 for the IDENTITY column is incorrect because Redshift is more suited for data warehousing demands that need parallel execution capabilities and columnar storage types.

Taking advantage of Amazon Aurora's performance speed and AUTO_INCREMENT feature for item updates is incorrect. Although Aurora is a scalable database service, using the AUTO_INCREMENT feature of SQL does not suit the scenario's requirement. Auto-increment simply allows a unique number to be generated automatically when a new record is inserted into a table.

References:

<https://aws.amazon.com/dynamodb/>

<https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/WorkingWithItems.html>

[#WorkingWithItems.AtomicCounters](#)

https://docs.aws.amazon.com/redshift/latest/dg/r_CREATE_TABLE_NEW.html

Check out this Amazon DynamoDB Cheat Sheet:

<https://tutorialsdjojo.com/amazon-dynamodb/>

25. Question

An application hosted in a multicontainer Docker platform in Elastic Beanstalk uses DynamoDB to handle the session data of its users. These data are only used in a particular timeframe and the stale data can be deleted after the user logged out of the system.

Which of the following is the most suitable way to delete the session data?

- Enable TTL for the session data in the DynamoDB table.
- Use `atomic counters` to track the validity of the session data and delete once it becomes stale.
- Delete the stale data by regularly performing a scan on the table.
- Use `conditional writes` to add the session data to the DynamoDB table and then automatically delete it based on the condition you specify.

Incorrect

Time To Live (TTL) for DynamoDB allows you to define when items in a table expire so that they can be automatically deleted from the database.

TTL is provided at no extra cost as a way to reduce storage usage and reduce the cost of storing irrelevant data without using provisioned throughput. With TTL enabled on a table, you can set a timestamp for deletion on a per-item basis, allowing you to limit storage usage to only those records that are relevant.

Enable Time to Live (TTL) [Info](#)

TTL settings

TTL attribute name
The name of the attribute that will be stored in the TTL timestamp.

Between 1 and 255 characters.

Preview
Confirm that your TTL attribute and values are working properly by specifying a date and time, and reviewing a sample of the items that will be deleted by then. Note that preview may show only some of the relevant items.

Simulated date and time
Specify the date and time to simulate which items would be expired.
Epoch time value September 24, 2021, 24:51:57 (UTC+08:00)

Enabling TTL can take up to one hour to be applied across all partitions. You will not be able to make additional TTL changes until this update is complete.

TTL is useful if you have continuously accumulated data that lose relevance after a specific time period. For example session data, event logs, usage patterns, and other temporary data. If you have sensitive data that must be retained only for a certain amount of time according to contractual or regulatory obligations, TTL helps you ensure that it is removed promptly and as scheduled.

Therefore, the correct answer in this scenario is to: **Enable TTL for the session data in the DynamoDB table.**

The option that says: **Delete the stale data by regularly performing a scan on the table** is incorrect because the Scan operation uses *eventually consistent* reads when accessing the data in a table and therefore, the result set might not include the changes to data in the table immediately before the operation began. This is an inefficient option that can simply be replaced by using TTL.

The option that says: **Use atomic counters to track the validity of the session data and deleting it once becomes stale** is incorrect because atomic counters are primarily used in updating data and for scenarios where you want the updates to not be idempotent.

The option that says: **Use conditional writes to add the session data to the DynamoDB table and then automatically deleting it based on the condition you specify** is incorrect because conditional writes are only helpful in cases where multiple users attempt to modify the same item.

References:

<https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/TTL.html>

<https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/WorkingWithItems.html>

Check out this Amazon DynamoDB Cheat Sheet:

<https://tutorialsdojo.com/amazon-dynamodb/>

Tutorials Dojo's AWS Certified Developer Associate Exam Study Guide:

<https://tutorialsdojo.com/aws-certified-developer-associate/>

26. Question

Your request to increase your account's concurrent execution limit to 2000 has been recently approved by AWS. There are 10 Lambda functions running in your account and you already specified a concurrency execution limit on one function at 400 and on another function at 200.

Which of the following statements are TRUE in this scenario? (Select TWO.)

- The remaining 1400 concurrent executions will be shared among the other 8 functions.
- The combined allocated 600 concurrent execution will be shared among the 2 functions.
- The unreserved concurrency pool is 600.
- You can still set a concurrency execution limit of 1300 to a third Lambda function.
- You can still set a concurrency execution limit of 1400 to a third Lambda function.

Incorrect

The unit of scale for AWS Lambda is a concurrent execution. However, scaling indefinitely is not desirable in all scenarios. For example, you may want to control your concurrency for cost reasons or to regulate how long it takes you to process a batch of events, or to simply match it with a downstream resource. To assist with this, Lambda provides a concurrent execution limit control at both the account level and the function level.

The *concurrent executions* refer to the number of executions of your function code that are happening at any given time. You can estimate the concurrent execution count, but the concurrent execution count will differ depending on whether or not your Lambda function is processing events from a poll-based event source.

Concurrency

Unreserved account concurrency 100

☐ Use unreserved account concurrency

☒ Reserve concurrency

The unreserved account concurrency can't go below 100.

Tutorials Dojo

If you create a Lambda function to process events from event sources that aren't poll-based (for example, Lambda can process every event from other sources, like Amazon S3 or API Gateway), each published event is a unit of work, in parallel, up to your account limits. Therefore, the

number of invocations these event sources make influences the concurrency. If you set the concurrent execution limit for a function, the value is deducted from the unreserved concurrency pool. For example, if your account's concurrent execution limit is 1000 and you have 10 functions, you can specify a limit on one function at 200 and another function at 100. The remaining 700 will be shared among the other 8 functions. AWS Lambda will keep the unreserved concurrency pool at a minimum of 100 concurrent executions so that functions that do not have specific limits set can still process requests. So, in practice, if your total account limit is 1000, you are limited to allocating 900 to individual functions.

In this scenario, you still have 1400 concurrent executions remaining which will be shared by the other 8 Lambda functions in your AWS account. Take note that the unreserved account concurrency can't go below 100, which means that you only set a concurrency execution limit of 1300 to a single function or spread out to the remaining 8 functions.

Hence, the correct answers in this scenario are:

– The remaining 1400 concurrent executions will be shared among the other 8 functions.

– You can still set a concurrency execution limit of 1300 to a third Lambda function.

The option that says: **the unreserved concurrency pool is 600** is incorrect because this is the value of the total reserved concurrency that you have allocated to the 2 Lambda functions.

The option that says: **you can still set a concurrency execution limit of 1400 to a third Lambda function** is incorrect because the unreserved account concurrency cannot go below 100, which means that you only set a concurrency execution limit of 1300 to the third function or spread out to the remaining 8 functions.

The option that says: **the combined allocated 600 concurrent execution will be shared among the 2 functions** is incorrect because the execution limit is per function only and will not be shared with other functions, which also have reserved concurrent executions.

References:

<https://docs.aws.amazon.com/lambda/latest/dg/concurrent-executions.html>

<https://docs.aws.amazon.com/lambda/latest/dg/scaling.html>

Check out this AWS Lambda Cheat Sheet:

<https://tutorialsdjojo.com/aws-lambda/>

Tutorials Dojo's AWS Certified Developer Associate Exam Study Guide:

<https://tutorialsdjojo.com/aws-certified-developer-associate/>

27. Question

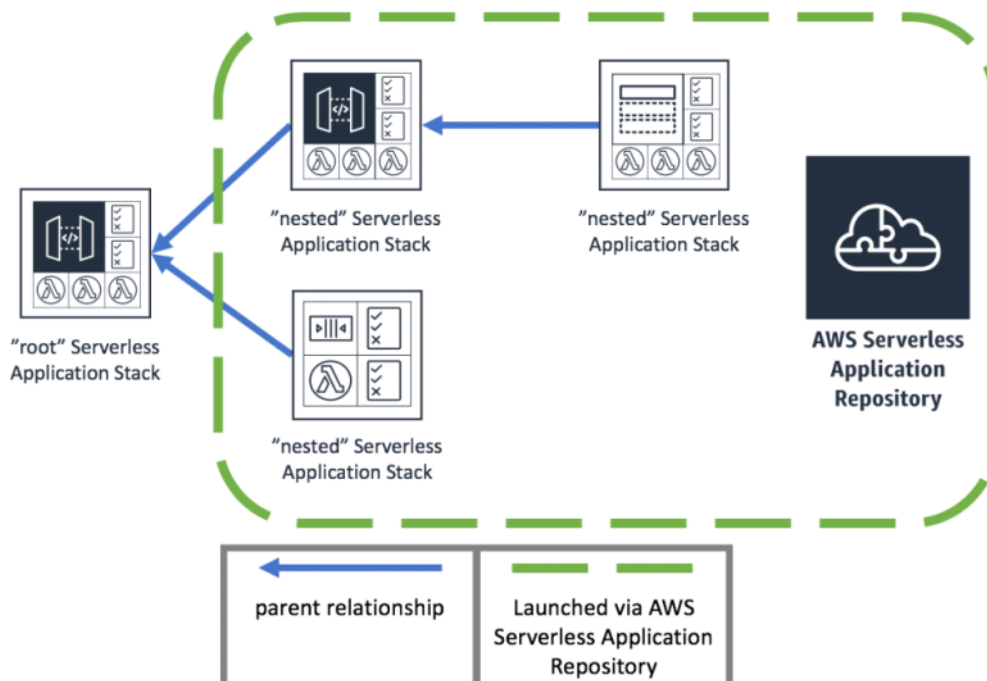
A developer has just finished writing a serverless application using AWS SAM (Serverless Application Model) on a local machine. There is a SAM template ready and the corresponding Lambda function code in a directory. The developer now wants to deploy this application to AWS.

Which combination of steps should the developer follow to successfully deploy the SAM application? (Select THREE)

- Build the SAM template in an Amazon EC2 instance.
- Build the SAM template in the local environment
- Deploy the SAM template from AWS CodePipeline.
- Deploy the SAM template from an Amazon S3 bucket.
- Build the SAM template using the AWS SDK for AWS CodeDeploy.
- Package the SAM application for deployment.

Correct

AWS SAM uses AWS CloudFormation as the underlying deployment mechanism. You can deploy your application by using AWS SAM command line interface (CLI) commands. You can also use other AWS services that integrate with AWS SAM to automate your deployments.



The typical AWS SAM deployment workflow starts with the `sam build` command, which compiles source code and readies deployment artifacts. Once built for deployment, the SAM template and the associated artifacts need to be stored in an S3 bucket. The `sam deploy` command takes care of this by first uploading the CloudFormation template to the S3 bucket. Though historically, the `sam package` command was used for this purpose, it's become somewhat legacy, as `sam deploy`, now implicitly handles the packaging. Once the template is in the S3 bucket, AWS CloudFormation references it to create or update the defined resources.

Hence, the correct answers are:

– Build the SAM template in the local environment

– Package the SAM application for deployment.

– Deploy the SAM template from an Amazon S3 bucket.

The option that says: **Deploy the SAM template from AWS CodePipeline** is incorrect. AWS CodePipeline is primarily a continuous integration and continuous delivery (CI/CD) service that automates the build, test, and deploy phases of your release process. While CodePipeline can

deploy SAM applications, it is not a required step for a local SAM deployment workflow. The option that says: **Build the SAM template using the AWS SDK for AWS CodeDeploy** is incorrect. The AWS SDK for CodeDeploy is typically used for management operations of the CodeDeploy service, not for building SAM templates. Building the SAM application is a separate process, typically done using the SAM CLI. The option that says: **Build the SAM template in an Amazon EC2 instance** is incorrect. This option is unnecessary. While you can technically build on an EC2 instance, it's not a requirement for SAM deployment. In the scenario, there's no condition that warrants the use of an EC2 instance.

References:

<https://docs.aws.amazon.com/serverless-application-model/latest/developerguide/serverless-deploying.html>
<https://docs.aws.amazon.com/serverlessrepo/latest/devguide/what-is-serverlessrepo.html>

Check out this AWS SAM Cheat Sheet:

<https://tutorialsdojo.com/aws-serverless-application-model-sam/>

28. Question

A serverless application, which is composed of multiple Lambda functions, has been deployed using AWS SAM. A developer was instructed to easily manage the deployments of the functions using CodeDeploy. When there is a new deployment, 10 percent of the incoming traffic should be shifted to the new version every 10 minutes until all traffic is shifted from the old version.

What should the developer do to properly deploy the functions that satisfies this requirement?

- Deploy the functions using a Canary deployment configuration.
- Deploy the functions using a Linear deployment configuration.
- Deploy the functions using an Immutable deployment configuration.
- Deploy the functions using an All-at-once deployment configuration.

Incorrect

CodeDeploy is a deployment service that automates application deployments to Amazon EC2 instances, on-premises instances, serverless Lambda functions, or Amazon ECS services. CodeDeploy can deploy application content that runs on a server and is stored in Amazon S3 buckets, GitHub repositories, or Bitbucket repositories. CodeDeploy can also deploy a serverless Lambda function. You do not need to make changes to your existing code before you can use CodeDeploy.

CodeDeploy supports the following deployment configurations:

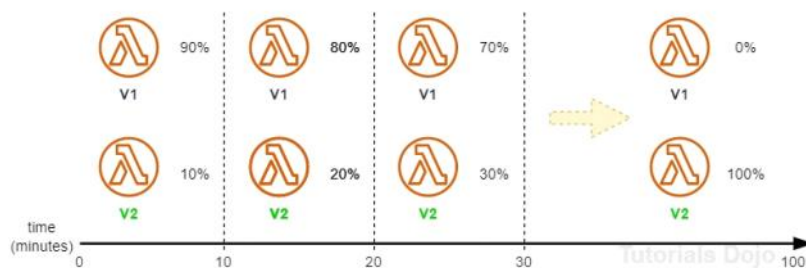
-**In-place (for EC2/On-premises)** – the application on each instance in the deployment group is stopped, the latest application revision is installed, and the new version of the application is started and validated.

-**Canary (for Lambda/ECS)** – traffic is shifted in two increments. You can choose from predefined canary options that specify the percentage of traffic shifted to your updated Lambda function or ECS task set in the first increment and the interval, in minutes, before the remaining traffic is shifted in the second increment.

-**Linear (for Lambda/ECS)** – traffic is shifted in equal increments with an equal number of minutes between each increment. You can choose from predefined linear options that specify the percentage of traffic shifted in each increment and the number of minutes between each increment.

-**All-at-once (for Lambda/ECS)** – all traffic is shifted from the original Lambda function or ECS task set to the updated function or task set all at once.

CodeDeployDefault.LambdaLinear10PercentEvery10Minutes



In a Linear deployment configuration, the traffic will be shifted in equal increments with an equal number of minutes between each increment. You can choose from predefined linear options that specify the percentage of traffic shifted in each increment and the number of minutes between each increment.

Hence, the correct answer is: **Deploy the functions using a Linear deployment configuration.**

The option that says: **Deploy the functions using a Canary deployment configuration** is incorrect because this will cause the traffic to be shifted in **two** increments. You can choose from predefined canary options that specify the percentage of traffic shifted to your updated Lambda function version in the first increment and the interval, in minutes, before the remaining traffic is shifted in the second increment.

The option that says: **Deploy the functions using an All-at-once deployment configuration** is incorrect because, with this deployment configuration, the traffic is shifted from the original Lambda function to the updated Lambda function version all at once.

The option that says: **Deploy the functions using an Immutable deployment configuration** is incorrect because this is only applicable in Elastic Beanstalk and not to Lambda.

References:

<https://docs.aws.amazon.com/codedeploy/latest/userguide/deployment-configurations.html>
<https://docs.aws.amazon.com/codedeploy/latest/userguide/welcome.html>

Check out this AWS CodeDeploy Cheat Sheet:

<https://tutorialsdojo.com/aws-codedeploy/>

29. Question

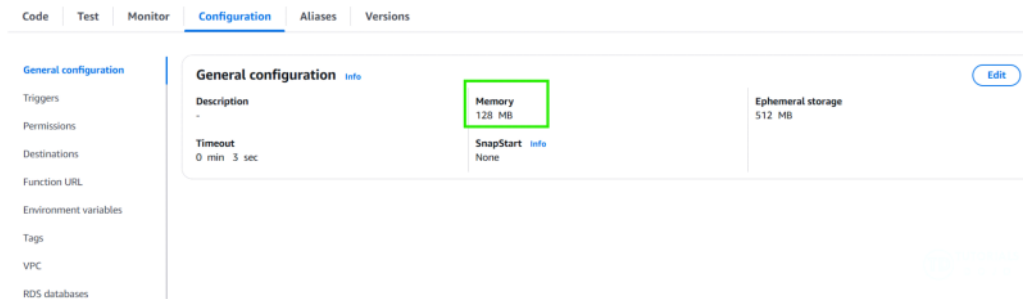
A developer is building an image processing utility using an AWS Lambda function. The function processes images in parallel using multiple threads to optimize performance. The images are stored in an Amazon S3 bucket and retrieved for processing. However, the function is not performing as efficiently as expected, with the processing time taking longer than anticipated, even when handling relatively small images.

Which action should the developer modify to achieve better performance in the AWS Lambda function?

- Optimize memory allocation for the Lambda function.
- Use AWS Step Functions to split tasks into smaller workflows.
- Utilize Amazon S3 Transfer Acceleration for image uploads.
- Increase the timeout setting of the Lambda function.

Correct

AWS Lambda is a serverless compute service that allows developers to run code without provisioning or managing servers. It automatically scales based on the workload and charges only for the compute time consumed. Developers can use Lambda to execute code in response to events such as changes in data, HTTP requests, or system state changes, making it ideal for event-driven architectures. Lambda supports multiple programming languages and integrates seamlessly with other AWS services, enabling flexible and scalable application development. AWS Lambda functions operate within a highly available infrastructure and manage resources automatically, ensuring reliability and performance. Lambda can execute specific business logic by using triggers like S3 events, DynamoDB streams, or API Gateway, making it a key component for building modern, agile applications.



AWS Lambda allows developers to configure memory allocation for 128 MB to 10,240 MB functions. This memory setting directly influences the CPU resources available to the function, as Lambda allocates CPU power proportionally to the configured memory. For instance, at 1,769 MB, a function has the equivalent of one vCPU. Increasing the memory allocation provides more RAM and enhances CPU capacity, which can lead to significant performance improvements for compute-intensive tasks.

Hence, the correct answer is: **Optimize memory allocation for the Lambda function.**

The option that says: **Use AWS Step Functions to split tasks into smaller workflows** is incorrect. AWS Step Functions are primarily used for orchestrating workflows and breaking down complex processes into smaller, manageable steps. However, this approach does not directly improve the execution performance of the Lambda function itself. The issue lies in the Lambda function's CPU resources, which Step Functions simply cannot address. While they can enhance task coordination, they typically do not optimize the speed of underlying tasks within a single function. The option that says: **Increase the timeout setting of the Lambda function** is incorrect. This option primarily focuses on extending the maximum runtime for the Lambda function. Increasing the timeout setting would allow the function to run longer but not address the underlying inefficiencies caused by insufficient memory or CPU resources. Timeout adjustments are typically useful for handling long-running tasks, not optimizing compute-intensive workloads.

The option that says: **Utilize Amazon S3 Transfer Acceleration for image uploads** is incorrect. Amazon S3 Transfer Acceleration is designed to improve the upload and download speed of objects to and from S3 by using Amazon's global edge network. However, this feature is only relevant when data transfer speed between the client and S3 is a bottleneck. In this case, the issue lies with the processing of images within the Lambda function. Transfer Acceleration simply cannot influence the performance of compute tasks, as it is unrelated to the Lambda execution environment.

References:

<https://docs.aws.amazon.com/lambda/latest/dg/best-practices.html>
<https://docs.aws.amazon.com/lambda/latest/dg/lambda-runtime-environment.html>
<https://docs.aws.amazon.com/lambda/latest/dg/configuration-memory.html>

Check out this AWS Lambda Cheat Sheet:

<https://tutorialsdodo.com/aws-lambda/>

30. 30. Question

A web application running in Amazon Elastic Beanstalk reads and writes a large number of related items in DynamoDB and processes each item one at a time. The network overhead of these transactions causes degradation in the application's performance. You were instructed by your manager to quickly refactor the application but without introducing major code changes such as implementing concurrency management or multithreading.

Which of the following solutions is the EASIEST method to implement that will improve the application performance in a cost-effective manner?

- Refactor the application to use DynamoDB transactional read and write APIs .
- Use DynamoDB Batch Operations API for GET, PUT, and DELETE operations.
- Enable DynamoDB Streams.
- Upgrade the EC2 instances to a higher instance type.

Correct

For applications that need to read or write multiple items, DynamoDB provides the `BatchGetItem` and `BatchWriteItem` operations. Using these operations can reduce the number of network round trips from your application to DynamoDB. In addition, DynamoDB performs the individual read or write operations in parallel. Your applications benefit from this parallelism without having to manage concurrency or threading.

```
aws dynamodb batch-get-item \  
--request-items file:///request-items.json
```

The arguments for `--request-items` are stored in the file `request-items.json`:

```
{  
  "Thread": {  
    "Keys": [  
      {  
        "ForumName": {"S": "Amazon DynamoDB"},  
        "Subject": {"S": "DynamoDB Thread 1"}  
      },  
      {  
        "ForumName": {"S": "Amazon S3"},  
        "Subject": {"S": "S3 Thread 1"}  
      }  
    ],  
    "ProjectionExpression": "ForumName, Subject, LastPostedDateTime, Replies"  
  }  
}
```

The batch operations are essentially wrappers around multiple read or write requests. For example, if a `BatchGetItem` request contains five items, DynamoDB performs five `GetItem` operations on your behalf. Similarly, if a `BatchWriteItem` request contains two put requests and four delete requests, DynamoDB performs two `PutItem` and four `DeleteItem` requests.

In general, a batch operation does not fail unless *all* of the requests in the batch fail. For example, suppose you perform a `BatchGetItem` operation but one of the individual `GetItem` requests in the batch fails. In this case, `BatchGetItem` returns the keys and data from the `GetItem` request that failed. The other `GetItem` requests in the batch are not affected.

Hence, the correct answer is to use **DynamoDB Batch Operations API for GET, PUT, and DELETE operations** in this scenario.

Upgrading the EC2 instances to a higher instance type is incorrect because the network overhead is the one that affects application performance and not the compute capacity. This is due to multiple read and write requests performed as single operations on DynamoDB, instead of a Batch operation.

Enabling DynamoDB Streams is incorrect because a DynamoDB stream is just an ordered flow of information about changes to items in an Amazon DynamoDB table. When you enable a stream on a table, DynamoDB captures information about every modification to data items in the table. Apparently, this feature does not solve the application issue where there is a large volume of data being processed one by one, and not by batch.

Refactoring the application to use DynamoDB transactional read and write APIs is incorrect because the Amazon DynamoDB transactions feature just simplifies the developer experience of making coordinated, all-or-nothing changes to multiple items both within and across tables. Transactions provide atomicity, consistency, isolation, and durability (ACID) in DynamoDB, enabling you to maintain data correctness in your applications easily. Take note that every transactional read and write API call consumes high RCUs and WCUs, unlike eventual or strong consistency requests. Hence, this entails a significant increase in costs which contradicts the requirements of the scenario.

References:

<https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/WorkingWithItems.html#WorkingWithItems.ConditionalUpdate>
https://docs.aws.amazon.com/amazondynamodb/latest/APIReference/API_BatchGetItem.html

Check out this Amazon DynamoDB Cheat Sheet:

<https://tutorialsdojo.com/amazon-dynamodb/>

31. Question

There is a requirement to postpone the delivery of new messages to an SQS queue for a number of seconds. You must configure the queue to ensure that any messages that you send remain invisible to consumers for a duration of time specified.

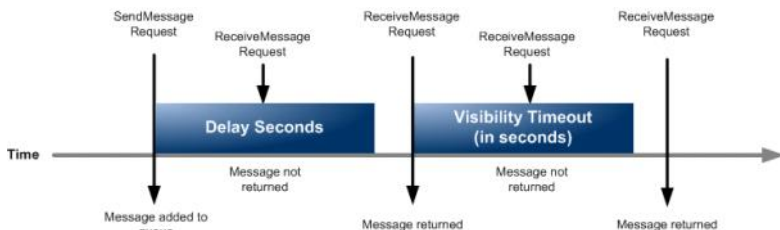
Which of the following SQS feature should you use to meet this requirement?

- Visibility Timeouts
- Delay Queue
- Long Polling
- Short Polling

Incorrect

Delay queues let you postpone the delivery of new messages to a queue for a number of seconds. If you create a delay queue, any messages that you send to the queue remain invisible to consumers for the duration of the delay period. The default (minimum) delay for a queue is 0 seconds. The maximum is 15 minutes.

Delay queues are similar to visibility timeouts because both features make messages unavailable to consumers for a specific period of time. The difference between the two is that, for delay queues, a message is hidden when it is first added to queue, whereas for visibility timeouts a message is hidden only after it is consumed from the queue.



To set delay seconds on individual messages rather than on an entire queue, use message timers to allow Amazon SQS to use the message timer's `DelaySeconds` value instead of the delay queue's `DelaySeconds` value.

Hence, the correct answer is to use a **Delay Queue**.

Short Polling is incorrect because this is just the default configuration of SQS that queries only a subset of its servers (based on a weighted random distribution), to determine whether any messages are available for a response.

Visibility Timeouts is incorrect because, with this configuration, a message is hidden only after it is consumed from the queue, and not before. Take note that the difference between the two is that, for delay queues, a message is hidden when it is first added to the queue, whereas for visibility timeouts a message is hidden only after it is consumed from the queue.

Long Polling is incorrect because this just helps reduce the cost of using Amazon SQS by eliminating the number of empty responses (when there are no messages available for a `ReceiveMessage` request) and false empty responses (when messages are available but aren't included in a response).

References:

<https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/sqs-delay-queues.html>
<https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/sqs-configure-delay-queue.html>

Check out this Amazon SQS Cheat Sheet:

<https://tutorialsdojo.com/amazon-sqs/>

32. Question

A company wants to centrally organize login credentials for its internal application. The application prompts users to change passwords every 35 days. Expired login credentials must be removed automatically, and an email notification should be sent to the users when their passwords are about to expire. A developer must create a solution with the least amount of development effort. Which solution meets the requirements?

- Use AWS Secrets Manager to store user credentials. Create a Lambda function that runs periodically to send Amazon SNS email notifications for passwords nearing expiration
- Store the credentials as Standard Parameters in AWS Systems Manager (SSM) Parameter Store and

configure `Expiration` and `ExpirationNotification` policies. Create an Amazon EventBridge rule that sends Amazon SNS email notifications.

- Use AWS Secret Managers to store user credentials and turn on automatic rotation.
- Store the credentials as Advanced Parameters in AWS Systems Manager (SSM) Parameter Store and configure `Expiration` and `ExpirationNotification` policies. Create an Amazon EventBridge rule that sends Amazon SNS email notifications.

Incorrect

Parameter Store, a capability of AWS Systems Manager, includes *standard parameters* and *advanced parameters*. You individually configure parameters to use either the standard-parameter tier (the default tier) or the advanced-parameter tier. You can change a standard parameter to an advanced parameter at any time, but you can't revert an advanced parameter to a standard parameter. This is because reverting an advanced parameter to a standard parameter would cause the system to truncate the size of the parameter from 8 KB to 4 KB, resulting in data loss. Reverting would also remove any policies attached to the parameter. Also, advanced parameters use a different form of encryption than standard parameters.

Parameter policies help you manage a growing set of parameters by allowing you to assign specific criteria to a parameter, such as an expiration date or *time to live*. Parameter policies are especially helpful in forcing you to update or delete passwords and configuration data stored in Parameter Store, a capability of AWS Systems Manager.

You can assign multiple policies to a parameter. For example, you can assign `Expiration` and `ExpirationNotification` policies so that the system initiates an EventBridge event to notify you about the impending deletion of a parameter. The `Expiration` policy lets you delete a parameter at a specified time while the `ExpirationNotification` policy is used to notify when a parameter is about to expire. These features are **only available for Advanced Parameters** in the AWS Systems Manager Parameter Store.

Hence, the correct answer is: **Store the credentials as Advanced Parameters in AWS Systems Manager (SSM) Parameter Store and configure `Expiration` and `ExpirationNotification` policies. Create an Amazon EventBridge rule that sends Amazon SNS email notifications.**

The option that says: **Use AWS Secrets Manager to store user credentials. Create a Lambda function that runs periodically to send Amazon SNS email notifications for passwords nearing expiration** is incorrect. Although it's possible to store login credentials in Secrets Manager, creating a cron-based Lambda function for checking password expiration and sending notifications via SNS takes more development overhead than simply using the `ExpirationNotification` policy in SSM Parameter Store.

The option that says: **Use AWS Secrets Manager to store user credentials and turn on automatic rotation** is incorrect. Automatic rotation in AWS Secrets Manager is typically used when secrets can be changed programmatically without user intervention (e.g, rotating database credentials). In the scenario's case, users must manually change their passwords.

The option that says: **Store the credentials as Standard Parameters in AWS Systems Manager (SSM) Parameter Store and configure `Expiration` and `ExpirationNotification` policies. Create an Amazon EventBridge rule that sends Amazon SNS email notifications** is incorrect because Standard Parameters do not support `Expiration` and `ExpirationNotification` policies.

References:

<https://docs.aws.amazon.com/systems-manager/latest/userguide/parameter-store-policies.html>
<https://docs.aws.amazon.com/systems-manager/latest/userguide/parameter-store-advanced-parameters.html>

Checkout this AWS Secrets Manager vs Systems Manager Parameter Store Cheat Sheet:
<https://tutorialsdojo.com/aws-secrets-manager/>

33. Question

A developer is launching a Lambda function that requires access to a MySQL RDS instance that is in a private subnet. Which of the following is the MOST secure way to achieve this?

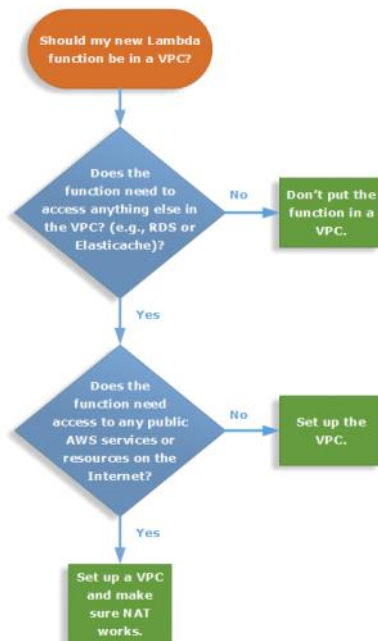
- Ensure that the Lambda function has proper IAM permission to access RDS.
- Expose an endpoint of your RDS to the Internet using an Elastic IP.
- Move your RDS instance to a public subnet.
- Configure the Lambda function to connect to your VPC.

Incorrect

You can configure a Lambda function to connect to a virtual private cloud (VPC) in your account. Use Amazon Virtual Private Cloud (Amazon VPC) to create a private network for resources such as databases, cache instances, or internal services. Connect your function to the VPC to access private resources during execution.

AWS Lambda runs your function code securely within a VPC by default. However, to enable your Lambda function to access resources inside your private VPC, you must provide additional VPC-specific configuration information that includes VPC subnet IDs and security group IDs. AWS Lambda uses this information to set up elastic network interfaces (ENIs) that enable your function to connect securely to other resources within your private VPC.

The following diagram guides you through a decision tree as to whether you should use a VPC (Virtual Private Cloud):



Don't put your Lambda function in a VPC unless you have to. There is no benefit outside of using this to access resources you cannot expose publicly, like a private Amazon Relational Database instance. Services like Amazon OpenSearch Service can be secured over IAM with access policies, so exposing the endpoint publicly is safe and wouldn't require you to run your function in the VPC to secure it.

Hence, **configuring the Lambda function to connect to your VPC** is the correct answer for this scenario.

Ensuring that the Lambda function has proper IAM permission to access RDS is incorrect. Even though you grant the necessary IAM permissions to the Lambda function to access RDS, the function would still not be able to connect to RDS since there is no established connection between Lambda and the private subnet of your VPC.

Exposing an endpoint of your RDS to the Internet using an Elastic IP is incorrect because this is not the most secure way of granting access to your Lambda function. It will be able to connect to RDS but so will the billions of people on the public Internet.

Moving your RDS instance to a public subnet is incorrect because this is an unnecessary change and not a best practice from a security perspective. You only need to configure your Lambda function to your VPC so it can connect to the RDS in the private subnet. If you move your RDS instance to a public subnet, it will introduce a critical security flaw to your entire architecture since your database will become accessible publicly.

References:

<https://docs.aws.amazon.com/lambda/latest/dg/best-practices.html#lambda-vpc>
<https://docs.aws.amazon.com/lambda/latest/dg/welcome.html>

Check out this AWS Lambda Cheat Sheet:

<https://tutorialsdojo.com/aws-lambda/>

34. Question

A leading insurance firm is hosting its customer portal in Elastic Beanstalk, which has an RDS database in AWS. The support team in your company discovered a lot of SQL injection attempts and cross-site scripting attacks on the portal, which is starting to affect the production environment.

Which of the following services should you implement to mitigate this attack?

- AWS Firewall Manager
- Amazon GuardDuty
- Network Access Control List
- AWS WAF

Incorrect

AWS WAF is a web application firewall that lets you monitor the HTTP and HTTPS requests that are forwarded to an Amazon API Gateway API, Amazon CloudFront or an Application Load Balancer. AWS WAF also lets you control access to your content. Based on conditions that you specify, such as the IP addresses that requests originate from or the values of query strings, API Gateway, CloudFront or an Application Load Balancer responds to requests either with the requested content or with an HTTP 403 status code (Forbidden). You also can configure CloudFront to return a custom error page when a request is blocked.



At the simplest level, AWS WAF lets you choose one of the following behaviors:

Allow all requests except the ones that you specify – This is useful when you want CloudFront or an Application Load Balancer to serve content for a public website, but you also want to block requests from attackers.

Block all requests except the ones that you specify – This is useful when you want to serve content for a restricted website whose users are readily identifiable by properties in web requests, such as the IP addresses that they use to browse to the website.

Count the requests that match the properties that you specify – When you want to allow or block requests based on new properties in web requests, you first can configure AWS WAF to count the

requests that match those properties without allowing or blocking those requests. This lets you confirm that you didn't accidentally configure AWS WAF to block all the traffic to your website. When you're confident that you specified the correct properties, you can change the behavior to allow or block requests.

Hence, the correct answer in this scenario is **AWS WAF**.

Amazon GuardDuty is incorrect because this is just a threat detection service that continuously monitors malicious activity and unauthorized behavior to protect your AWS accounts and workloads.

AWS Firewall Manager is incorrect because this just simplifies your AWS WAF and AWS Shield Advanced administration and maintenance tasks across multiple accounts and resources.

Network Access Control List is incorrect because this is an optional layer of security for your VPC that acts as a firewall for controlling traffic in and out of one or more subnets.

References:

<https://aws.amazon.com/waf/>

<https://docs.aws.amazon.com/waf/latest/developerguide/what-is-aws-waf.html>

<https://aws.amazon.com/blogs/security/three-most-important-aws-waf-rate-based-rules/>

Check out this AWS WAF Cheat Sheet:

<https://tutorialsdojo.com/aws-waf/>

35. Question

A developer is managing an application hosted in EC2, which stores data in an S3 bucket. The application also uses HTTPS for secure communication. To comply with the new security policy, the developer must ensure that the data is encrypted at rest using an encryption key that is provided and managed by the company. The change should also provide AES-256 encryption to their data. Which of the following actions could the developer take to achieve this? (Select TWO.)

- Use SSL to encrypt the data while in transit to Amazon S3.
- Implement Amazon S3 server-side encryption with AWS KMS Keys (SSE-KMS).
- Implement Amazon S3 server-side encryption with customer-provided keys (SSE-C).
- Implement Amazon S3 server-side encryption with Amazon S3-Managed Encryption Keys.
- Encrypt the data on the client-side before sending to Amazon S3 using their own master key.

Incorrect

Data protection refers to protecting data while in transit (as it travels to and from Amazon S3) and at rest (while it is stored on disks in Amazon S3 data centers). You can protect data in transit by using SSL or by using client-side encryption.

You have the following options for protecting data at rest in Amazon S3:

Use Server-Side Encryption – You request Amazon S3 to encrypt your object before saving it on disks in its data centers and decrypt it when you download the objects.

1. Use Server-Side Encryption with Amazon S3-Managed Keys (SSE-S3)

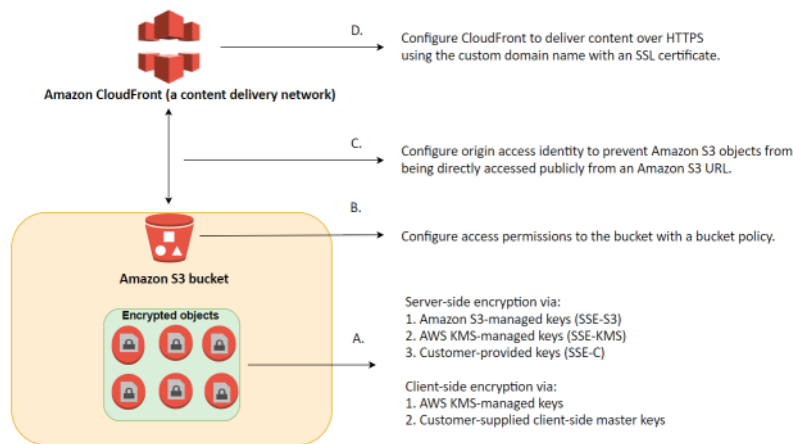
2. Use Server-Side Encryption with AWS KMS Keys (SSE-KMS)

3. Use Server-Side Encryption with Customer-Provided Keys (SSE-C)

Use Client-Side Encryption – You can encrypt data client-side and upload the encrypted data to Amazon S3. In this case, you manage the encryption process, the encryption keys, and related tools.

1. Use Client-Side Encryption with AWS KMS Key

2. Use Client-Side Encryption Using a Client-Side Master Key



Hence, the valid actions that the developer can implement in this scenario are:

– **Implement Amazon S3 server-side encryption with customer-provided keys (SSE-C)**

– **Encrypt the data on the client-side before sending to Amazon S3 using their own master key.**

Using SSL to encrypt the data while in transit to Amazon S3 is incorrect because the requirement is to only secure the data at rest and not data in transit. Hence, you have to use server-side encryption instead. Moreover, the scenario explicitly states that the application already uses HTTPS for secure communication.

Implementing Amazon S3 server-side encryption with AWS KMS Keys (SSE-KMS) is incorrect.

Although you can upload the company's KMS keys (CMKs), the keys will be managed by KMS and not your company. This does not comply with the security policy mandated by the company.

Implementing Amazon S3 server-side encryption with Amazon S3-Managed Encryption Keys is incorrect because the Amazon S3-Managed encryption does not comply with the policy mentioned in the given scenario since the keys are managed by AWS (through Amazon S3) and not by the company. The suitable server-side encryption that you should use here is SSE-C.

References:

<http://docs.aws.amazon.com/AmazonS3/latest/dev/UsingEncryption.html>

<https://docs.aws.amazon.com/AmazonS3/latest/dev/DataDurability.html>

Check out this Amazon S3 Cheat Sheet:

<https://tutorialsdojo.com/amazon-s3/>

36. Question

A developer is planning to use the AWS Elastic Beanstalk console to run the AWS X-Ray daemon on the EC2 instances in her application environment. She will use X-Ray to construct a service map to help identify issues with her application and to provide insight on which application component to optimize. The environment is using a default Elastic Beanstalk instance profile.

Which IAM managed policy does Elastic Beanstalk use for the X-Ray daemon to upload data to X-Ray?

- `AWSXrayReadOnlyAccess`

- **AWSXRayDaemonWriteAccess**
- **AWSXRayElasticBeanstalkWriteAccess**
- **AWSXrayFullAccess**

Incorrect

You can use **AWS Identity and Access Management (IAM)** to grant X-Ray permissions to users and compute resources in your account. IAM controls access to the X-Ray service at the API level to enforce permissions uniformly, regardless of which client (console, AWS SDK, AWS CLI) your users employ. To use the X-Ray console to view service maps and segments, you only need read permissions. To enable console access, add the **AWSXrayReadOnlyAccess** managed policy to your IAM user. For local development and testing, create an IAM user with read and write permissions. Generate access keys for the user and store them in the standard AWS SDK location. You can use these credentials with the X-Ray daemon, the AWS CLI, and the AWS SDK.



To deploy your instrumented app to AWS, create an IAM role with write permissions and assign it to the resources running your application. **AWSXRayDaemonWriteAccess** includes permission to upload traces, and some read permissions as well to support the use of sampling rules.

The read and write policies do not include permission to configure encryption key settings and sampling rules. Use **AWSXrayFullAccess** to access these settings, or add configuration APIs in a custom policy. For encryption and decryption with a customer-managed key that you create, you also need permission to use the key.

On supported platforms, you can use a configuration option to run the X-Ray daemon on the instances in your environment. You can enable the daemon in the Elastic Beanstalk console or by using a configuration file. To upload data to X-Ray, the X-Ray daemon requires IAM permissions in the **AWSXRayDaemonWriteAccess** managed policy. These permissions are included in the Elastic Beanstalk instance profile.

Hence, the correct answer is the **AWSXRayDaemonWriteAccess** managed policy.

AWSXrayReadOnlyAccess is incorrect because this policy is primarily used if you just want a read-only access to X-Ray.

AWSXrayFullAccess is incorrect. Although this can provide the required access to the daemon, this is not being used in Elastic Beanstalk as it does not abide by the standard security advice of granting the least privilege.

AWSXRayElasticBeanstalkWriteAccess is incorrect because this is not an available managed policy.

References:

<https://docs.aws.amazon.com/elasticbeanstalk/latest/dg/environment-configuration-debugging.html>
<https://docs.aws.amazon.com/xray/latest/devguide/xray-permissions.html>
<https://docs.aws.amazon.com/xray/latest/devguide/security.html>

Check out this AWS X-Ray Cheat Sheet:

<https://tutorialsdojo.com/aws-x-ray/>

Instrumenting your Application with AWS X-Ray:

<https://tutorialsdojo.com/instrumenting-your-application-with-aws-x-ray/>

37. Question

A developer is managing a distributed system that consists of an Application Load Balancer, an SQS queue, and an Auto Scaling group of EC2 instances. The system has been integrated with CloudFront to better serve clients worldwide. To enhance the security of the in-flight data, the developer was instructed to establish an end-to-end SSL connection between the origin and the end-users.

Which TWO options will allow the developer to meet this requirement using CloudFront? (Select TWO.)

- Configure the Origin Protocol Policy to use HTTPS only
- Configure your ALB to only allow traffic on port 443 using an SSL certificate from AWS Config.
- Set up an Origin Access Control (OAC) setting
- Associate a Web ACL using AWS Web Application Firewall (WAF) with your CloudFront Distribution.
- Configure the Viewer Protocol Policy to use HTTPS only

Incorrect

For web distributions, you can configure CloudFront to require that viewers use HTTPS to request your objects, so connections are encrypted when CloudFront communicates with viewers. You can also configure CloudFront to use HTTPS to get objects from your origin, so connections are encrypted when CloudFront communicates with your origin.

If you configure CloudFront to require HTTPS both to communicate with viewers and to communicate with your origin, here's what happens when CloudFront receives a request for an object. The process works basically the same way whether your origin is an Amazon S3 bucket or a custom origin such as an HTTP/S server:

1. A viewer submits an HTTPS request to CloudFront. There's some SSL/TLS negotiation here between the viewer and CloudFront. In the end, the viewer submits the request in an encrypted format.
2. If the object is in the CloudFront edge cache, CloudFront encrypts the response and returns it to the viewer, and the viewer decrypts it.
3. If the object is not in the CloudFront cache, CloudFront performs SSL/TLS negotiation with your origin and, when the negotiation is complete, forwards the request to your origin in an encrypted format.
4. Your origin decrypts the request, encrypts the requested object, and returns the object to CloudFront.
5. CloudFront decrypts the response, re-encrypts it, and forwards the object to the viewer. CloudFront also saves the object in the edge cache so that the object is available the next time it's requested.
6. The viewer decrypts the response.

You can configure one or more cache behaviors in your CloudFront distribution to require HTTPS for communication between viewers and CloudFront. You also can configure one or more cache behaviors to allow both HTTP and HTTPS, so that CloudFront requires HTTPS for some objects but not for others.

To implement this setup, you have to change the **Origin Protocol Policy** setting for the applicable origins in your distribution. If you're using the domain name that CloudFront assigned to your

distribution, such as `dtut0rial5d0j0.cloudfront.net`, you change the **Viewer Protocol Policy** setting for one or more cache behaviors to require HTTPS communication. With this configuration, CloudFront provides the SSL/TLS certificate.

Hence, the correct answers are: **Configure the Origin Protocol Policy to use HTTPS only** and **Configure the Viewer Protocol Policy to use HTTPS only** are correct answers in this scenario.

The option that says: **Configure your ALB to only allow traffic on port 443 using an SSL certificate from AWS Config** is incorrect because you can't store a certificate in AWS Config.

The option that says: **Set up an Origin Access Control (OAC) setting** is incorrect because this CloudFront feature only allows you to secure S3 origins by granting access to S3 buckets for designated CloudFront distributions. This method is applicable only to S3 origins and cannot be used to establish end-to-end SSL connections for other origins.

The option that says: **Associate a Web ACL using AWS Web Application Firewall (WAF) with your CloudFront Distribution** is incorrect because AWS WAF is primarily used to protect your web applications from common web exploits that could affect application availability, compromise security, or consume excessive resources. This will not allow you to establish an SSL connection between your origin and your clients.

References:

<https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/using-https.html>
<https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/using-https-cloudfront-to-custom-origin.html#using-https-cloudfront-to-origin-certificate>
<https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/using-https-viewers-to-cloudfront.html>

Check out this Amazon CloudFront Cheat Sheet:

<https://tutorialsdojo.com/amazon-cloudfront/>

38. Question

You are managing an application which is composed of an SQS queue and an Auto Scaling group of EC2 instances. Recently, your customers are complaining that there are a lot of incidents where their orders are being erroneously sent twice.

What should you do to rectify this problem?

- Use a FIFO (First-In-First-Out) Queue by disabling the content-based deduplication.
- Use a FIFO (First-In-First-Out) Queue and provide the Message Deduplication ID for each message.
- Use a Standard Queue and provide the Message Group ID for each message.
- Use a Standard Queue and provide the Message Deduplication ID for each message.

Incorrect

Unlike standard queues, FIFO queues don't introduce duplicate messages. FIFO queues help you avoid sending duplicates to a queue. If you retry the `SendMessage` action within the 5-minute deduplication interval, Amazon SQS doesn't introduce any duplicates into the queue.

To configure deduplication, you must do one of the following:

- Enable content-based deduplication. This instructs Amazon SQS to use a SHA-256 hash to generate the message deduplication ID using the body of the message – but not the attributes of the message.
- Explicitly provide the message deduplication ID (or view the sequence number) for the message.

Queue Attributes

Default Visibility Timeout ⓘ	30	seconds	Value must be between 0 seconds and 12 hours.
Message Retention Period ⓘ	4	days	Value must be between 1 minute and 14 days.
Maximum Message Size ⓘ	256	KB	Value must be between 1 and 256 KB.
Delivery Delay ⓘ	0	seconds	Value must be between 0 seconds and 15 minutes.
Receive Message Wait Time ⓘ	0	seconds	Value must be between 0 and 20 seconds.
Content-Based Deduplication ⓘ	☐		

Dead Letter Queue Settings

Use Redrive Policy ⓘ	☐		
Dead Letter Queue ⓘ		Value must be an existing queue name.	
Maximum Receives ⓘ		Value must be between 1 and 1000.	

Cancel

Create Queue

The message deduplication ID is the token used for deduplication of sent messages. If a message with a particular message deduplication ID is sent successfully, any messages sent with the same message deduplication ID are accepted successfully but aren't delivered during the 5-minute deduplication interval.

Message deduplication applies to an entire queue, not to individual message groups. Amazon SQS continues to keep track of the message deduplication ID even after the message is received and deleted.

Hence, the correct answer in this scenario is to **use a FIFO (First-In-First-Out) Queue and provide the Message Deduplication ID for each message**.

Using a FIFO (First-In-First-Out) Queue by disabling the content-based deduplication is incorrect. Although the use of FIFO queue is valid, it is wrong to disable the content-based deduplication.

This should be enabled to avoid duplicate messages in the queue.

Using a Standard Queue and providing the Message Group ID for each message is incorrect because you should use a FIFO queue instead to avoid duplicate messages.

Using a Standard Queue and providing the Message Deduplication ID for each message is incorrect. Although it is a valid answer to provide the Message Deduplication ID, this feature can't be enabled for Standard Queues. You have to use the FIFO queues instead for this scenario.

References:

<https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/FIFO-queues.html#FIFO-queues-exactly-once-processing>
<https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/using-message-deduplication-id-property.html>

Check out this Amazon SQS Cheat Sheet:

<https://tutorialsdojo.com/amazon-sqs/>

39. Question

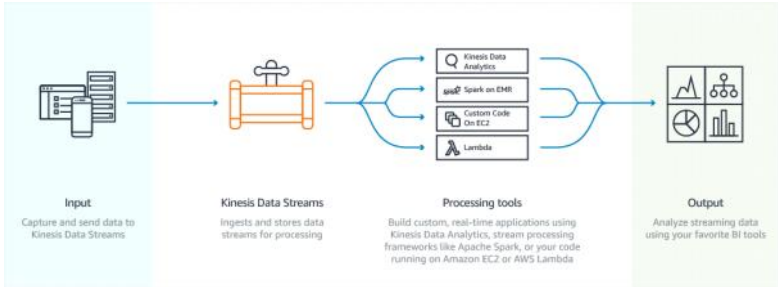
A clickstream application uses Amazon Kinesis Data Stream for real-time

processing. `PutRecord` API calls are being used by the producer to send data to the stream. However, there are cases where the producer intermittently restarted while doing the processing, which resulted in sending the same data twice to the stream. This inadvertently causes duplication of entries in the data stream, which affects the processing of the consumers. Which of the following should you implement to resolve this issue?

- Add more shards.
- Split shards of the data stream.
- Merge shards of the data stream.
- Embed a primary key within the record.

Correct

There are two primary reasons why records may be delivered more than one time to your Amazon Kinesis Data Streams application: producer retries and consumer retries. Your application must anticipate and appropriately handle processing individual records multiple times.



Consider a producer that experiences a network-related timeout after it makes a call to `PutRecord`, but before it can receive an acknowledgment from Amazon Kinesis Data Streams. The producer cannot be sure if the record was delivered to Kinesis Data Streams. Assuming that every record is important to the application, the producer would have written to retry the call with the same data. If both `PutRecord` calls on that same data were successfully committed to Kinesis Data Streams, then there will be two Kinesis Data Streams records. Although the two records have identical data, they also have unique sequence numbers. Applications that need strict guarantees should embed a primary key within the record to remove duplicates later when processing. Note that the number of duplicates due to producer retries is usually low compared to the number of duplicates due to consumer retries.

Hence, the correct answer in this scenario is to **embed a primary key within the record** to remove duplicates later when processing.
Adding more shards is incorrect because this is not a suitable solution for handling duplicate records in the Kinesis data stream. This is primarily used to increase the rate of data flowing through the stream.
Splitting shards of the data stream is incorrect because this is used to increase the capacity of the stream and not to avoid any duplicate data.
Merging shards of the data stream is incorrect because this is primarily used to make better use of the unused capacity in the stream and to save on costs.

References:
<https://docs.aws.amazon.com/streams/latest/dev/kinesis-record-processor-duplicates.html>
<https://docs.aws.amazon.com/streams/latest/dev/kinesis-record-processor-scaling.html>

Check out this Amazon Kinesis Cheat Sheet:
<https://tutorialsdjojo.com/amazon-kinesis/>

40. 40. Question

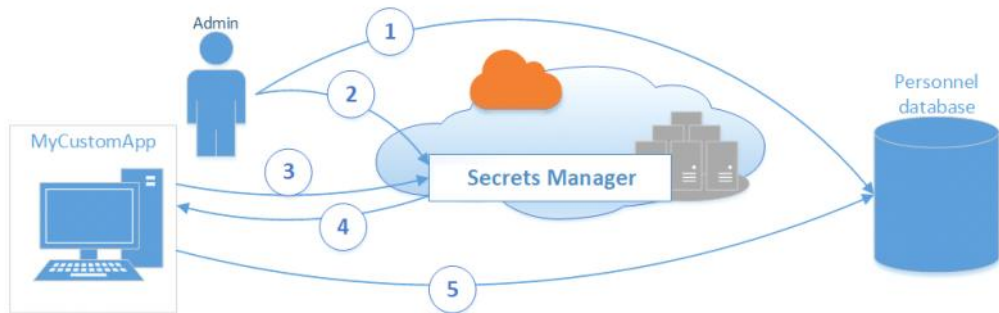
A company has a suite of web applications that is heavily using RDS database in Multi-AZ Deployments configuration with several Read Replicas. For improved security, you were instructed to ensure that all of their database credentials, API keys, and other secrets are encrypted and rotated on a regular basis. You should also configure your applications to use the latest version of the encrypted credentials when connecting to the RDS database.

Which of the following is the MOST appropriate solution to secure the credentials?

- Use AWS Secrets Manager to store and encrypt the credentials and enable automatic rotation.
- Store the credentials in AWS KMS.
- Store the credentials to Systems Manager Parameter Store with a `SecureString` data type.
- Store the credentials to AWS ACM.

Correct

AWS Secrets Manager is an AWS service that makes it easier for you to manage secrets. *Secrets* can be database credentials, passwords, third-party API keys, and even arbitrary text. You can store and control access to these secrets centrally by using the Secrets Manager console, the Secrets Manager command line interface (CLI), or the Secrets Manager API and SDKs. In the past, when you created a custom application that retrieves information from a database, you typically had to embed the credentials (the secret) for accessing the database directly in the application. When it came time to rotate the credentials, you had to do much more than just create new credentials. You had to invest time to update the application to use the new credentials. Then you had to distribute the updated application. If you had multiple applications that shared credentials and you missed updating one of them, the application would break. Because of this risk, many customers have chosen not to regularly rotate their credentials, which effectively substitutes one risk for another.



Secrets Manager enables you to replace hardcoded credentials in your code (including passwords), with an API call to Secrets Manager to retrieve the secret programmatically. This

helps ensure that the secret can't be compromised by someone examining your code, because the secret simply isn't there. Also, you can configure Secrets Manager to automatically rotate the secret for you according to a schedule that you specify. This enables you to replace long-term secrets with short-term ones, which helps to significantly reduce the risk of compromise.

Hence, **using AWS Secrets Manager to store and encrypt the credentials and enabling automatic rotation** is the most appropriate solution for this scenario.

Storing the credentials to Systems Manager Parameter Store with a SecureString data type is incorrect because, by default, Systems Manager Parameter Store doesn't rotate its parameters which is one of the requirements in the above scenario.

Storing the credentials to AWS ACM is incorrect because it is just a managed private CA service that helps you easily and securely manage the lifecycle of your private certificates to allow SSL communication to your application. This is not a suitable service to store database or any other confidential credentials.

Storing the credentials in AWS KMS is incorrect because this only makes it easy for you to create and manage encryption keys and control the use of encryption across a wide range of AWS services. This is primarily used for encryption and not for hosting your credentials.

References:

<https://docs.aws.amazon.com/systems-manager/latest/userguide/systems-manager-paramstore.html>

<https://aws.amazon.com/blogs/compute/sharing-secrets-with-aws-lambda-using-aws-systems-manager-parameter-store/>

Check out these AWS Systems Manager and Secrets Manager Cheat Sheets:

<https://tutorialsdojo.com/aws-systems-manager/>

<https://tutorialsdojo.com/aws-secrets-manager/>

41. Question

A developer is creating a new global secondary index on a provisioned mode DynamoDB table. Since the application will store large quantities of data, the write capacity units must be specified for the expected workload on both the base table and its secondary index.

Which of the following should the developer do to avoid any potential request throttling?

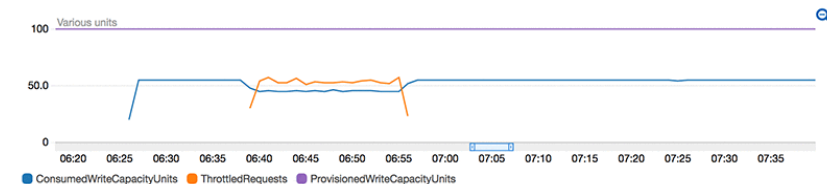
- Ensure that the global secondary index's provisioned RCU is equal or less than the RCU of the base table.
- Ensure that the global secondary index's provisioned WCU is equal or less than the WCU of the base table.
- Ensure that the global secondary index's provisioned RCU is equal or greater than the RCU of the base table.
- Ensure that the global secondary index's provisioned WCU is equal or greater than the WCU of the base table.

Correct

A **global secondary index (GSI)** is an index with a partition key and a sort key that can be different from those on the base table. It is considered "global" because queries on the index can span all of the data in the base table, across all partitions.

Every global secondary index has its own provisioned throughput settings for read and write activity. Queries or scans on a global secondary index consume capacity units from the index, not from the base table. The same holds true for global secondary index updates due to table writes. When you create a global secondary index on a provisioned mode table, you must specify read and write capacity units for the expected workload on that index. The provisioned throughput settings of a global secondary index are separate from those of its base table. A **Query** operation on a global secondary index consumes read capacity units from the index, not the base table.

When you put, update, or delete items in a table, the global secondary indexes on that table are also updated; these index updates consume write capacity units from the index, not from the base table.



For example, if you **Query** a global secondary index and exceed its provisioned read capacity, your request will be throttled. If you perform heavy write activity on the table but a global secondary index on that table has insufficient write capacity, then the write activity on the table will be throttled.

To avoid potential throttling, the provisioned write capacity for a global secondary index should be equal or greater than the write capacity of the base table since new updates will write to both the base table and global secondary index.

Hence, the correct answer in this scenario is to **ensure that the global secondary index's provisioned WCU is equal to or greater than the WCU of the base table**.

Ensuring that the global secondary index's provisioned WCU is equal or less than the WCU of the base table is incorrect because it should be the other way around, just as what is mentioned above. The provisioned write capacity for a global secondary index should be equal to or greater than the write capacity of the base table.

Ensuring that the global secondary index's provisioned RCU is equal to or greater than the RCU of the base table is incorrect because you have to set the WCU and not the RCU.

Ensuring that the global secondary index's provisioned RCU is equal or less than the RCU of the base table is incorrect because this should be WCU and in addition, the global secondary index's provisioned WCU should be set to a value that is equal or greater than the WCU of the base table to prevent request throttling.

References:

<https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/GSI.html#GSI.ThroughputConsiderations>

<https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/Programming.Errors.html#Programming.Errors.MessagesAndCodes>

Check out this Amazon DynamoDB Cheat Sheet:

<https://tutorialsdojo.com/amazon-dynamodb/>

42. Question

A leading financial company has recently deployed its application to AWS using Lambda and API Gateway. However, they noticed that all metrics are being populated in their CloudWatch dashboard except for **CacheHitCount** and **CacheMissCount**.

What could be the MOST likely cause of this issue?

- API Caching is not enabled in API Gateway.
- API Gateway Private Integrations has not been configured yet.
- They have not provided an IAM role to their API Gateway yet.
- The provided IAM role to their API Gateway only has read access but no write

privileges to CloudWatch.

Correct

You can monitor API execution using CloudWatch, which collects and processes raw data from API Gateway into readable, near-real-time metrics. These statistics are recorded for a period of two weeks so that you can access historical information and gain a better perspective on how your web application or service is performing. By default, API Gateway metric data is automatically sent to CloudWatch in one-minute periods.



The metrics reported by API Gateway provide information that you can analyze in different ways. The list below shows some common uses for the metrics. These are suggestions to get you started, not a comprehensive list.

- Monitor the **IntegrationLatency** metrics to measure the responsiveness of the backend.
- Monitor the **Latency** metrics to measure the overall responsiveness of your API calls.
- Monitor the **CacheHitCount** and **CacheMissCount** metrics to optimize cache capacities to achieve a desired performance. CacheMissCount tracks the number of requests served from the backend in a given period, **when API caching is enabled**. On the other hand, CacheHitCount track the number of requests served from the API cache in a given period.

Hence, the root cause of this issue is that the **API Caching is not enabled in API Gateway** which is why the **CacheHitCount** and **CacheMissCount** metrics are not populated.

The option that says: **they have not provided an IAM role to their API Gateway yet** is incorrect because, in the first place, the scenario already mentioned that all metrics are being populated in their CloudWatch dashboard except for two metrics. This implies that some of the metrics are populated which means that the API Gateway already has an IAM Role associated with it.

The option that says: **the provided IAM role to their API Gateway only has read access but no write privileges to CloudWatch** is incorrect because just as what is mentioned above, there is no issue with the IAM Role since all metrics are being populated except only for **CacheHitCount** and **CacheMissCount**. This means that the associated IAM Role already has **write** privileges to write logs to CloudWatch to begin with. The only reason why those two metrics are not being populated is that the API Caching is not enabled.

The option that says: **API Gateway Private Integrations has not been configured yet** is incorrect because this feature only makes it easier to expose your HTTP/HTTPS resources behind an Amazon VPC for access by clients outside of the VPC.

References:

<https://docs.aws.amazon.com/apigateway/latest/developerguide/api-gateway-metrics-and-dimensions.html>

<https://docs.aws.amazon.com/apigateway/latest/developerguide/monitoring-cloudwatch.html>

Check out this Amazon API Gateway Cheat Sheet:

<https://tutorialsdodo.com/amazon-api-gateway>

43. 43. Question

A developer needs to configure the environment name, solution stack, and environment links of his application environment which will be hosted in Elastic Beanstalk. Which configuration file should the developer add in the source bundle to meet the above requirement?

- `Dockerrun.aws.json`
- `cron.yaml`
- `env.config`
- `env.yaml`

Correct

In Elastic Beanstalk, you can include a YAML formatted environment manifest in the root of your application source bundle to configure the environment name, solution stack and environment links to use when creating your environment. An environment manifest uses the same format as Saved Configurations.

This file format includes support for environment groups. To use groups, specify the environment name in the manifest with a + symbol at the end. When you create or update the environment, specify the group name with `--group-name` (AWS CLI) or `--env-group-suffix` (EB CLI).

The following example manifest defines a web server environment for the **tutorialsdodo** frontend application, with a link to a worker environment component that it is dependent upon. The manifest uses groups to allow creating multiple environments with the same source bundle:

`~/tutorialsdodo/frontend/env.yaml`

`AWSConfigurationTemplateVersion: 1.1.0.0`

`SolutionStack: 64bit Amazon Linux 2015.09 v2.0.6 running Multi-container Docker 1.7.1 (Generic)`

`OptionSettings:`

```
aws:elasticbeanstalk:command:
  BatchSize: '30'
  BatchSizeType: Percentage
aws:elasticbeanstalk:sns:topics:
  Notification Endpoint: me@tutorialsdodo.com
aws:elb:policies:
  ConnectionDrainingEnabled: true
  ConnectionDrainingTimeout: '20'
aws:elb:loadbalancer:
  CrossZone: true
aws:elasticbeanstalk:environment:
  ServiceRole: aws-elasticbeanstalk-service-role
aws:elasticbeanstalk:application:
  Application Healthcheck URL: /
aws:elasticbeanstalk:healthreporting:system:
  SystemType: enhanced
aws:autoscaling:launchconfiguration:
  IamInstanceProfile: aws-elasticbeanstalk-ec2-role
  InstanceType: t2.micro
  EC2KeyName: workstation-uswest2
aws:autoscaling:updatepolicy:rollingupdate:
  RollingUpdateType: Health
  RollingUpdateEnabled: true
```

```
Tags:
  Cost Center: Tutorials Dojo Dev
CName: front-A08G28LG+
EnvironmentName: front+
EnvironmentLinks:
  "WORKERQUEUE" : "worker+"
```

Hence, using the `env.yaml` is the correct configuration file to be used in this scenario.

`Docker.run.aws.json` is incorrect because this configuration file is primarily used in multicontainer Docker environments that are hosted in Elastic Beanstalk. This can be used alone or combined with source code and content in a source bundle to create an environment on a Docker platform.

`env.config` is incorrect because this is just a custom configuration file which is not readily available in Elastic Beanstalk. Configuration files are YAML- or JSON-formatted documents with a `.config` file extension that you place in a folder named `.ebextensions` and deploy in your application source bundle. The more appropriate configuration file to use here is the `env.yaml` which can help you configure the environment name, solution stack, and environment links to use when creating your environment.

`cron.yaml` is incorrect because this configuration file is primarily used to define periodic tasks that add jobs to your worker environment's queue automatically at a regular interval.

References:

<https://docs.aws.amazon.com/elasticbeanstalk/latest/dg/environment-cfg-manifest.html>
<https://docs.aws.amazon.com/elasticbeanstalk/latest/dg/applications-sourcebundle.html>

Check out this AWS Elastic Beanstalk Cheat Sheet:
<https://tutorialsdojo.com/aws-elastic-beanstalk/>

44. Question

A multinational investment bank has a hybrid cloud architecture with AWS. To improve the security of the applications, the company decided to use AWS Key Management Service (KMS) with customer-managed keys to create and manage the encryption keys across a wide range of AWS services. A software developer has been assigned to integrate AWS KMS with the financial applications of the company.

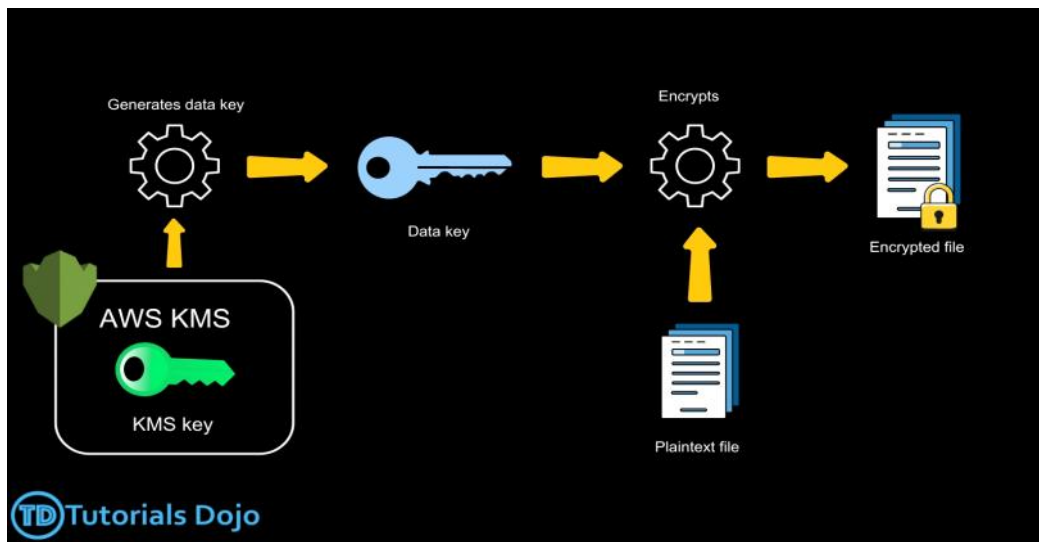
Which of the following are the recommended steps to locally encrypt data using AWS KMS that should be followed? (Select TWO.)

- Erase the plaintext data key from memory and store the encrypted data key alongside the locally encrypted data.
- Use the `GenerateDataKeyWithoutPlaintext` operation to get a data encryption key then use the plaintext data key in the response to encrypt data locally.
- Encrypt data locally using the `Encrypt` operation.
- Use the `GenerateDataKey` operation to get a data encryption key then use the plaintext data key in the response to encrypt data locally.
- Erase the encrypted data key from memory and store the plaintext data key alongside the locally encrypted data.

Incorrect

When you encrypt your data, your data is protected, but you have to protect your encryption key. One strategy is to encrypt it. *Envelope encryption* is the practice of encrypting plaintext data with a data key, and then encrypting the data key under another key.

You can even encrypt the data encryption key under another encryption key, and encrypt that encryption key under another encryption key. But, eventually, one key must remain in plaintext so you can decrypt the keys and your data. This top-level plaintext encryption key is known as the *root key*.



AWS KMS helps you to protect your encryption keys by storing and managing them securely. Root keys stored in AWS KMS, known as AWS KMS keys, never leave the AWS KMS FIPS validated hardware security modules unencrypted. To use an AWS KMS key, you must call AWS KMS.

It is recommended that you use the following pattern to encrypt data locally in your application:

1. Use the `GenerateDataKey` operation to get a data encryption key.
2. Use the plaintext data key (returned in the `Plaintext` field of the response) to encrypt data locally, then erase the plaintext data key from memory.
3. Store the encrypted data key (returned in the `CiphertextBlob` field of the response) alongside the locally encrypted data.

Hence, the correct answers are:

- Use the `GenerateDataKey` operation to get a data encryption key then use the plaintext data key in the response to encrypt data locally.
- Erase the plaintext data key from memory and store the encrypted data key alongside the locally encrypted data.

The option that says: Use the `GenerateDataKeyWithoutPlaintext` operation to get a data encryption key then using the plaintext data key in the response to encrypt data locally is incorrect because you have to typically use the `GenerateDataKey` operation instead. This is because the `GenerateDataKeyWithoutPlaintext` operation will not return the plaintext data key just as its name implies.

The option that says: Erase the encrypted data key from memory and storing the plaintext data key alongside the locally encrypted data is incorrect because it should be the other way around. You have to erase the plaintext data key from memory and store the encrypted data key alongside the locally encrypted data.

The option that says: **Encrypt data locally using the Encrypt operation** is incorrect because the `Encrypt` operation is primarily used to encrypt RSA keys, database passwords, or other sensitive information. This operation can also be used to move encrypted data from one AWS region to another; however, this is not recommended if you want to encrypt your data locally. You have to use the `GenerateDataKey` operation instead.

References:

https://docs.aws.amazon.com/kms/latest/APIReference/API_GenerateDataKey.html
<https://docs.aws.amazon.com/kms/latest/developerguide/concepts.html#enveloping>

Check out these Amazon S3 and AWS KMS Cheat Sheets:

<https://tutorialsdojo.com/amazon-s3/>
<https://tutorialsdojo.com/aws-key-management-service-aws-kms/>

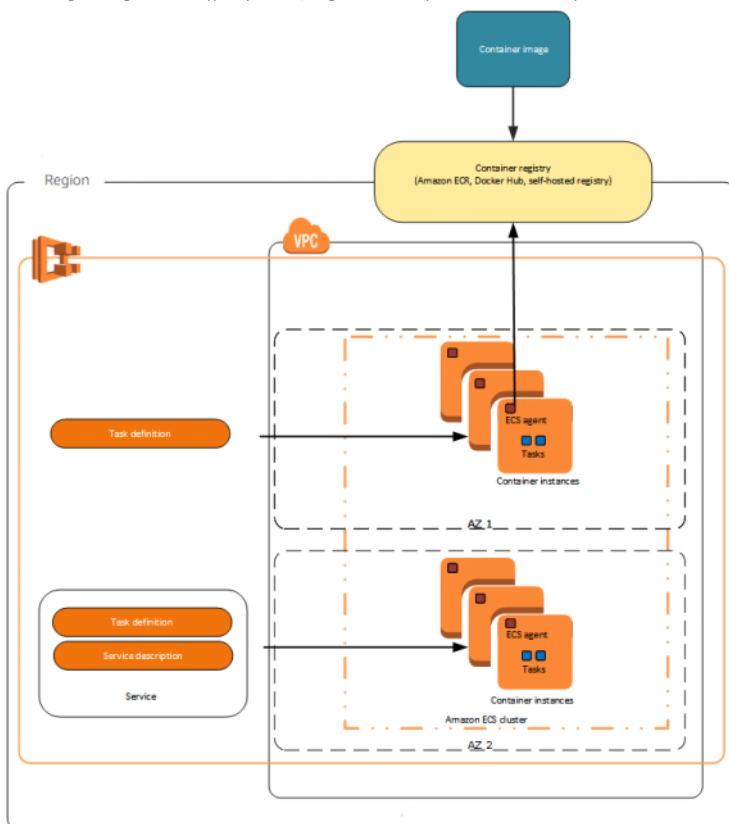
45. 45. Question

A developer is designing an application which will be hosted in ECS and uses an EC2 launch type. You need to group your container instances by certain attributes such as Availability Zone, instance type, or custom metadata. After you have defined a group of container instances, you will need to customize Amazon ECS to place tasks on container instances based on the group you specified. Which of the following ECS features provides you with expressions that you can use to group container instances by a specific attribute?

- Cluster Query Language
- Task Placement Strategies
- Task Groups
- Task Placement Constraints

Incorrect

When a task that uses the EC2 launch type is launched, Amazon ECS must determine where to place the task based on the requirements specified in the task definition, such as CPU and memory. Similarly, when you scale down the task count, Amazon ECS must determine which tasks to terminate. You can apply task placement strategies and constraints to customize how Amazon ECS places and terminates tasks. Task placement strategies and constraints are not supported for tasks using the Fargate launch type. By default, Fargate tasks are spread across Availability Zones.



Cluster queries are expressions that enable you to group objects. For example, you can group container instances by attributes such as Availability Zone, instance type, or custom metadata. You can add custom metadata to your container instances, known as *attributes*. Each attribute has a name and an optional string value. You can use the built-in attributes provided by Amazon ECS or define custom attributes.

After you have defined a group of container instances, you can customize Amazon ECS to place tasks on container instances based on group. Running tasks manually is ideal in certain situations. For example, suppose that you are developing a task but you are not ready to deploy this task with the service scheduler. Perhaps your task is a one-time or periodic batch job that does not make sense to keep running or restart when it finishes.

Hence, the correct ECS feature which provides you with expressions that you can use to group container instances by a specific attribute is **Cluster Query Language**.

Task Group is incorrect because this is just a set of related tasks. This does not provide expressions that enable you to group objects. All tasks with the same task group name are considered as a set when performing spread placement.

Task Placement Constraint is incorrect because it is just a rule that is considered during task placement. Although it uses cluster queries when you are placing tasks on container instances based on a specific expression, it does not provide the actual expressions which are used to group those container instances.

Task Placement Strategies is incorrect because this is just an algorithm for selecting instances for task placement or tasks for termination.

References:

<https://aws.amazon.com/blogs/compute/amazon-ecs-task-placement/>
<https://docs.aws.amazon.com/AmazonECS/latest/developerguide/cluster-query-language.html>

Check out this Amazon ECS Cheat Sheet:

<https://tutorialsdojo.com/amazon-elastic-container-service-amazon-ecs/>

46. 46. Question

A software engineer is developing a serverless application which will use a DynamoDB database. One of the requirements is that each write request should return the total number of write capacity units consumed, with subtotals for the table and any secondary indexes that were affected by the operation.

What should be done to accomplish this feature?

- Add the `ReturnConsumedCapacity` parameter with a value of `INDEXES` in every write request.
- Add the `ReturnValues` parameter with a value of `INDEXES` in every write request.
- Add the `ReturnValues` parameter with a value of `TOTAL` in every write request.
- Add the `ReturnConsumedCapacity` parameter with a value of `TOTAL` in every write request.

Incorrect

To create, update, or delete an item in a DynamoDB table, use one of the following operations:

- `PutItem`
- `UpdateItem`
- `DeleteItem`

For each of these operations, you need to specify the entire primary key, not just part of it. For example, if a table has a composite primary key (partition key and sort key), you must supply a value for the partition key and a value for the sort key.

To return the number of write capacity units consumed by any of these operations, set the `ReturnConsumedCapacity` parameter to one of the following:

TOTAL — returns the total number of write capacity units consumed.

INDEXES — returns the total number of write capacity units consumed, with subtotals for the table and any secondary indexes that were affected by the operation.

NONE — no write capacity details are returned. (This is the default.)

Hence, the correct answer is to **add the `ReturnConsumedCapacity` parameter with a value of `INDEXES` in every write request.**

Adding the `ReturnValues` parameter with a value of `INDEXES` in every write request is incorrect because you should use a `ReturnConsumedCapacity` parameter instead.

Adding the `ReturnConsumedCapacity` parameter with a value of `TOTAL` in every write request is incorrect because this will not return the consumed WCU subtotals for the table and any secondary indexes that were affected by the operation just as what is required by the application. You have to use `INDEXES` instead.

Adding the `ReturnValues` parameter with a value of `TOTAL` in every write request is incorrect because you should use a `ReturnConsumedCapacity` parameter instead. In addition, the value of the parameter is also incorrect as it doesn't return the consumed WCU subtotals for the table and any secondary indexes that were affected by the operation.

References:

<https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/WorkingWithItems.html#WorkingWithItems.WritingData>
https://docs.aws.amazon.com/amazondynamodb/latest/APIReference/API_PutItem.html#API_PutItem_RequestParameters

Check out this Amazon DynamoDB Cheat Sheet:

<https://tutorialsdojo.com/amazon-dynamodb/>

47. 47. Question

A developer needs to encrypt all objects being uploaded by their application to the S3 bucket to comply with the company's security policy. The bucket will use server-side encryption with Amazon S3-Managed encryption keys (SSE-S3) to encrypt the data using 256-bit Advanced Encryption Standard (AES-256) block cipher.

Which of the following request headers should the developer use?

- `x-amz-server-side-encryption-customer-algorithm`
- `x-amz-server-side-encryption`
- `x-amz-server-side-encryption-customer-key`
- `x-amz-server-side-encryption-customer-key-MD5`

Correct

Server-side encryption protects data at rest. If you use Server-Side Encryption with Amazon S3-Managed Encryption Keys (SSE-S3), Amazon S3 will encrypt each object with a unique key and as an additional safeguard, it encrypts the key itself with a master key that it rotates regularly. Amazon S3 server-side encryption uses one of the strongest block ciphers available, 256-bit Advanced Encryption Standard (AES-256), to encrypt your data.

Edit default encryption Info

Default encryption

Server-side encryption is automatically applied to new objects stored in this bucket.

Encryption type Info

- ☒ Server-side encryption with Amazon S3 managed keys (SSE-S3)
- ☐ Server-side encryption with AWS Key Management Service keys (SSE-KMS)
- ☐ Dual-layer server-side encryption with AWS Key Management Service keys (DSSE-KMS)
Secure your objects with two separate layers of encryption. For details on pricing, see [DSSE-KMS pricing on the Storage tab of the Amazon S3 pricing page](#).

Bucket Key

Using an S3 Bucket Key for SSE-KMS reduces encryption costs by lowering calls to AWS KMS. S3 Bucket Keys aren't supported for DSSE-KMS. [Learn more](#)

- ☐ Disable
- ☒ Enable



Cancel

Save changes

If you need server-side encryption for all of the objects that are stored in a bucket, use a bucket policy. For example, the following bucket policy denies permissions to upload an object unless the request includes the `x-amz-server-side-encryption` header to request server-side encryption:


```
{
  "Version": "2012-10-17",
  "Id": "PutObjectPolicy",
  "Statement": [
    {
      "Sid": "DenyIncorrectEncryptionHeader",
      "Effect": "Deny",
      "Principal": "*",
      "Action": "s3:PutObject",
      "Resource": "arn:aws:s3:::awsexamplebucket1/*",
      "Condition": {
        "StringNotEquals": {
          "s3:x-amz-server-side-encryption": "AES256"
        }
      }
    }
  ]
}
```

However, if you chose to use server-side encryption with customer-provided encryption keys (SSE-C), you must provide encryption key information using the following request headers:

x-amz-server-side-encryption-customer-algorithm
x-amz-server-side-encryption-customer-key
x-amz-server-side-encryption-customer-key-MD5

Hence, using the **x-amz-server-side-encryption** header is correct as this is the one being used for Amazon S3-Managed Encryption Keys (SSE-S3). All other options are incorrect since they are used for SSE-C.

References:

<https://docs.aws.amazon.com/AmazonS3/latest/dev/serv-side-encryption.html>
<https://docs.aws.amazon.com/AmazonS3/latest/dev/UsingServerSideEncryption.html>
<https://docs.aws.amazon.com/AmazonS3/latest/dev/ServerSideEncryptionCustomerKeys.html>

Check out this Amazon S3 Cheat Sheet:

<https://tutorialsdojo.com/amazon-s3/>

48. Question

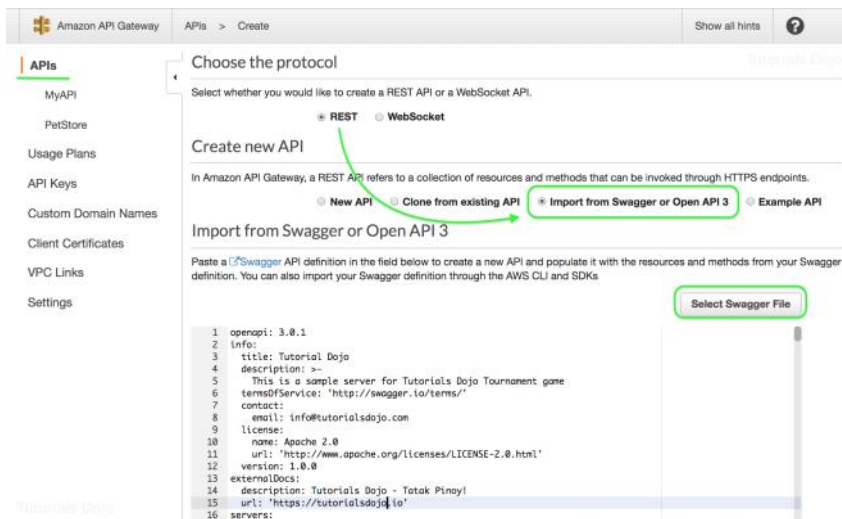
A company is using OpenAPI, which is also known as Swagger, for the API specifications of their REST web services that are hosted on their on-premises data center. They want to migrate their system to AWS using Lambda and API Gateway. In line with this, you are instructed to create a new API and populate it with the resources and methods from their Swagger definition.

Which of the following is the EASIEST way to accomplish this task?

- Create models and templates for request and response mappings based on the company's API definitions.
- Use AWS SAM to migrate and deploy the company's web services to API Gateway.
- Import their Swagger or OpenAPI definitions to API Gateway using the AWS Console.
- Use CodeDeploy to migrate and deploy the company's web services to API Gateway.

Correct

You can use the API Gateway Import API feature to import a REST API from an external definition file into API Gateway. Currently, the Import API feature supports OpenAPI v2.0 and OpenAPI v3.0 definition files. You can update an API by overwriting it with a new definition or merge a definition with an existing API. You specify the options using a mode query parameter in the request URL. You can paste a [Swagger](#) API definition in the AWS Console to create a new API and populate it with the resources and methods from your Swagger or OpenAPI definition, just as shown below:



You can also import your Swagger definition through the AWS CLI and SDKs.

Hence, the correct answer in this scenario is to **import their Swagger or OpenAPI definitions to API Gateway using the AWS Console**.

Using **CodeDeploy to migrate and deploy the company's web services to API Gateway** is incorrect because using CodeDeploy alone is not enough to deploy new custom APIs. This is mainly used in conjunction with AWS SAM where you can add deployment preferences to manage the way traffic is shifted during an AWS Lambda application deployment.

Using **AWS SAM to migrate and deploy the company's web services to API Gateway** is incorrect. Although using AWS SAM is the preferred way to deploy your serverless application, it is not the easiest way to import the Swagger API definitions file. As mentioned above, you can simply import Swagger or OpenAPI files directly to AWS.

Creating models and templates for request and response mappings based on the company's API definitions is incorrect because this is primarily done for API Gateway integration to other services and not for importing API definitions file.

References:

<https://docs.aws.amazon.com/apigateway/latest/developerguide/api-gateway-import-api.html>
<https://docs.aws.amazon.com/apigateway/latest/developerguide/api-gateway-create-api-from-example.html>

Check out this Amazon API Gateway Cheat Sheet:

<https://tutorialsdojo.com/amazon-api-gateway/>

49. Question

A website hosted in AWS has a custom CloudWatch metric to track all HTTP server errors in the site every minute, which occurs intermittently. An existing CloudWatch Alarm has already been configured for this metric but you would like to re-configure this to properly monitor the application. The alarm should only be triggered when all three data points in the most recent three consecutive periods are above the threshold.

Which of the following options is the MOST appropriate way to monitor the website based on the given threshold?

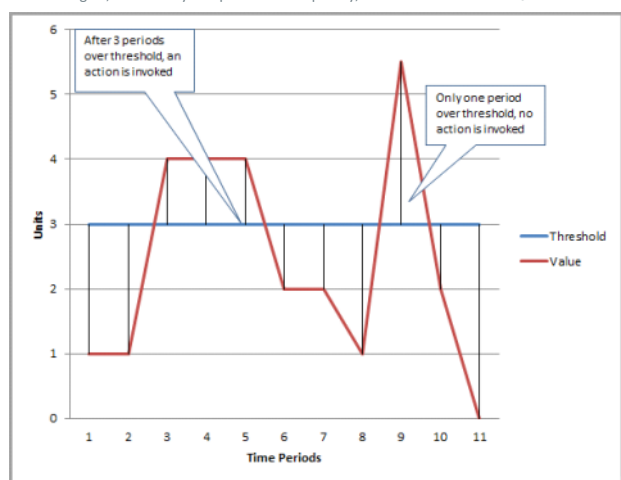
- Use metric math in CloudWatch to properly compute the threshold.
- Use high-resolution metrics.
- Set both the Period and Datapoints to Alarm to 3.
- Set both the Evaluation Period and Datapoints to Alarm to 3.

Correct

When you create an alarm, you specify three settings to enable CloudWatch to evaluate when to change the alarm state:

- **Period** is the length of time to evaluate the metric or expression to create each individual data point for an alarm. It is expressed in seconds. If you choose one minute as the period, there is one datapoint every minute.
- **Evaluation Period** is the number of the most recent periods, or data points, to evaluate when determining alarm state.
- **Datapoints to Alarm** is the number of data points within the evaluation period that must be breaching to cause the alarm to go to the `ALARM` state. The breaching data points do not have to be consecutive, they just must all be within the last number of data points equal to **Evaluation Period**.

In the following figure, the alarm threshold is set to three units. The alarm is configured to go to the `ALARM` state and both **Evaluation Period** and **Datapoints to Alarm** are 3. That is, when all three datapoints in the most recent three consecutive periods are above the threshold, the alarm goes to the `ALARM` state. In the figure, this happens in the third through fifth time periods. At period six, the value dips below the threshold, so one of the periods being evaluated is not breaching, and the alarm state changes to `OK`. During the ninth time period, the threshold is breached again, but for only one period. Consequently, the alarm state remains `OK`.



Hence, the option that says: **Set both the Evaluation Period and Datapoints to Alarm to 3** is the correct answer.

The option that says: **Use high-resolution metrics** is incorrect because the scenario says that it only needs to monitor the HTTP server errors every minute, and not its sub-minute activity. If you set an alarm on a high-resolution metric, you can specify a high-resolution alarm with a period of 10 seconds or 30 seconds. Hence, this option is irrelevant in this scenario.

The option that says: **Set both the Period and Datapoints to Alarm to 3** is incorrect because you should set the **Evaluation Period** and not the **Period** setting.

The option that says: **Use metric math in CloudWatch to properly compute the threshold** is incorrect because the Metric Math feature is only applicable for scenarios where you need to query multiple CloudWatch metrics or if you want to use math expressions to create new time series based on selected metrics.

References:

- <https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/AlarmThatSendsEmail.html>
- <https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/ConsoleAlarms.html>

Check out this Amazon CloudWatch Cheat Sheet:

<https://tutorialsdojo.com/amazon-cloudwatch/>

50. Question

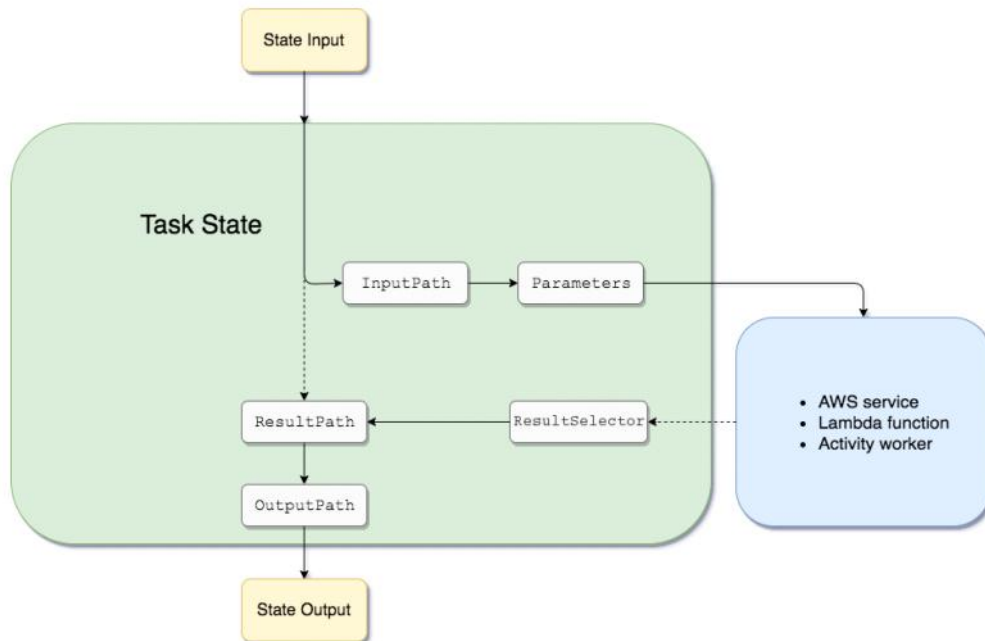
A transcoding media service is being developed in AWS. Photos uploaded to Amazon S3 will trigger Step Functions to coordinate a series of processes that will perform image analysis tasks. The final output should contain the input plus the result of the final state to conform to the application's logic flow.

What should the developer do?

- Declare an `InputPath` field filter on the Amazon States Language specification.
- Declare an `OutputPath` field filter on the Amazon States Language specification.
- Declare a `Parameters` field filter on the Amazon States Language specification.
- Declare a `ResultPath` field filter on the Amazon States Language specification.

Incorrect

A Step Functions execution receives a JSON text as input and passes that input to the first state in the workflow. Individual states receive JSON as input and usually pass JSON as output to the next state. Understanding how this information flows from state to state and learning how to filter and manipulate this data is key to effectively designing and implementing workflows in AWS Step Functions.



In the Amazon States Language, these fields filter and control the flow of JSON from state to state:

- InputPath
- OutputPath
- ResultPath
- Parameters

Both the **InputPath** and **Parameters** fields provide a way to manipulate JSON as it moves through your workflow. InputPath can limit the input that is passed by filtering the JSON notation by using a path. The Parameters field enables you to pass a collection of key-value pairs, where the values are either static values that you define in your state machine definition, or that are selected from the input using a path.

AWS Step Functions applies the InputPath field first, and then the Parameters field. You can first filter your raw input to a selection you want using InputPath, and then apply Parameters to manipulate that input further, or add new values.

The output of a state can be a copy of its input, the result it produces (for example, the output from a Task state's Lambda function), or a combination of its input and result. Use **ResultPath** to control which combination of these is passed to the state output.

OutputPath enables you to select a portion of the state output to pass to the next state. This enables you to filter out unwanted information, and pass only the portion of JSON that you care about.

Out of these field filters, the ResultPath field filter is the only one that can control input values and its previous results to be passed to the state output. Hence, the correct answer is: **Declare a ResultPath field filter on the Amazon States Language specification.**

The option that says: **Declare an InputPath field filter on the Amazon State Language specification** is incorrect because it just operates on the input level by filtering the JSON notation by using a path. It cannot control both ends of a state (input and output).

The option that says: **Declare an OutputPath field filter on the Amazon State Language specification** is incorrect because it just operates on the output level. It is used to filter out unwanted information and pass only the portion of JSON that you care about that will be passed onto the next state.

The option that says: **Declare a Parameters field filter on the Amazon State Language specification** is incorrect because this is used in conjunction with the InputPath field filter, which means it can only be used on the input level of a state.

References:

- <https://docs.aws.amazon.com/step-functions/latest/dg/concepts-input-output-filtering.html>
- <https://docs.aws.amazon.com/step-functions/latest/dg/how-step-functions-works.html>

Check out this AWS Step Functions Cheat Sheet:

- <https://tutorialsdojo.com/aws-step-functions/>

51. Question

A financial mobile application has a serverless backend API which consists of DynamoDB, Lambda, and Cognito. Due to the confidential financial transactions handled by the mobile application, there is a new requirement provided by the company to add a second authentication method that doesn't rely solely on user name and password.

Which of the following is the MOST suitable solution that the developer should implement?

- Use a new IAM policy to a user pool in Cognito.
- Use Cognito with SNS to allow additional authentication via SMS.
- Integrate multi-factor authentication (MFA) to a user pool in Cognito to protect the identity of your users.
- Create a custom application that integrates with Amazon Cognito which implements the second layer of authentication.

Correct

You can add multi-factor authentication (MFA) to a user pool to protect the identity of your users. MFA adds a second authentication method that doesn't rely solely on usernames and passwords. You can choose to use SMS text messages, or time-based one-time (TOTP) passwords as second factors in signing in your users. You can also use adaptive authentication with its risk-based model to predict when you might need another authentication factor. It's part of the user pool's advanced security features, which also include protections against compromised credentials.

Edit multi-factor authentication (MFA) info

Amazon Cognito provides your app users with additional authentication factors using SMS messages and time-based one-time passwords (TOTP).

Multi-factor authentication

Configure secure access to your app by enforcing multi-factor authentication (MFA) during the user sign-in process. MFA settings are applied to all app clients.

MFA enforcement info

☐ Require MFA - Recommended

Users must provide an additional authentication factor when signing in.

☒ Optional MFA

Users can sign in with a single authentication factor, and can choose to add additional authentication factors.

☐ No MFA

Users can only sign in with a single authentication factor. This is the least secure option.

MFA methods info

Choose the MFA methods that are allowed in your user pool. TOTP-based MFA offers a higher level of security. Recipient message and data rates apply.

☒ Authenticator apps

Users can authenticate with a TOTP from an authenticator app such as Authy or Google Authenticator.

☒ SMS message

Users can authenticate with a code sent by SMS message to a verified phone number. SMS messages are charged separately by Amazon SNS. [Learn more about pricing](#) This option must be selected because SMS is configured.

Cancel

Save changes

Multi-factor authentication (MFA) increases security for your app by adding another authentication method, and not relying solely on user name and password. You can choose to use SMS text messages, or time-based one-time (TOTP) passwords as second factors in signing in your users.

With adaptive authentication, you can configure your user pool to require second-factor authentication in response to an increased risk level.

Hence, the correct answer in this scenario is to **integrate multi-factor authentication (MFA) to a user pool in Cognito to protect the identity of your users.**

Creating a custom application that integrates with Amazon Cognito which implements the second layer of authentication is incorrect. Although this option is viable, it is not the most suitable solution in this scenario since you can simply use MFA as a second-factor authentication for the mobile app.

Using a new IAM policy to a user pool in Cognito is incorrect because an IAM Policy alone cannot implement a second-factor authentication. You have to configure Cognito to use MFA instead.

Using Cognito with SNS to allow additional authentication via SMS is incorrect. Although this is part of the MFA setup, using this solution alone is not enough if you didn't enable MFA in the first place.

References:

<https://docs.aws.amazon.com/cognito/latest/developerguide/managing-security.html>

<https://docs.aws.amazon.com/cognito/latest/developerguide/cognito-user-identity-pools.html>

Tutorials Dojo's AWS Certified Developer Associate Exam Study Guide:

<https://tutorialsdojo.com/aws-certified-developer-associate/>

52. Question

A developer is planning to add a global secondary index in a DynamoDB table. This will allow the application to query a specific index that can span all of the data in the base table, across all partitions.

Which of the following should the developer consider when using this type of index? (Select TWO.)

- Queries or scans on this index consume capacity units from the index, not from the base table.
- When you query this index, you can choose either eventual consistency or strong consistency.
- For each partition key value, the total size of all indexed items must be 10 GB or less.
- Queries or scans on this index consume read capacity units from the base table.
- Queries on this index support eventual consistency only.

Incorrect

A global secondary index is an index with a partition key and a sort key that can be different from those on the base table. A global secondary index is considered "global" because queries on the index can span all of the data in the base table, across all partitions.

To create a table with one or more global secondary indexes, use the CreateTable operation with the GlobalSecondaryIndexes parameter. For maximum query flexibility, you can create up to 20 global secondary indexes (default limit) per table. You must specify one attribute to act as the index partition key; you can optionally specify another attribute for the index sort key. It is not necessary for either of these key attributes to be the same as a key attribute in the table. Global secondary indexes inherit the read/write capacity mode from the base table.

Characteristic	Global Secondary Index	Local Secondary Index
Key Schema	The primary key of a global secondary index can be either simple (partition key) or composite (partition key and sort key).	The primary key of a local secondary index must be composite (partition key and sort key).
Key Attributes	The index partition key and sort key (if present) can be any base table attributes of type string, number, or binary.	The partition key of the index is the same attribute as the partition key of the base table. The sort key can be any base table attribute of type string, number, or binary.
Size Restrictions Per Partition Key Value	There are no size restrictions for global secondary indexes.	For each partition key value, the total size of all indexed items must be 10 GB or less.
Online Index Operations	Global secondary indexes can be created at the same time that you create a table. You can also add a new global secondary index to an existing table, or delete an existing global secondary index. For more information, see Managing Global Secondary Indexes .	Local secondary indexes are created at the same time that you create a table. You cannot add a local secondary index to an existing table, nor can you delete any local secondary indexes that currently exist.
Queries and Partitions	A global secondary index lets you query over the entire table, across all partitions.	A local secondary index lets you query over a single partition, as specified by the partition key value in the query.
Read Consistency	Queries on global secondary indexes support eventual consistency only.	When you query a local secondary index, you can choose either eventual consistency or strong consistency.
Provisioned Throughput Consumption	Every global secondary index has its own provisioned throughput settings for read and write activity. Queries or scans on a global secondary index consume capacity units from the index, not from the base table. The same holds true for global secondary index updates due to table writes.	Queries or scans on a local secondary index consume read capacity units from the base table. When you write to a table, its local secondary indexes are also updated; these updates consume write capacity units from the base table.
Projected Attributes	With global secondary index queries or scans, you can only request the attributes that are projected into the index. DynamoDB will not fetch any attributes from the table.	If you query or scan a local secondary index, you can request attributes that are not projected in to the index. DynamoDB will automatically fetch those attributes from the table.

As shown in the above table, the following are the things that the developer should consider when using a global secondary index:

- **Queries or scans on this index consume capacity units from the index, not from the base table.**
- **Queries on this index support eventual consistency only.**

The following options are incorrect because these are about local secondary indexes:

- When you query this index, you can choose either eventual consistency or strong consistency.
- Queries or scans on this index consume read capacity units from the base table
- For each partition key value, the total size of all indexed items must be 10 GB or less.

References:

<https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/SecondaryIndexes.html>
<https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/GSI.html>

Check out this Amazon DynamoDB Cheat Sheet:
<https://tutorialsdojo.com/amazon-dynamodb/>

DynamoDB Scan vs Query:
<https://tutorialsdojo.com/dynamodb-scan-vs-query/>

Comparison of AWS Services Cheat Sheets:
<https://tutorialsdojo.com/comparison-of-aws-services/>

53. Question

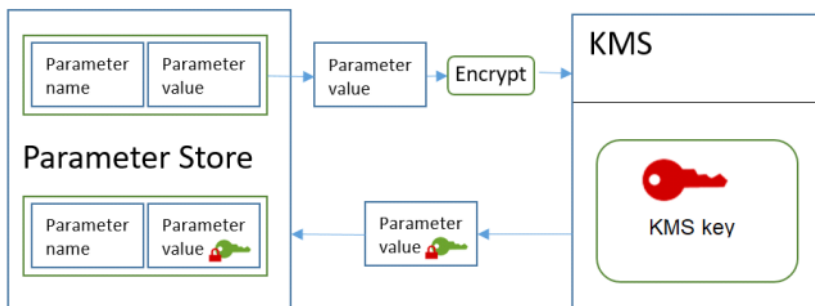
Your team is developing a serverless application, which is composed of multiple Lambda functions which process data from an SQS queue and stores the results to an RDS database. To comply with the strict IT policy of the company, you were instructed to configure these functions to share the same connection string that should be properly secured and encrypted.

What should you do to protect, encrypt, and share your database credentials in AWS?

- Store the database credentials as environment variables with KMS encryption which will be shared by the Lambda functions.
- Use AWS Systems Manager Parameter Store as a Secure String Parameter.
- Encrypt the database credentials and store them in an S3 bucket which the Lambda functions can fetch.
- Use IAM DB Authentication in RDS to allow encrypted connections from each Lambda function.

Incorrect

AWS Systems Manager Parameter Store provides secure, hierarchical storage for configuration data management and secrets management. You can store data such as passwords, database strings, and license codes as parameter values. You can store values as plain text or encrypted data. You can then reference values by using the unique name that you specified when you created the parameter.



Parameter Store offers the following benefits and features:

- Use a secure, scalable, hosted secrets management service (No servers to manage).
- Improve your security posture by separating your data from your code.
- Store configuration data and secure strings in hierarchies and track versions.
- Control and audit access at granular levels.
- Configure change notifications and trigger automated actions.
- Tag parameters individually, and then secure access from different levels, including operational, parameter, Amazon EC2 tag, or path levels.
- Reference AWS Secrets Manager secrets by using Parameter Store parameters.

Hence, the correct solution for this scenario is to **use AWS Systems Manager Parameter Store as a Secure String Parameter.**

Encrypting the database credentials and storing them in an S3 bucket which the Lambda functions can fetch is incorrect because it is a security risk to store sensitive database passwords and credentials in S3, even though the data is encrypted. A more suitable and secure way is to use the AWS Secrets Manager or the Systems Manager Parameter Store.

Storing the database credentials as environment variables with KMS encryption which will be shared by the Lambda functions is incorrect because even though the credentials will be encrypted, these environment variables will only be used by an individual Lambda function and cannot be shared.

Using IAM DB Authentication in RDS to allow encrypted connections from each Lambda function is incorrect. While using IAM DB Authentication can significantly enhance security for database connections in AWS environments, it's more about controlling access rather than managing the sharing of encrypted credentials like a database connection string across multiple applications or services. Additionally, this method is limited to certain types of databases supported by AWS, such as Amazon RDS for MySQL and PostgreSQL, and might not be available for all database engines. Lastly, in the scenario where Lambda functions need to share and securely access a connection string, other AWS services like the Systems Manager Parameter Store would be more appropriate to meet all the specified needs.

References:

<https://docs.aws.amazon.com/systems-manager/latest/userguide/systems-manager-paramstore.html>
<https://aws.amazon.com/blogs/compute/sharing-secrets-with-aws-lambda-using-aws-systems-manager-parameter-store/>

Check out this AWS Systems Manager Cheat Sheet:
<https://tutorialsdojo.com/aws-systems-manager/>

54. Question

A technology firm manages an internal portal that contains proprietary information. The firm intends to make this portal available to the general public. However, access must be restricted solely to employees authenticated through the firm's OpenID Connect (OIDC) identity provider (IdP). This authentication mechanism must be implemented without modifying the existing website code.

Which action will accomplish this requirement?

- Set up an internet-facing Application Load Balancer. Create a listener rule for the load balancer for HTTPS on port 80. Add a default authenticating operation that returns the OIDC IdP configuration.
- Set up an internet-facing Network Load Balancer. Create a listener rule for the load balancer for HTTPS on port 443. Configure the rule's default action to authenticate users using the OIDC IdP configuration.

- Set up an internet-facing Application Load Balancer. Create a listener rule for the load balancer for HTTPS on port 443. Configure the rule's default action to invoke an AWS Lambda function for OIDC authentication.
- Set up an internet-facing Application Load Balancer. Create a listener rule for the load balancer for HTTPS on port 443. Configure the rule's default action to authenticate users using the OIDC IdP configuration.

Incorrect

Application Load Balancer (ALB) is created to handle both HTTP and HTTPS traffic at the application layer (Layer 7). It provides advanced routing capabilities and supports features such as user authentication and SSL termination, which are essential for securing web applications exposed to the internet. By configuring a public ALB, the company ensures that all incoming traffic to the internal website is managed through a central point that can enforce security policies, including OIDC authentication. The ALB can be easily integrated with the company's OIDC identity provider to authenticate users, making it an ideal solution for this requirement.

Listener configuration

The listener will be identified by the protocol and port.

Protocol

Used for connections from clients to the load balancer.

HTTPS

Port

The port on which the load balancer is listening for connections.

443

1-65535

Default actions Info

The default action is used if no other rules apply. Choose the default action for traffic on this listener.

Authentication Info

Authentication requires IPv4 connectivity to authentication endpoints. [Learn more](#)

- ☒ Use OpenID or Amazon Cognito
- Include authentication using either OpenID Connect (OIDC) or Amazon Cognito.

Identity provider

An identity provider (IdP) is a service that stores and manages digital identities.

OIDC (OpenID Connect)

Issuer

OpenID provider.

Authorization endpoint

OpenID provider server endpoint.

Token endpoint

URL of your token endpoint.

User info endpoint

URL of your user info endpoint.

Setting up a listener on HTTPS port 443 ensures that all data transferred between users and the website is encrypted, protecting sensitive information from being intercepted. The listener can be configured with rules to authenticate users using the company's OIDC IdP, which verifies the identity of the users before granting access. This setup leverages the ALB's native support for OIDC authentication, streamlining the authentication process without requiring changes to the website itself. This approach aligns with best practices for securing web applications, as outlined in the latest AWS documentation, ensuring robust protection for the company's internal resources. Hence, the correct answer is: **Set up an internet-facing Application Load Balancer. Create a listener rule for the load balancer for HTTPS on port 443. Configure the rule's default action to authenticate users using the OIDC IdP configuration.**

The option that says: **Set up an internet-facing Network Load Balancer. Create a listener rule for the load balancer for HTTPS on port 443. Configure the rule's default action to authenticate users using the OIDC IdP configuration** is incorrect because Network Load Balancers only operate at the transport layer (Layer 4) and does not support OIDC authentication, which is required for this scenario.

The option that says: **Set up an internet-facing Application Load Balancer. Create a listener rule for the load balancer for HTTPS on port 80. Add a default authenticating operation that returns the OIDC IdP configuration** is incorrect This is incorrect because port 80 is typically used for HTTP, not HTTPS. OIDC authentication only works for HTTPS listeners.

The option that says: **Set up an internet-facing Application Load Balancer. Create a listener rule for the load balancer for HTTPS on port 443. Configure the rule's default action to invoke an AWS Lambda function for OIDC authentication** is incorrect. While this approach is technically feasible, it adds unnecessary complexity and requires custom coding, which might not align with the requirement to avoid modifying the existing code infrastructure.

References:

<https://docs.aws.amazon.com/elasticloadbalancing/latest/application/introduction.html>
<https://docs.aws.amazon.com/elasticloadbalancing/latest/application/listener-authenticate-users.html>

Check out this AWS Elastic Load Balancing Cheat Sheet:
<https://tutorialsdjojo.com/aws-elastic-load-balancing-elb/>

55. Question

For application deployments, a company is using CloudFormation templates, which are regularly updated to map the latest AMI IDs. A developer was assigned to automate the process since the current set up takes a lot of time to execute on a regular basis. Which of the following is the MOST suitable solution that the developer should implement to satisfy this requirement?

- Integrate AWS Service Catalog with AWS Config to automatically fetch the latest AMI and use it for succeeding deployments.
- Set up your Systems Manager State Manager to store the latest AMI IDs and integrate it with your CloudFormation template. Call the update-stack API in CloudFormation whenever you decide to update the EC2 instances in your CloudFormation template.
- Integrate CloudFormation with AWS Service Catalog to fetch the latest AMI IDs and

automatically use them for succeeding deployments.

- Set up CloudFormation with Systems Manager Parameter Store to retrieve the latest AMI IDs for your template. Whenever you decide to update the EC2 instances, call the update-stack API in CloudFormation in your CloudFormation template.

Incorrect

You can use the existing Parameters section of your CloudFormation template to define Systems Manager parameters, along with other parameters. Systems Manager parameters are a unique type that is different from existing parameters because they refer to actual values in the Parameter Store. The value for this type of parameter would be the Systems Manager (SSM) parameter key instead of a string or other value. CloudFormation will fetch values stored against these keys in Systems Manager in your account and use them for the current stack operation. If the parameter referenced in the template does not exist in Systems Manager, there will be synchronous validation error that will be thrown. Also, if you have defined any parameter value validations (AllowedValues, AllowedPattern, etc.) for Systems Manager parameters, they will be performed against SSM keys which are given as input values for template parameters, not actual values stored in Systems Manager.



Parameters stored in Systems Manager are mutable. Any time you use a template containing Systems Manager parameters to create/update your stacks, CloudFormation uses the values for these Systems Manager parameters at the time of the create/update operation. So, as parameters are updated in Systems Manager, you can have the new value of the parameter take effect by just executing a stack update operation. The Parameters section in the output for Describe API will show an additional 'ResolvedValue' field that contains the resolved value of the Systems Manager parameter that was used for the last stack operation.

Hence, the correct answer is to **set up CloudFormation with Systems Manager Parameter Store to retrieve the latest AMI IDs for your template. Whenever you decide to update the EC2 instances, call the update-stack API in CloudFormation in your CloudFormation template.**

The option that says: **Set up your Systems Manager State Manager to store the latest AMI IDs and integrate it with your CloudFormation template. Call the update-stack API in CloudFormation whenever you decide to update the EC2 instances in your CloudFormation template** is incorrect because the Systems Manager State Manager service simply automates the process of keeping your Amazon EC2 and hybrid infrastructure in a state that you define. This can't be used as a parameter store that refers to the latest AMI of your application.

The option that says: **Integrate AWS Service Catalog with AWS Config to automatically fetch the latest AMI and use it for succeeding deployments** is incorrect because AWS Service Catalog is not suitable in this scenario since this service just allows organizations to create and manage catalogs of IT services that are approved for use on AWS. In addition, AWS Config is simply a service that enables you to assess, audit, and evaluate the configurations of your AWS resources, which clearly is irrelevant in this case as the developer won't be able to use this to store the latest AMI IDs.

The option that says: **Integrate CloudFormation with AWS Service Catalog to fetch the latest AMI IDs and automatically use them for succeeding deployments** is incorrect because, just as mentioned above, the AWS Service Catalog just allows organizations to create and manage catalogs of IT services that are approved for use on AWS. A more appropriate solution for this scenario would be to use the Systems Manager Parameter Store to retrieve the latest AMI IDs for your template.

References:

<https://aws.amazon.com/blogs/mt/integrating-aws-cloudformation-with-aws-systems-manager-parameter-store/>
<https://docs.aws.amazon.com/systems-manager/latest/userguide/systems-manager-parameter-store.html>

Check out this AWS Systems Manager Cheat Sheet:

<https://tutorialsdojo.com/aws-systems-manager/>

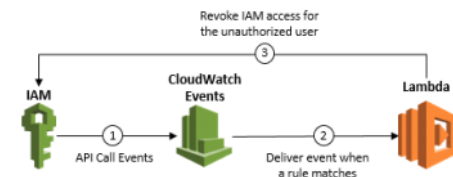
56. Question

A company has developed a Lambda function that will send status updates to a third-party provider for analytics. You need to schedule this function to run every 30 minutes. Which of the following is the MOST manageable and cost-effective way of setting up this task?

- Use the Task Scheduler of your Windows PC to trigger the Lambda function every 30 minutes.
- Launch an EC2 instance that has a cron job that triggers the Lambda function every 30 minutes.
- Integrate Amazon EventBridge (Amazon CloudWatch Events) with Lambda, which will automatically trigger the function every 30 minutes.
- Enable scheduling on the AWS Console of your Lambda function. Define a schedule to run it at 30-minute intervals.

Incorrect

Amazon EventBridge (Amazon CloudWatch Events) helps you respond to state changes in your AWS resources. When your resources change state, they automatically send events into an event stream. You can create rules that match selected events in the stream and route them to your AWS Lambda function to take action. For example, you can automatically invoke an AWS Lambda function to log the state of an EC2 instance or AutoScaling Group. You maintain event source mapping in Amazon CloudWatch Events by using a rule target definition.



You can also create a Lambda function and direct AWS Lambda to execute it on a regular schedule. You can specify a fixed rate (for example, execute a Lambda function every hour or 15 minutes), or you can specify a Cron expression.

Hence, the correct answer is: **Integrate Amazon EventBridge (Amazon CloudWatch Events) with Lambda, which will automatically trigger the function every 30 minutes.**

The option that says: **Launch an EC2 instance that has a cron job that triggers the Lambda function every 30 minutes** is incorrect because provisioning a new instance incurs additional costs. There is also a possibility that the Lambda function will not be invoked in the event that the instance was stopped or terminated.

The option that says: **Use the Task Scheduler of your Windows PC to trigger the Lambda function every 30 minutes** is incorrect because this setup is difficult to manage due to the fact that you are using your own computer to trigger the function. This may be the most cost-effective solution but it certainly is not the most manageable option. The best way is to integrate CloudWatch Events with Lambda.

The option that says: **Enable scheduling on the AWS Console of your Lambda function. Define a schedule to run it at 30-minute intervals** is incorrect because there is no feature like this in Lambda.

References:

<https://docs.aws.amazon.com/AmazonCloudWatch/latest/events/RunLambdaSchedule.html>
<https://docs.aws.amazon.com/lambda/latest/dg/with-scheduled-events.html>

Check out these Amazon CloudWatch and AWS Lambda Cheat Sheets:

<https://tutorialsdojo.com/amazon-cloudwatch/>
<https://tutorialsdojo.com/aws-lambda/>

57. Question

A developer has recently launched a new API Gateway service which is integrated with AWS Lambda. He enabled API caching and per-key cache invalidation features in the API Gateway to comply with the requirement of the front-end development team which will use the API. The front-end team will have to invalidate an existing cache entry in some scenarios and fetch the latest data from the integration endpoint.

Which of the following should the consumers of the API do to invalidate the cache in API Gateway?

- Send a request with the `Cache-Control: max-age=0` header.
- Send a request with the `Cache-Control: INVALIDATE_CACHE` header.
- Send a request with the `Cache-Control: no-cache` header.
- Configure the front-end application to clear the browser cache before fetching data from API Gateway.

Incorrect

A client of your API can invalidate an existing cache entry and reload it from the integration endpoint for individual requests. The client must send a request that contains the `Cache-Control: max-age=0` header. The client receives the response directly from the integration endpoint instead of the cache, provided that the client is authorized to do so. This replaces the existing cache entry with the new response, which is fetched from the integration endpoint.

test Stage Editor

Delete Stage

Invoke URL: <https://b.execute-api.us-east-1.amazonaws.com/test>

Settings Stage Variables SDK Generation Export Deployment History

Configure the metering and caching settings for the **test** stage.

Cache Settings

Cache status: AVAILABLE Flush entire cache

Enable API cache ☒

Enabling API cache increases cost and is not covered by the free tier. [See pricing for more details](#)

Cache capacity: 0.5GB

Encrypt cache data ☒

Cache time-to-live (TTL): 300

Per-key cache invalidation

Require authorization ☒

Handle unauthorized requests Ignore cache control header, Add a warning in response header

CloudWatch Settings

Enable CloudWatch Logs ☐

Ticking the `Require authorization` checkbox ensures that not every client can invalidate the API cache. If most or all of the clients invalidate the API cache, this could significantly increase the latency of your API.

Hence, to only allow authorized clients to invalidate an API Gateway cache entry when submitting API requests, you can just **send a request with the `Cache-Control: max-age=0` header**.

Sending a request with the `Cache-Control: no-cache` header is incorrect because you have to use value of the `max-age` directive in API Gateway instead of the `no-cache` directive. This just forces the cache to submit the request to the origin server for validation before releasing a cached copy.

Configuring the frontend application to clear the browser cache before fetching data from API Gateway is incorrect because the browser cache and the API Gateway cache are not connected with each other. The correct method of invalidating the cache is to add the `Cache-Control: max-age=0` header.

Sending a request with the `Cache-Control: INVALIDATE_CACHE` header is incorrect because there is no directive called `INVALIDATE_CACHE` in the `Cache-Control` header.

References:

<https://docs.aws.amazon.com/apigateway/latest/developerguide/api-gateway-caching.html#invalidate-method-caching>
https://aws.amazon.com/api-gateway/faqs/#Throttling_and_Caching

Check out this Amazon API Gateway Cheat Sheet:

<https://tutorialsdojo.com/amazon-api-gateway/>

58. Question

A serverless application, which uses a Lambda function integrated with API Gateway, provides data to a front-end application written in ReactJS. The users are complaining that they are getting HTTP 504 errors intermittently when they are using the application in peak times. The developer found no errors in the CloudWatch logs of the Lambda function.

Which of the following is the MOST likely cause of this issue?

- There is an authorization failure occurring between API Gateway and the Lambda function.
- The underlying Lambda function has been running for more than 29 seconds causing the API Gateway request to time out.
- The API Gateway automatically enabled throttling in peak times which caused the HTTP 504 errors.
- The memory allocated for the Lambda function is insufficient

Incorrect

A gateway response is identified by a response type defined by API Gateway. The response consists of an HTTP status code, a set of additional headers that are specified by parameter mappings, and a payload that is generated by a non-VTL (Apache Velocity Template Language) mapping template.



You can set up a gateway response for a supported response type at the API level. Whenever API Gateway returns a response of the type, the header mappings and payload mapping templates defined in the gateway response are applied to return the mapped results to the API caller. The following are the Gateway response types which are associated with the HTTP 504 error in API Gateway:

INTEGRATION_FAILURE – The gateway response for an integration failed error. If the response type is unspecified, this response defaults to the `DEFAULT_5XX` type.

INTEGRATION_TIMEOUT – The gateway response for an integration timed-out error. If the response type is unspecified, this response defaults to the `DEFAULT_5XX` type.

For the integration timeout, the range is from 50 milliseconds to 29 seconds for all integration types, including Lambda, Lambda proxy, HTTP, HTTP proxy, and AWS integrations.

In this scenario, there is an issue where the users are getting HTTP 504 errors in the serverless application. This means the Lambda function is working fine at times, but there are instances when it throws an error. Based on this analysis, the most likely cause of the issue is the **INTEGRATION_TIMEOUT** error since you will only get an **INTEGRATION_FAILURE** error if your AWS Lambda integration does not work at all in the first place.

Hence, the correct answer is: **The underlying Lambda function has been running for more than 29 seconds causing the API Gateway request to time out.**

The option that says: **The memory allocated for the Lambda function is insufficient** is incorrect. The fact that no errors were found in the CloudWatch Logs suggests that the function is not the bottleneck.

The option that says: **The API Gateway automatically enabled throttling in peak times which caused the HTTP 504 errors** is incorrect because a large number of incoming requests will most likely produce an HTTP 502 or 429 error but not a 504 error. If executing the function would cause you to exceed a concurrency limit at either the account level (`ConcurrentInvocationLimitExceeded`) or function level (`ReservedFunctionConcurrentInvocationLimitExceeded`), Lambda may return a `TooManyRequestsException` as a response. For functions with a long timeout, your client might be disconnected during synchronous invocation while it waits for a response and returns an HTTP 504 error.

The option that says: **There is an authorization failure occurring between API Gateway and the Lambda function** is incorrect because an authentication issue usually produces HTTP 403 errors and not 504s. The gateway response for authorization failures for missing authentication token errors, invalid AWS signature errors, or Amazon Cognito authentication problems is HTTP 403, which is why this option is unlikely to cause this issue.

References:

<https://docs.aws.amazon.com/apigateway/latest/developerguide/limits.html>
<https://aws.amazon.com/about-aws/whats-new/2017/11/customize-integration-timeouts-in-amazon-api-gateway/>
<https://docs.aws.amazon.com/apigateway/latest/developerguide/supported-gateway-response-types.html>

Check out this Amazon API Gateway Cheat Sheet:

<https://tutorialsdojo.com/amazon-api-gateway/>

59. Question

You are designing the DynamoDB table that will be used by your Node.js application. It will have to handle 10 writes per second and then 20 eventually consistent reads per second where all the items have a size of 2 KB for both operations.

Which of the following are the most optimal WCU and RCU that you should provision to the table?

- 20 RCU and 20 WCU
- 10 RCU and 20 WCU
- 40 RCU and 20 WCU
- 40 RCU and 40 WCU

Incorrect

When you create a new provisioned table in DynamoDB, you must specify its *provisioned throughput capacity*—the amount of read and write activity that the table will be able to support. DynamoDB uses this information to reserve sufficient system resources to meet your throughput requirements.

You can optionally allow DynamoDB auto-scaling to manage your table's throughput capacity. However, you still must provide initial settings for read and write capacity when you create the table. DynamoDB auto scaling uses these initial settings as a starting point and then adjusts them

dynamically in response to your application's requirements. You specify throughput requirements in terms of capacity units—the amount of data your application needs to read or write per second. You can modify these settings later, if needed, or enable DynamoDB auto-scaling to modify them automatically.

1 WCU can do 1 write per second for an item up to 1KB. To get the required WCU, simply multiply the given average item size by the required writes per second. In the scenario, the DynamoDB table is expected to perform 10 writes per second of a 2KB item. Multiplying 10 by 2 gives **20 WCU**.

1 RCU can do 1 strongly consistent read or 2 eventually consistent reads for an item up to 4KB. To get the RCU with eventually consistent reads, do the following steps:

Step #1 Divide the average item size by 4 KB. Round up the result

Average Item Size = 2 KB

= 2KB/4KB

= 0.5 ≈ 1

Step #2 Multiply the number of reads per second by the resulting value from Step 1. Divide the product by 2 for eventually consistent reads.

= 20 reads per second x 1

= **20 RCU**

Since the type of read being asked is eventually consistent, we get half of 20, which is 10.

= 20/2 = **10 RCU**

Hence, the correct answer is to provision **10 RCU and 20 WCU** to your DynamoDB table.

The **20 RCU and 20 WCU** setting is incorrect because this would be the result if you use strong consistency reads. Remember that the scenario explicitly said that eventual consistency reads would be used.

The **40 RCU and 20 WCU** is incorrect because 40 RCU is overkill for the required eventual consistency reads. If the scenario was asking for transactional read requests, then this option could have been correct.

The **40 RCU and 40 WCU** setting is incorrect because this would be the result if you chose transactional requests both on your reads and writes. Take note that the scenario didn't say that the database is using DynamoDB Transactions.

References:

<https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/ProvisionedThroughput.html#ItemSizeCalculations.Writes>

<https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/HowItWorks.ReadWriteCapacityMode.html>

Check out this Amazon DynamoDB Cheat Sheet:

<https://tutorialsdojo.com/amazon-dynamodb/>

Calculating the Required Read and Write Capacity Unit for your DynamoDB Table:

<https://tutorialsdojo.com/calculating-the-required-read-and-write-capacity-unit-for-your-dynamodb-table/>

60. Question

A company is transitioning their systems to AWS due to the limitations of their on-premises data center. As part of this project, a developer was assigned to build a brand new serverless architecture in AWS, which will be composed of AWS Lambda, API Gateway, and DynamoDB in a single stack. She needs a simple and reliable framework that will allow her to share configuration such as memory and timeouts between resources and deploy all related resources together as a single, versioned entity.

Which of the following is the MOST appropriate service that the developer should use in this scenario?

- AWS CloudFormation
- Serverless Application Framework
- AWS Systems Manager
- AWS SAM

Incorrect

The AWS Serverless Application Model (AWS SAM) is an open source framework for building serverless applications. It provides shorthand syntax to express functions, APIs, databases, and event source mappings. You define the application you want with just a few lines per resource and model it using YAML.

AWS SAM is natively supported by AWS CloudFormation and provides a simplified way of defining the Amazon API Gateway APIs, AWS Lambda functions, and Amazon DynamoDB tables needed by your serverless application. During deployment, SAM transforms and expands the SAM syntax into AWS CloudFormation syntax. Then, CloudFormation provisions your resources with reliable deployment capabilities.

Hence, the correct answer is **AWS SAM**.

AWS CloudFormation is incorrect. Although this service can deploy the serverless application to AWS, it is still more appropriate to use AWS SAM instead. AWS SAM can simplify the deployment of the serverless application by deploying all related resources together as a single, versioned entity.

AWS Systems Manager is incorrect because it is more focused on management and operations of AWS resources, such as automation, patching, and configuration, but it is not a deployment or application modeling tool.

Serverless Application Framework is incorrect. Although it is a well-known framework for building and deploying serverless applications into the AWS cloud, this is not an AWS native solution. It also does not allow configuration of DynamoDB databases or API Gateway APIs, unlike AWS SAM.

References:

<https://aws.amazon.com/serverless/sam/>

<https://aws.amazon.com/serverless/developer-tools/>

<https://docs.aws.amazon.com/serverless-application-model/latest/developerguide/what-is-sam.html>

Check out this AWS Serverless Application Model Cheat Sheet:

61. Question

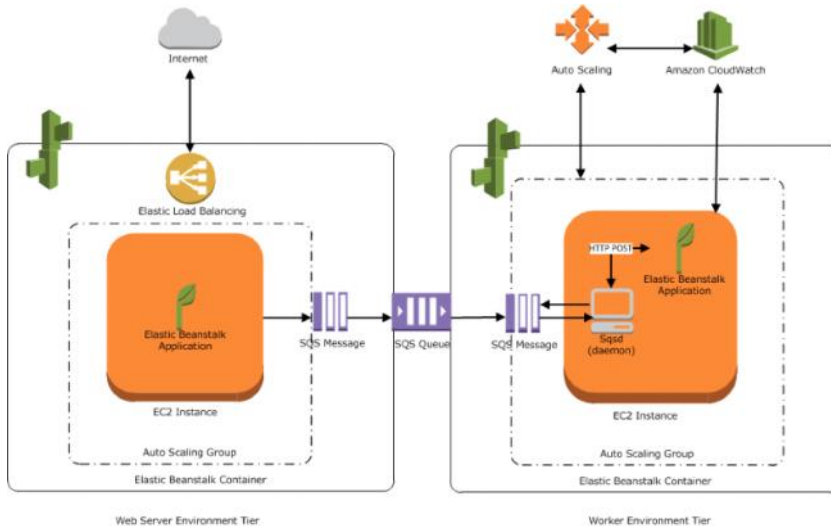
A full-stack developer has developed an application written in Node.js to host an upcoming mobile game tournament. The developer has decided to deploy the application using AWS Elastic Beanstalk because of its ease-of-use. Upon experimenting, he learned that he could configure the webserver environment with several resources.

Which of the following services can the developer configure with Elastic Beanstalk? (Select THREE.)

- AWS Lambda
- Application Load Balancer
- Amazon CloudWatch
- Amazon EC2 Instance
- Amazon CloudFront
- Amazon Athena

Incorrect

AWS Elastic Beanstalk is an easy-to-use service for deploying and scaling web applications and services developed with Java, .NET, PHP, Node.js, Python, Ruby, Go, and Docker on familiar servers such as Apache, Nginx, Passenger, and IIS. You can upload your code and Elastic Beanstalk automatically handles the deployment, from capacity provisioning, load balancing, auto-scaling to application health monitoring. At the same time, you retain full control over the AWS resources powering your application and can access the underlying resources.



With ElasticBeanstalk, you can:

- Select the operating system that matches your application requirements (e.g., Amazon Linux or Windows Server 2016)
- Choose from several Amazon EC2 instances, including On-Demand, Reserved Instances, and Spot Instances.
- Choose from several available database and storage options.
- Enable login access to Amazon EC2 instances for immediate and direct troubleshooting
- Quickly improve application reliability by running in more than one Availability Zone.
- Enhance application security by enabling HTTPS protocol on the load balancer
- Access built-in Amazon CloudWatch monitoring and getting notifications on application health and other important events
- Adjust application server settings (e.g., JVM settings) and pass environment variables
- Run other application components, such as a memory caching service, side-by-side in Amazon EC2.
- Access log files without logging in to the application servers

Hence, the correct answers are: **Amazon EC2 Instance, Amazon CloudWatch, and Application Load Balancer.**

You cannot configure **Amazon Athena, AWS Lambda, and Amazon CloudFront** on ElasticBeanstalk.

References:

- <https://aws.amazon.com/elasticbeanstalk/faqs/>
- <https://aws.amazon.com/elasticbeanstalk/>

Check out this AWS Elastic Beanstalk Cheat Sheet:

<https://tutorialsdojo.com/aws-elastic-beanstalk/>

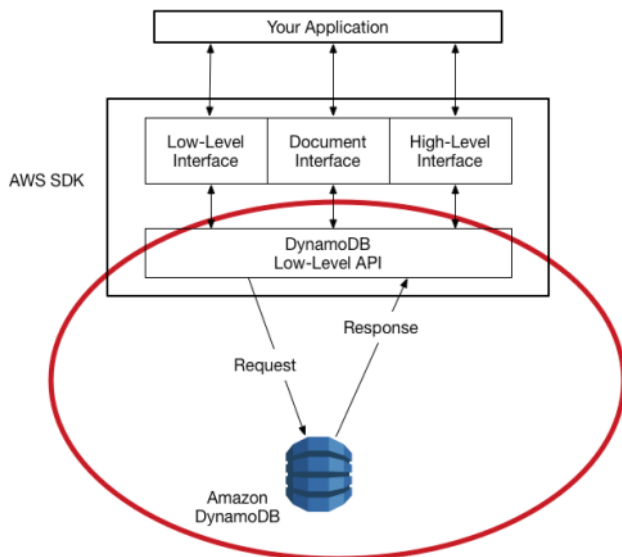
62. Question

A code that runs on a Lambda function performs a `GetItem` call from a DynamoDB table. The function runs three times every week. You noticed that the application kept receiving a `ProvisionedThroughputExceededException` error for 10 seconds most of the time. How should you handle this error?

- Enable DynamoDB Accelerator (DAX) to reduce response times from milliseconds to microseconds.
- Create a Local Secondary Index (LSI) to the existing DynamoDB table to increase the provisioned throughput.
- Reduce the frequency of requests using error retries and exponential backoff.
- Refactor the code in the Lambda function to optimize its performance.

Incorrect

When your program sends a request, DynamoDB attempts to process it. If the request is successful, DynamoDB returns an HTTP success status code (200 OK), along with the results from the requested operation. If the request is unsuccessful, DynamoDB returns an error. An HTTP 400 status code indicates a problem with your request, such as authentication failure, missing required parameters, or exceeding a table's provisioned throughput. You have to fix the issue in your application before submitting the request again.



`ProvisionedThroughputExceededException` means that your request rate is too high. The AWS SDKs for DynamoDB automatically retries requests that receive this exception. Your request is eventually successful unless your retry queue is too large to finish. To handle this error, you can reduce the frequency of requests using error retries and exponential backoff. Hence, the correct answer is: **Reduce the frequency of requests using error retries and exponential backoff.**

The option that says: **Enable DynamoDB Accelerator (DAX) to reduce response times from milliseconds to microseconds** is incorrect because DAX is used to provide a fully managed, in-memory caching solution. This option is not the right way to handle errors due to high request rates.

The option that says: **Refactor the code in the Lambda function to optimize its performance** is incorrect because this will just improve the code's readability and maintainability. This won't have any impact on reducing the frequency of requests.

The option that says: **Create a Local Secondary Index (LSI) to the existing DynamoDB table to increase the provisioned throughput** is incorrect. LSI is used to give flexibility to your queries against the DynamoDB table. LSI uses an alternative sort key aside from the original sort key defined at the creation of the table. Additionally, you cannot create an LSI on an existing table. It can only be added during the creation of a DynamoDB table.

References:

<https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/Programming.Errors.html#Programming.Errors.RetryAndBackoff>
<https://docs.aws.amazon.com/general/latest/gr/api-retries.html>

Check out this Amazon DynamoDB Cheat Sheet:

<https://tutorialsdojo.com/amazon-dynamodb/>

Amazon DynamoDB Overview:

63. Question

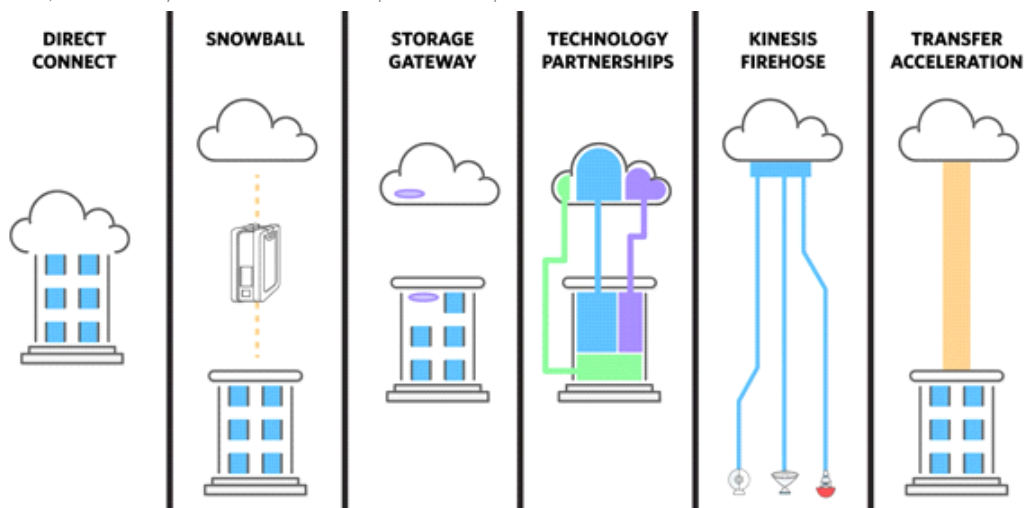
A global financial company has hundreds of users from all over the world who regularly upload terabytes of transactional data to a centralized Amazon S3 bucket. Users from different parts of the globe are experiencing delays in uploading data, which in turn affects processing times. The goal is to improve data throughput and ensure consistently fast data transfer to the S3 bucket regardless of the user's location.

Which of the following should be used to satisfy the above requirement?

- S3 Transfer Acceleration
- AWS Transfer for SFTP
- Amazon CloudFront
- AWS Direct Connect

Incorrect

Amazon S3 Transfer Acceleration enables fast, easy, and secure transfers of files over long distances between your client and your Amazon S3 bucket. Transfer Acceleration leverages Amazon CloudFront's globally distributed AWS Edge Locations. As data arrives at an AWS Edge Location, data is routed to your Amazon S3 bucket over an optimized network path.



Hence, the correct answer is: **S3 Transfer Acceleration.**

AWS Transfer for SFTP is incorrect because this is just a fully managed service that enables the transfer of files directly into and out of Amazon S3 using the Secure File Transfer Protocol (SFTP) which is also known as Secure Shell (SSH) File Transfer Protocol. It does not provide a fast, easy,

and secure way to transfer files over long distances between your client and your Amazon S3 bucket.

AWS Direct Connect is incorrect because you have users all around the world and not just on your on-premises data center. Direct Connect would be too costly and is definitely not suitable for this purpose.

Amazon CloudFront is incorrect because this service is primarily used to serve static content and not as a transfer accelerator going to or from Amazon S3. CloudFront is a fast content delivery network (CDN) service that securely delivers data, videos, applications, and APIs to customers globally with low latency and high transfer speeds.

References:

<http://docs.aws.amazon.com/AmazonS3/latest/dev/transfer-acceleration.html>

<https://docs.aws.amazon.com/AmazonS3/latest/dev/transfer-acceleration-examples.html>

Check out this Amazon S3 Cheat Sheet:

<https://tutorialsdojo.com/amazon-s3/>

Comparison of AWS Services Cheat Sheets:

<https://tutorialsdojo.com/comparison-of-aws-services/>

64. Question

You are developing an application that will use a Lambda function, which will be invoked asynchronously. The application will be implemented with exponential back-off that will handle failures so that the requests will be retried twice before the event is discarded. If the retries fail with an unexpected error, you have to direct unprocessed events to another service which will analyze the failure.

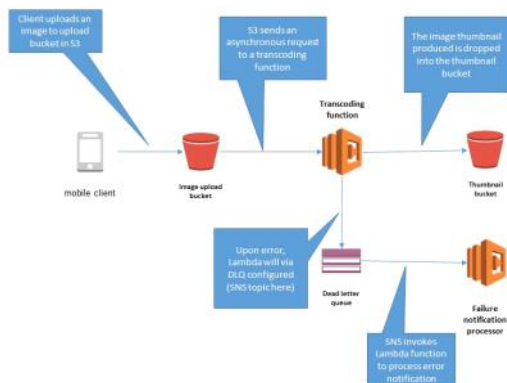
Which of the following is the MOST suitable component that you should implement in the application architecture to meet the above requirement?

- Delay Queue
- Dead Letter Queue
- FIFO Queue
- Amazon MQ

Incorrect

Function invocation can result in an error for several reasons. Your code might raise an exception, time out, or run out of memory. The runtime executing your code might encounter an error and stop. You might run out of concurrency and be throttled.

When an error occurs, your code might have run completely, partially, or not at all. In most cases, the client or service that invokes your function retries if it encounters an error, so your code must be able to process the same event repeatedly without unwanted effects. If your function manages resources or writes to a database, you need to handle cases where the same request is made several times.



AWS Lambda directs events that cannot be processed to the specified Amazon SNS topic or Amazon SQS queue. Functions that don't specify a DLQ will discard events after they have exhausted their retries. You configure a DLQ by specifying the Amazon Resource Name *TargetArn* value on the Lambda function's *DeadLetterConfig* parameter.

Hence, the setting up a **Dead Letter Queue** is the correct answer in this scenario.

Delay Queue is incorrect because this just lets you postpone the delivery of new messages to a queue for a number of seconds. This is not relevant in this scenario since you can't use a delay queue within Lambda.

FIFO Queue is incorrect because this is primarily used to enhance messaging between applications when the order of operations and events is critical, or where duplicates can't be tolerated. Although a DLQ is just a normal SQS queue, this option is still incorrect because you don't necessarily need a FIFO SQS queue since you can also use a Standard SQS queue to be the Dead Letter Queue of your Lambda function.

Amazon MQ is incorrect because this is simply a managed message broker service for Apache ActiveMQ that makes it easy to set up and operate message brokers in the cloud.

References:

<https://docs.aws.amazon.com/lambda/latest/dg/dlq.html>

<https://docs.aws.amazon.com/lambda/latest/dg/retries-on-errors.html>

Check out this AWS Lambda Cheat Sheet:

<https://tutorialsdojo.com/aws-lambda/>

65. Question

A company uses AWS Systems Manager (SSM) Parameter Store to manage configuration details for multiple applications. The parameters are currently stored in the Standard tier. The company wants its operations team to be notified if there are sensitive parameters that haven't been rotated within 90 days.

Which must be done to meet the requirement?

- Configure a *NoChangeNotification* policy with a value of 90 days. Use Amazon EventBridge (Amazon CloudWatch Events) to send a notification via Amazon SNS.
- Set up an Amazon EventBridge (Amazon CloudWatch Events) event pattern that captures SSM Parameter-related events. Use Amazon SNS to send notifications.
- Convert the sensitive parameters from Standard tier into Advanced tier. Set a *NoChangeNotification* policy with a value of 90 days. Use Amazon

EventBridge (Amazon CloudWatch Events) to send a notification via Amazon SNS.

- Convert the sensitive parameters from Standard tier into Advanced tier. Set a `ExpirationNotification` policy with a value of 90 days. Use Amazon EventBridge (Amazon CloudWatch Events) to send a notification via Amazon SNS.

Incorrect

Parameter policies help you manage a growing set of parameters by allowing you to assign specific criteria to a parameter, such as an expiration date or time to live. Parameter policies are especially helpful in forcing you to update or delete passwords and configuration data stored in Parameter Store, a capability of AWS Systems Manager. Take note that parameter policies are only available for parameters in the **Advanced tier**.

Parameter Store offers the following types of policies:

Expiration – deletes the parameter at a specific date

ExpirationNotification – sends an event to Amazon EventBridge (Amazon CloudWatch Events) when the specified expiration time is reached.

NoChangeNotification – sends an event to Amazon EventBridge (Amazon CloudWatch Events) when a parameter has not been modified for a specified period of time.

Parameter policies — Optional

Parameter expiration
Specify how long the parameter is available in Parameter Store, as either a point-in-time or lifetime. The parameter is deleted after the specified date.

☒ No expiration policy
☐ Absolute time
☐ Relative time

Notification policies
Create notification policies to send notifications from CloudWatch Events. Notifications can be based on when this parameter expires, or if the parameter has not changed for a given period of time.

Policy type	Policy value	Value type	
Notify if no change h...	90	Days	Remove policy

[Add policy](#)

[Cancel](#) [Save changes](#)

The **NoChangeNotification** policy sends a notification based on the `LastModifiedTime` attribute of the parameter. If you change or edit a parameter, the system resets the notification time period based on the new value of `LastModifiedTime`. In the scenario's case, we want to be notified if specific parameters were not rotated in the last 90 days.

In the scenario, the goal is to be notified if specific sensitive parameters have not been rotated within the past 90 days. Configuring the **NoChangeNotification** policy with a value of 90 days allows SSM to emit a notification to EventBridge whenever the `LastModifiedTime` of the sensitive parameters exceeds the specified time frame. However, setting the notification policy alone is not enough. You must configure Amazon EventBridge (Amazon CloudWatch Events) to capture the emitted events and route them to an Amazon SNS topic.

Hence, the correct answer is: **Convert the sensitive parameters from Standard tier into Advanced tier. Set a NoChangeNotification policy with a value of 90 days. Use Amazon EventBridge (Amazon CloudWatch Events) to send a notification via Amazon SNS.**

The option that says: **Configure a NoChangeNotification policy with a value of 90 days. Use Amazon EventBridge (Amazon CloudWatch Events) to send a notification via Amazon SNS** is incorrect because notification policies are not supported in the Standard tier. You must convert the parameters first into the Advanced tier.

The option that says: **Convert the sensitive parameters from Standard tier into Advanced tier. Set a ExpirationNotification policy with a value of 90 days. Use Amazon EventBridge (Amazon CloudWatch Events) to send a notification via Amazon SNS** is incorrect because the `ExpirationNotification` policy is for notifying when a parameter is about to expire, not when it hasn't been rotated. In this case, the `NoChangeNotification` policy should be used instead.

The option that says: **Set up an Amazon EventBridge (Amazon CloudWatch Events) event pattern that captures SSM Parameter-related events. Use Amazon SNS to send notifications** is incorrect. A notification policy must be enabled as well, otherwise, Amazon EventBridge (Amazon CloudWatch Events) won't be able to receive any notifications.

References:

<https://docs.aws.amazon.com/systems-manager/latest/userguide/parameter-store-policies.html>

[https://aws.amazon.com/about-aws/whats-](https://aws.amazon.com/about-aws/whats-new/2019/04/aws-systems-manager-parameter-store-introduces-advanced-parameters/)

[new/2019/04/aws-systems-manager-parameter-store-introduces-advanced-parameters/](https://aws.amazon.com/about-aws/whats-new/2019/04/aws-systems-manager-parameter-store-introduces-advanced-parameters/)

Check out this cheat sheet on AWS Secrets Manager vs Systems Manager Parameter Store:

<https://tutorialsdojo.com/aws-secrets-manager-vs-systems-manager-parameter-store/>

From <<https://portal.tutorialsdojo.com/courses/aws-certified-developer-associate-practice-exams/lessons/randomized-test-6/quizzes/randomized-test-aws-certified-developer-associate/>>